BÁO CÁO BÀI TẬP LỚN AHR-IDPS

HỆ THỐNG GIÁM SÁT XÂM NHẬP LAI VÀ SINH QUY TẮC BẰNG GENTI

Báo cáo hoàn chỉnh về hệ thống AHR-IDPS

Phân tích thiết kế, triển khai, dataset, huấn luyện, kiểm thử và đánh giá

Môn học An toàn ứng dụng Web

Đối tượng thực nghiệm DVWA trong môi trường Docker cục bộ

Hoàn thiện ngày 09 tháng 11 năm 2026

Báo cáo trình bày giới thiệu, cơ sở lý thuyết, phân tích thiết kế, triển khai, cơ sở toán học GenTI và OOF, dataset, huấn luyện và kết quả đánh giá của AHR-IDPS. Các phần chỉnh sửa của người thực hiện được giữ làm nội dung chính; dữ liệu chưa đo được phân biệt với kết quả đã có.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI....3
1.1. Bối cảnh và vấn đề nghiên cứu....3
1.2. Mục tiêu và câu hỏi nghiên cứu....4
1.3. Đối tượng phạm vi và điều kiện thực nghiệm....5
1.4. Phương pháp nghiên cứu và kiểm chứng....6
1.5. Đóng góp và tổ chức báo cáo....6
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ....8
2.1. An ninh ứng dụng web và môi trường DVWA....8
2.2. Cơ chế IDS IPS và phát hiện theo quy tắc....10
2.3. Học máy tương quan sự kiện và biểu diễn graph....11
2.4. Kiến trúc telemetry SIEM và bảo toàn evidence....13
2.5. Cơ sở đánh giá chất lượng và hiệu năng....13
CHƯƠNG 3. PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG....15
3.1. Yêu cầu hệ thống mô hình đe dọa và giả định....15
3.2. Kiến trúc tổng thể và trách nhiệm các thành phần....19
3.3. Luồng xử lý dữ liệu và hình thành quyết định....23
3.4. Mô hình dữ liệu nhãn và khả năng truy vết....25
3.5. Vòng đời quy tắc và kiểm soát tác động....26
CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG....29
4.1. Nền tảng triển khai và môi trường ứng dụng web....29
4.2. Cảm biến mạng runtime và tích hợp Wazuh....30
4.3. Pipeline dữ liệu backend và dashboard....43
4.4. Tích hợp mô hình AI và tự động sinh quy tắc....44
4.5. Enforcement IPS evidence và điều khiển vận hành....51
CHƯƠNG 5. CƠ SỞ TOÁN HỌC VÀ THUẬT TOÁN GENTI OOF....54
5.1. Mô hình hóa bài toán và hợp đồng đầu ra....54

5.2. Tối ưu SFT cập nhật QLoRA và chương trình học 55
5.3. Thuật toán GenTI nguyên bản và áp dụng trong hệ thống 57
5.4. OOF Fusion và huấn luyện dự đoán ngoài nhóm 65
5.5. Chỉ số đánh giá quan hệ thuật toán và giới hạn 68
CHƯƠNG 6. DATASET VÀ HUẤN LUYỆN MÔ HÌNH AI 70
6.1. Thiết kế dataset và phân biệt các mục tiêu học 70
6.2. Thu thập chuẩn hóa chia tập và mở rộng dữ liệu.....72
6.3. Huấn luyện đăng ký và tái lập mô hình phát hiện 76
6.4. Corpus GenTI hợp đồng tác vụ và phân tích chất lượng.....80
6.5. Fine tune Qwen và lựa chọn checkpoint.....88
CHƯƠNG 7. KẾT QUẢ VÀ ĐÁNH GIÁ 93
7.1. Kết quả triển khai và mức bao phủ giám sát 93
7.2. Đánh giá phân loại hiệu năng và phương pháp hybrid.....95
7.3. Kiểm chứng tổng quát hóa phản ứng và sai sót.....101
7.4. Đánh giá GenTI từ cú pháp đến phát hiện trên PCAP.....105
7.5. Mức đáp ứng mục tiêu và hướng phát triển.....111
TÀI LIỆU THAM KHẢO VÀ NGUỒN THỰC NGHIỆM.....114

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. Bối cảnh và vấn đề nghiên cứu

Bài toán được xem xét từ nhu cầu giám sát web đến những khoảng trống của dữ liệu, phát hiện và phản ứng. Các vấn đề này liên hệ trực tiếp với nhau: dữ liệu thiếu truy vết sẽ làm cả quyết định AI lẫn bằng chứng ngăn chặn khó kiểm chứng.

Ứng dụng web tiếp nhận dữ liệu từ người dùng qua nhiều đường dẫn, tham số và phiên truy cập. Cùng một endpoint có thể phục vụ thao tác hợp lệ và nhận các chuỗi có ý định khai thác. Trong môi trường này, quan sát riêng một status HTTP hoặc tên trang không đủ để kết luận hoạt động độc hại. Người quản trị cần liên kết nội dung request với cảm biến, ngữ cảnh phiên và tác động lên ứng dụng để đưa ra quyết định có thể giải thích.

Hệ thống phát hiện dựa trên luật có ưu điểm là điều kiện đối sánh rõ ràng và có thể kiểm tra bằng engine. Hạn chế xuất hiện khi hành vi mới chưa có chữ ký thích hợp hoặc rule quá hẹp so với các biến thể. Các mô hình học máy có thể bổ sung tín hiệu từ đặc trưng mạng, log và quan hệ sự kiện, nhưng một điểm bất thường cao vẫn cần được diễn giải. Nếu phản ứng tự động dựa trên một tín hiệu chưa được kiểm chứng, hệ thống có thể làm gián đoạn truy cập hợp lệ.

Mô hình ngôn ngữ mở ra khả năng chuyển yêu cầu phát hiện và bằng chứng thành một rule cụ thể. Tuy nhiên, đầu ra có hình thức đúng chưa bảo đảm engine chấp nhận, khớp đúng attack hoặc tránh benign. AHR-IDPS được xây dựng để nghiên cứu khoảng nối này: thu nhận bằng chứng, sử dụng mô hình fine-tune sinh đề xuất, kiểm định độc lập rồi áp dụng có điều kiện trong lab. Động lực của đề tài là làm cho quá trình đó có thể quan sát, tái lập và kiểm chứng từ đầu đến cuối.

Vấn đề cần giải quyết. Vấn đề thứ nhất là dữ liệu phân tán giữa các nguồn. Nginx quan sát request và response, Suricata quan sát mạng, Falco quan sát hành vi runtime, còn Wazuh phân tích log theo luật SIEM. Một hoạt động có thể tạo nhiều record với cấu trúc và thời gian khác nhau. Nếu không chuẩn hóa và giữ định danh, người vận hành khó xác định một alert thuộc request nào và khó xây dataset không bị đếm lặp.

Vấn đề thứ hai là phân biệt phát hiện, sinh rule và ngăn chặn. Một model trả nhãn attack không tự tạo ra rule; một rule được lưu vào registry không tự có hiệu lực trên

đường request; một audit phản ứng không tự chứng minh lưu lượng đã bị chặn. Báo cáo cần chỉ ra bằng chứng cụ thể cho từng chức năng, tránh đồng nhất những trạng thái có tên tương tự nhưng tác động khác nhau.

Vấn đề thứ ba là chất lượng dữ liệu và đánh giá. Dữ liệu event phục vụ phân loại khác với cặp yêu cầu–evidence–benign–rule phục vụ SFT. Các mẫu tổng hợp hoặc paraphrase có tương quan không thể được xem như những capture độc lập. Đối với GenTI, tỷ lệ đạt JSON, tỷ lệ đạt cú pháp và tỷ lệ phát hiện trên PCAP trả lời những câu hỏi khác nhau. Hệ thống phải giữ mẫu số và provenance để các kết quả không bị diễn giải vượt quá dữ liệu.

1.2. Mục tiêu và câu hỏi nghiên cứu

Mục tiêu tổng quát được cụ thể hóa thành chức năng và câu hỏi có thể kiểm tra bằng thực nghiệm. Cách tổ chức này giúp nối kết yêu cầu xây dựng hệ thống với bằng chứng cần có, thay vì đánh giá thành công chỉ qua số lượng tính năng.

Mục tiêu của đề tài là xây dựng một hệ thống giám sát và phản ứng xâm nhập lai cho ứng dụng DVWA trong môi trường Docker cục bộ, kết hợp cảm biến dựa trên luật, các nhánh AI phân loại và LLM fine-tune sinh quy tắc theo hướng GenTI. Hệ thống phải liên kết được nguồn dữ liệu, đầu ra model, kết quả kiểm định và tác động thực thi, đồng thời cung cấp giao diện để theo dõi toàn bộ chu trình.

Trong triển khai chính đã có bằng chứng, tự động hóa được giới hạn ở một profile HTTP GET XSS cụ thể tại gateway. Việc giữ giới hạn này là một phần của thiết kế nghiên cứu: đầu ra nằm ngoài profile vẫn được lưu để xem xét nhưng không mặc nhiên được quyền chặn. Mục tiêu không phải tuyên bố phòng vệ mọi kiểu tấn công, mà là chứng minh một quy trình có kiểm soát và xác định rõ cách mở rộng nó.

Mục tiêu cụ thể. Các mục tiêu chức năng gồm thu nhận đa nguồn, chuẩn hóa sự kiện, lưu dữ liệu truy vấn được, tương quan thành incident, hiển thị dashboard và giữ evidence. Đối với phần mô hình, đề tài xây dữ liệu phân loại, huấn luyện các nhánh Network, Log, Graph và Fusion, đồng thời chuẩn bị corpus riêng để fine-tune Qwen phục vụ sinh rule. Các artifact phải có định danh và metadata phù hợp với bài toán tương ứng.

Đối với vận hành tự động, hệ thống cần tự chọn quan sát, phân vai evidence và phản ví dụ, tạo Candidate, chạy kiểm định native, quan sát Shadow và chuyển Active

khi đủ điều kiện. Request bị chặn phải được đối chiếu với rule hash, audit và dấu vết trước upstream. Hệ thống cũng cần cách dừng, vô hiệu hóa và hết hạn thực thi để hỗ trợ xử lý ngoại lệ.

Đối với đánh giá, báo cáo công bố kết quả theo đơn vị event, task, rule và case PCAP, nêu rõ dữ liệu đã chạy và phép thử còn thiếu. Chỉ tiêu kỹ thuật phải gắn với cách đo. Những đại lượng chưa có log đầy đủ như p95 latency dưới tải hoặc FPR ngoài thực địa được giữ ở trạng thái cần nghiệm thu tiếp.

Câu hỏi nghiên cứu. Đề tài đặt ra bốn nhóm câu hỏi. Thứ nhất, dữ liệu từ cảm biến có được đưa vào một pipeline thống nhất và nối về hoạt động nguồn không? Thứ hai, mô hình đã fine-tune có sinh đầu ra đúng hợp đồng và được engine đích chấp nhận không? Thứ ba, quy trình native–Shadow–Active có vận hành tự động theo các điều kiện đo được không? Thứ tư, request bị chặn có thực sự dừng trước DVWA và có thể truy ngược về rule do model sinh không?

Một câu hỏi bổ sung liên quan tới phương pháp học: các nhánh phân loại và OOF Fusion có mang lại chất lượng trên cùng tập đánh giá ra sao, và kết quả đó có giới hạn nào khi đưa sang nguồn khác? Câu hỏi này được tách khỏi chất lượng rule GenTI vì hai bài toán có đầu vào, hàm mục tiêu và đầu ra khác nhau. Chương toán trình bày sự khác nhau này trước khi chương dataset mô tả các tập dữ liệu thực tế.

1.3. Đối tượng phạm vi và điều kiện thực nghiệm

Phạm vi bao gồm cả tài sản được bảo vệ, vị trí cảm biến và đường thực thi. Những giới hạn về HTTP, môi trường lab và kết nối máy ảo quyết định kết quả nào có thể được công bố và kết quả nào còn cần nghiệm thu.

Đối tượng thực nghiệm là DVWA, gateway Nginx, các container dữ liệu và cảm biến trong lab do người thực hiện quản lý. Đường vào được nghiệm thu sử dụng loopback của Windows, cùng Docker Linux Engine và WSL. Máy ảo Kali trên VMware là phương tiện kiểm thử tùy chọn; khả năng đi từ Kali tới gateway và vị trí cảm biến cần được kiểm riêng trước khi dùng trong một kết luận thực nghiệm.

Phạm vi nghiên cứu bao gồm giám sát HTTP, log và runtime theo cấu hình đã có; phân loại offline; fine-tune LLM; kiểm định quy tắc; và enforcement tại gateway cho grammar được cho phép. Phạm vi hiện chưa tương đương một IPS inline toàn

mạng. Những khả năng như phân tích POST body, TLS ngoài gateway, phản ứng firewall trên host và cảnh báo ra kênh bên ngoài đòi hỏi triển khai và nghiệm thu riêng.

Các hoạt động kiểm thử chỉ tác động mục tiêu lab được phép. Tính an toàn và tính đúng của đánh giá được bảo đảm bằng cách giới hạn lưu lượng, giữ bản ghi đầu ra và không suy rộng kết quả của chữ ký hẹp thành khả năng phòng vệ tổng quát. Đây cũng là điều kiện để người khác có thể tái lập kết quả mà không cần thay đổi mạng ngoài phạm vi.

1.4. Phương pháp nghiên cứu và kiểm chứng

Phương pháp được triển khai theo chuỗi từ thiết kế đến dữ liệu, huấn luyện và phép thử đầu cuối. Tại mỗi bước, cấu hình và artifact được giữ để có thể đối chiếu nguyên nhân khi kết quả khác kỳ vọng.

Đề tài kết hợp phân tích yêu cầu, thiết kế kiến trúc, xây dựng nguyên mẫu và thực nghiệm có bằng chứng. Ở bước thiết kế, từng node được xác định theo đầu vào, xử lý, đầu ra và ranh giới tin cậy. Ở bước triển khai, các chức năng được kết nối qua schema và API. Ở bước đánh giá, kết quả được kiểm từ artifact thay vì chỉ từ trạng thái hiển thị trên giao diện.

Đối với học máy, phương pháp gồm làm sạch dữ liệu, tách nhóm để giảm leakage, huấn luyện các baseline và mô hình tổng hợp, lưu confusion count cùng cấu hình. Đối với LLM, phương pháp gồm corpus SFT có provenance, QLoRA, chương trình học theo độ khó, chọn checkpoint bằng validation và giữ raw output để kiểm độc lập. Công thức toán được dùng để làm rõ mục tiêu tối ưu và các điều kiện chuyển trạng thái.

Đối với kiểm chứng hệ thống, đề tài dùng native engine, PCAP theo task, request live qua gateway và audit liên kết. Kiểm thử thành phần được phân biệt với kiểm thử tích hợp và đánh giá tổng quát hóa. Những số liệu trong các chương sau được giữ cùng thời điểm nguồn; một ảnh đã lưu không được dùng để xác nhận trạng thái live ở một ngày khác.

1.5. Đóng góp và tổ chức báo cáo

Đóng góp được đọc cùng cấu trúc báo cáo để xác định nơi trình bày thiết kế, mô hình toán và bằng chứng thực nghiệm. Việc tách các phần này giúp làm rõ giá trị đã đạt và những hướng phát triển chưa có số liệu.

Đóng góp thứ nhất là một pipeline quan sát đa nguồn có khả năng truy vết từ request đến sự kiện chuẩn và incident. Đóng góp thứ hai là việc đưa model LLM do người thực hiện fine-tune vào ứng dụng, giữ đúng đầu ra và identity thay vì thay bằng rule viết sẵn. Đóng góp thứ ba là chu trình kiểm định và triển khai có điều kiện, trong đó trạng thái registry được nối với điểm thực thi HTTP có bằng chứng.

Đề tài còn cung cấp bộ artifact phân loại, corpus sinh rule, hồ sơ loss, kiểm tra syntax, kiểm thử parity và hướng dẫn tái lập. Giá trị của các artifact nằm ở việc người đọc có thể kiểm lại mẫu số và phạm vi. Những hạn chế về nguồn tổng hợp, human review, capture độc lập và hiệu năng dưới tải được công bố để làm cơ sở cho nghiên cứu tiếp theo.

Cấu trúc báo cáo. Báo cáo gồm bảy chương. Chương 1 giới thiệu bài toán, mục tiêu và phạm vi. Chương 2 trình bày cơ sở lý thuyết và công nghệ. Chương 3 phân tích yêu cầu, kiến trúc và luồng quyết định. Chương 4 trình bày triển khai. Chương 5 xây dựng mô hình toán và thuật toán GenTI, OOF. Chương 6 mô tả dataset, phân tích dữ liệu và huấn luyện. Chương 7 trình bày kết quả, diễn giải và giới hạn đánh giá.

Kịch bản quay video được cung cấp trong tài liệu thao tác riêng. Tài liệu đó giúp kiểm chứng từng bước bằng trạng thái hệ thống và request có thể quan sát; nó không thay thế số liệu đo trong báo cáo. Khi quay lại sau một lần khởi động mới, người thực hiện cần tạo evidence của phiên mới và không dùng ảnh cũ để minh họa như một kết quả vừa chạy.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1. An ninh ứng dụng web và môi trường DVWA

Các khái niệm tấn công cần được đặt trong điều kiện ứng dụng và phiên truy cập cụ thể. Môi trường DVWA cho phép kiểm soát đầu vào, nhưng mức bảo mật, session và vị trí thu dữ liệu vẫn ảnh hưởng cách diễn giải tín hiệu quan sát.

Khái niệm lỗ hổng web. Lỗ hổng web là một điểm yếu trong thiết kế, triển khai hoặc cấu hình cho phép tác động ngoài ý định lên dữ liệu và chức năng ứng dụng. Một chỉ báo ở request có thể phù hợp với kỹ thuật khai thác nhưng chưa chứng minh lỗ hổng đã được khai thác thành công. Phát hiện payload, phát hiện hành vi bất thường và xác nhận tác động là các mức quan sát cần được phân biệt.

Các phân loại rủi ro như OWASP Top 10 giúp tổ chức kiến thức và phạm vi kiểm thử, nhưng không phải danh sách chữ ký đầy đủ cho sensor. Trong AHR-IDPS, mỗi phép thử phải được cụ thể hóa bằng trường quan sát, endpoint, phiên và expected result. Việc dùng tên một nhóm rủi ro trong dashboard không thay cho rule hoặc oracle có thể kiểm tra. [12]

SQL Injection XSS Command Injection File Inclusion và Authentication Abuse. SQL Injection liên quan tới dữ liệu không tin cậy làm thay đổi ý nghĩa truy vấn khi ứng dụng xử lý không đúng. XSS liên quan tới nội dung không tin cậy được trình duyệt xử lý như mã trong ngữ cảnh phù hợp. Command Injection có thể làm dữ liệu đầu vào ảnh hưởng lệnh hệ điều hành; File Inclusion hoặc file access bất thường liên quan tới lựa chọn tài nguyên ngoài phạm vi mong muốn. Authentication Abuse khai thác luồng xác thực hoặc chuỗi thử đăng nhập theo cách cần kiểm soát.

Các nhóm này có visibility khác nhau. Chuỗi URI có thể nhìn thấy ở gateway và sensor HTTP, còn tác động process có thể cần cảm biến runtime. Một dấu hiệu trong query không bảo đảm có syscall đáng chú ý, và một runtime alert không tự xác định request web nguyên nhân. Vì vậy thiết kế hybrid kết hợp bằng chứng nhưng vẫn giữ nguồn và điều kiện của từng sensor.

Trong profile IPS chính, chữ ký được kiểm chứng chỉ gồm method GET, prefix đường dẫn XSS và content literal sau chuẩn hóa URI. Các dạng XSS khác, method khác hoặc payload ở body chưa nằm trong phạm vi đó. Bài toán mở rộng không chỉ là thêm tên tấn công; nó cần một grammar, dữ liệu và điểm thực thi tương ứng.

Nguyên tắc kiểm thử có ủy quyền. Một phép thử phải xác định trước mục tiêu, địa chỉ, tài khoản lab, loại thao tác, giới hạn và dữ liệu cần lưu. Chỉ số tấn công được tạo trong lab cần có marker hoặc lịch thao tác để truy vết, nhưng marker đó phải được giữ ngoài đặc trưng học. Khi thử xong, cần kiểm khả năng trở về trạng thái đã định và giữ evidence của cả lượt lỗi.

Môi trường cô lập giúp giới hạn tác động nhưng không tự loại mọi sai số phương pháp. Ví dụ capture sau gateway có thể không chứa request đã bị chặn, dẫn tới selection bias. Vì vậy điều kiện mạng và vị trí thu là một phần của thiết kế kiểm thử, không phải chi tiết có thể bỏ qua khi báo cáo kết quả.

Ứng dụng DVWA và môi trường lab. Mục đích của DVWA. DVWA là ứng dụng được dùng để học và kiểm thử các tình huống web trong một môi trường được kiểm soát. Trong đề tài, DVWA đóng vai trò mục tiêu để tạo request hợp lệ, request có ý định khai thác và dữ liệu cảm biến có thể lặp lại. MariaDB hỗ trợ dữ liệu ứng dụng, còn Nginx cung cấp một điểm vào chung cho quan sát và kiểm tra trước upstream. [13]

Tính dễ tái lập của lab là lợi thế cho kiểm thử tích hợp, nhưng cũng giới hạn tính đại diện. Template, endpoint và cấu trúc traffic có thể lặp nhiều lần. Vì vậy chất lượng cao trên DVWA cần được kiểm tiếp trên nguồn hoặc ứng dụng khác trước khi suy thành hiệu quả ngoài môi trường nghiên cứu.

Mức bảo mật và giới hạn lab. Mức bảo mật của DVWA ảnh hưởng cách ứng dụng xử lý input và mức thành công của một thao tác khai thác. Mọi phép so sánh cần ghi mức cấu hình, trạng thái khởi tạo và bộ dữ liệu. Nếu thay mức giữa các lần đo mà không lưu lại, một khác biệt response có thể bị gán nhầm cho IDS hoặc IPS.

Gateway bind loopback giới hạn đường truy cập đã nghiệm thu. Một VM sử dụng loopback của chính nó không nhìn thấy dịch vụ loopback trên Windows. Khi thêm Kali, cần xác minh mạng lab và port binding có chủ đích, đồng thời giữ DVWA khỏi mạng không thuộc phạm vi. Video hướng dẫn sử dụng Windows và loopback làm đường chính để không phụ thuộc một kết nối VM chưa được đo.

Đăng nhập và session. Phiên đăng nhập quyết định request có đến đúng chức năng DVWA hay chỉ bị chuyển về login. Một response 302 trong bước chuẩn bị có thể phản ánh thiếu session, không phải chặn tấn công. Khi quay video, cần đăng nhập trước và kiểm một benign request ở cùng endpoint để xác nhận đường thao tác.

Cookie và token là dữ liệu nhạy cảm cần tránh xuất công khai trong ảnh hoặc file quay. Khi đo HTTP, có thể quan sát status và một số header bằng công cụ trình duyệt nhưng không cần trình chiếu giá trị xác thực. Thông tin phiên phục vụ tái lập được quản lý riêng khỏi evidence dùng chia sẻ.

2.2. Cơ chế IDS IPS và phát hiện theo quy tắc

Phát hiện, sinh quy tắc và thực thi phản ứng được phân biệt theo chức năng và bằng chứng. Ngôn ngữ rule, semantics engine và quy trình kiểm định tạo cơ sở để chuyển một đề xuất thành tác động có kiểm soát.

Phát hiện phân loại và phản ứng. IDS quan sát dữ liệu và tạo tín hiệu khi phát hiện điều kiện hoặc độ bất thường. IPS có một điểm thực thi ảnh hưởng đường request hay lưu lượng. Hệ thống phản ứng sử dụng bằng chứng và chính sách để lựa chọn hành động, có thể chỉ ghi nhận hoặc có thể ngăn chặn. Tên service không quyết định vai trò; bằng chứng về đường xử lý mới quyết định chức năng đã thực hiện.

Trong AHR-IDPS, Suricata, Falco và Wazuh cung cấp phát hiện theo cấu hình của mình. Gateway Nginx cùng checker thực hiện đường IPS HTTP đã kiểm chứng. LLM sinh nội dung quy tắc và không trực tiếp quyết định thay đổi host firewall. Cách tách vai trò này giúp xác định nơi cần kiểm thử khi một request đi qua hoặc bị từ chối.

Signature anomaly và hybrid. Phát hiện theo signature có điều kiện rõ, thuận tiện đối chiếu bằng test case nhưng phụ thuộc độ phủ chữ ký. Phát hiện bất thường học từ dữ liệu và có thể nhận ra khác biệt với phân bố đã học, song độ bất thường không tự đồng nghĩa độc hại. Hybrid kết hợp nguồn tín hiệu để cân đối coverage và báo nhầm, nhưng không bảo đảm tốt hơn khi các nguồn cùng mắc một sai lệch dữ liệu.

Do đó cần baseline trên cùng tập, giữ ngưỡng và phân tích từng nhóm lỗi. Một nhánh có recall cao nhưng FPR lớn có thể không thích hợp để tự chặn. Một nhánh precision cao nhưng chỉ bắt rất ít attack cũng chưa đủ. Quyết định triển khai phải dựa trên mục tiêu và chi phí lỗi của phạm vi được bảo vệ.

Phát hiện dựa trên luật. Luật Suricata Falco và Wazuh. Rule Suricata mô tả điều kiện trên traffic và các buffer ứng dụng theo ngôn ngữ của engine. Falco mô tả điều kiện đối với sự kiện runtime, process và container. Wazuh sử dụng decoder và rule để phân tích log và tạo alert. Rule của một hệ không được nạp trực tiếp vào hệ khác chỉ vì đều được gọi là quy tắc phát hiện.

Suricata cần giải mã và chuẩn hóa dữ liệu theo buffer mà rule sử dụng. Method, URI raw, URI normalized, body và header có semantics khác nhau. Một matcher ở gateway chỉ được xem là tương đương trong grammar được hỗ trợ và đã có parity test; không thể suy nó thực hiện toàn bộ ngôn ngữ Suricata.

Rule tự sinh và kiểm định trước sử dụng. GenTI tổ chức tác vụ sinh rule từ yêu cầu và bằng chứng có cấu trúc. AHR-IDPS vận dụng hướng này bằng SFT, prompt có evidence và benign, output JSON và vòng kiểm định bên ngoài model. LLM tạo Candidate; engine xác minh cú pháp và case; worker quản lý Shadow/Active; gateway thực thi trong profile. Các công thức ở chương 5 là mô hình hóa triển khai này, không gán mọi chi tiết ứng dụng cho nguyên bản bài báo GenTI [9].

Một rule hợp lệ cần nhiều mức đánh giá. Hợp đồng kiểm output có đúng schema; syntax kiểm engine chấp nhận; replay kiểm quyết định trên PCAP; Shadow kiểm mức quan sát live; enforcement kiểm tác động trước ứng dụng. Các tầng được nối bằng rule hash và model identity để tránh dùng một receipt cho nội dung khác.

Giảm false positive. Benign cùng endpoint và gần điều kiện attack giúp kiểm tra rule có phân biệt đúng dấu hiệu hay chỉ dựa vào đường dẫn. Encoding, chữ hoa thường và chuỗi gần giống cần được kiểm theo semantics engine. Nếu hai hoạt động khác ý định nhưng không phân biệt được bằng những field đang thấy, không nên ép một rule đơn giản phải phân loại đúng; cần thêm ngữ cảnh hoặc giữ uncertain.

Shadow cung cấp số quan sát khớp và không khớp, nhưng thiếu oracle live thì chưa cho TP hoặc FP. Để ước lượng FPR thực, cần benign được xác nhận theo một mẫu độc lập. Vì vậy Shadow là cổng triển khai hỗ trợ quản lý tác động, không thay benchmark có nhãn.

2.3. Học máy tương quan sự kiện và biểu diễn graph

Các biểu diễn mạng, log và graph khai thác những góc nhìn khác nhau của hoạt động. Tương quan giúp tổ chức ngữ cảnh, còn mô hình học tạo score; hai vai trò này cần được phân biệt khi giải thích lợi ích của hệ thống lai.

Đặc trưng mạng và log. Đặc trưng chuyển một event thành biểu diễn mà model có thể xử lý. Biểu diễn số có thể mô tả thuộc tính flow hoặc HTTP; biểu diễn text giữ các trường log được phép; biểu diễn graph giữ quan hệ node và edge. Sự lựa chọn phải khớp giữa train và inference, gồm thứ tự trường, missing value và cách chuẩn hóa.

Thông tin nhãn, marker chiến dịch, điểm model cũ và kết quả phản ứng cần được tách khỏi input học nếu chúng tiết lộ đáp án hoặc thông tin sau thời điểm dự đoán. Việc xóa một cột chưa đủ nếu cùng thông tin còn nằm trong URI hoặc text đã nối. Cần kiểm tra xuyên suốt quá trình tạo vector và graph.

XGBoost Isolation Forest MLP và biểu diễn log. XGBoost tổng hợp các cây được xây theo gradient của hàm mục tiêu. MLP học ánh xạ phi tuyến giữa vector đầu vào và nhãn. Isolation Forest học cấu trúc cô lập để nhận diện điểm khác phân bố benign. Chúng có giả định và cách tạo score khác nhau; điểm bất thường không mặc nhiên là xác suất attack đã hiệu chuẩn.

Trong nhánh Log của đề tài, lớp có tên MiniLMEmbedder sử dụng backend hashing 384 chiều theo mã đã đối chiếu. Báo cáo vì vậy mô tả thuật toán thực thi, không gọi nó là model Transformer MiniLM. Điều này quan trọng khi diễn giải chất lượng với template chưa gặp và khi ước lượng tài nguyên.

Thích nghi và drift. Thích nghi cần một vòng dữ liệu mới có kiểm soát: phát hiện thay đổi, xác nhận nhãn, xây tập phát triển, retrain và đánh giá trên holdout. Không nên tự đưa mọi request bị chặn trở lại train như attack đã xác nhận. Feedback do hệ thống tạo ra có thể khiến model học lại quyết định của chính nó thay vì học hành vi.

Các chỉ báo drift có thể gồm tỷ lệ missing, phân bố field, điểm model và tỷ lệ template chưa gặp. Chúng cần được diễn giải cùng thay đổi cấu hình hoặc phần mềm. Trong phạm vi báo cáo, đây là thiết kế mở rộng; không công bố một mô-đun tự retrain liên tục đã được nghiệm thu khi chưa có hồ sơ tương ứng.

Tương quan sự kiện và graph. Sessionization và incident. Sessionization nhóm các hoạt động theo phiên hoặc khóa liên kết phù hợp. Correlation sử dụng thời gian, thực thể và loại bằng chứng để tạo incident có timeline. Cửa sổ quá ngắn có thể tách một chuỗi; cửa sổ quá dài có thể gộp nhiều hoạt động không liên quan. Vì vậy chất lượng incident cần được đánh giá ở mức liên kết event, không chỉ bằng số incident tạo ra.

Graph điều tra và Graph AI. Graph điều tra phục vụ hiển thị quan hệ của event, client, process hoặc incident. Graph AI dùng các đặc trưng quan hệ để tạo vector và chấm điểm. Hai chức năng có thể dùng chung dữ liệu nhưng không đồng

nhất. Một graph hiển thị trên dashboard không chứng minh mô hình Graph đã suy luận cho request đó.

Nhánh Graph đã train sử dụng mean aggregation theo phong cách GraphSAGE rồi pooling và MLP. Cần giữ cách mô tả này để phản ánh mã thực tế. Những tuyên bố về temporal GNN đa tầng cần một triển khai và thí nghiệm khác; không suy từ tên artifact.

2.4. Kiến trúc telemetry SIEM và bảo toàn evidence

Dữ liệu vận chuyển, dữ liệu lưu trữ và gói bằng chứng phục vụ những mục đích khác nhau. Tính truy vết đòi hỏi định danh, provenance và kiểm toàn vẹn, đồng thời vẫn phải giữ căn cứ gán nhãn ngoài checksum.

Redis Streams hỗ trợ chuyển các bản ghi giữa bước xử lý; PostgreSQL lưu dữ liệu chuẩn để truy vấn; Wazuh bổ sung phân tích theo luật log. Những thành phần này cần checkpoint, định danh ổn định và cách xử lý lỗi. Một message đã vào stream không chứng minh nó đã được lưu thành công hoặc tạo incident.

Evidence bundle giữ các file mô tả điều đã xảy ra, gồm raw, event chuẩn, rule, receipt và audit. SHA-256 giúp phát hiện thay đổi nội dung sau xuất nhưng không xác nhận nhãn là đúng. Vì vậy checksum phải đi cùng provenance, đơn vị đo và mô tả oracle. Tính kiểm chứng đến từ cả nội dung và cách tạo nội dung đó.

2.5. Cơ sở đánh giá chất lượng và hiệu năng

Một chỉ số chỉ có ý nghĩa khi đơn vị đánh giá, mẫu số và điều kiện đo được xác định. Phần này liên kết các chỉ số phân loại với đặc điểm vận hành, làm cơ sở đọc kết quả của từng tầng.

Precision đo tỷ lệ cảnh báo đúng trong các cảnh báo, recall đo tỷ lệ attack được phát hiện, F1 tổng hợp precision và recall, còn FPR đo benign bị báo nhầm. Chúng cần TP, TN, FP, FN từ một oracle phù hợp. Chỉ số có mẫu số bằng không phải được giữ undefined hoặc null, không tự điền 100%.

ROC-AUC và PR-AUC mô tả chất lượng xếp hạng ở nhiều ngưỡng. Khi lớp attack hiếm, số FP và PR-AUC thường giúp đọc rõ hơn tác động của báo nhầm. Đối với nhiều family, cần cả micro và macro để tránh một family lớn chi phối toàn kết quả. Số dòng, rule khác nhau và capture độc lập phải được công bố cùng tỷ lệ.

Latency và throughput phản ánh mặt vận hành, cần cùng cấu hình và điều kiện tải. Train runtime không phải latency suy luận; thời gian đủ Shadow không phải tốc độ xử lý gateway. Công thức và ranh giới các phép đo được trình bày tiếp ở chương toán, rồi đối chiếu với kết quả thực tế ở phần đánh giá.

CHƯƠNG 3. PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG

Chương này xác lập những yêu cầu mà AHR-IDPS phải đáp ứng và giải thích cách các thành phần phối hợp để thực hiện chúng. Bài toán không dừng ở việc đưa một mô hình AI vào dashboard. Hệ thống cần nối được request thực tế với dữ liệu cảm biến, quyết định phát hiện, nội dung quy tắc, biên nhận kiểm định và tác động phản ứng. Mỗi bước phải có bằng chứng riêng để người đọc phân biệt được khả năng đã thực hiện với khả năng mới được đề xuất.

Đối tượng được bảo vệ là ứng dụng DVWA trong lab Docker cục bộ. Thiết kế sử dụng cảm biến mạng và runtime để quan sát, pipeline dữ liệu để lưu dấu vết, các nhánh AI để nghiên cứu phát hiện và mô hình ngôn ngữ đã fine-tune để sinh quy tắc theo phương pháp GenTI. Luồng chính cho phép tự động chuyển Candidate sang Shadow và Active trong một phạm vi HTTP được kiểm chứng; các trường hợp ngoài phạm vi phải được giữ lại cùng lý do.

3.1. Yêu cầu hệ thống mô hình đe dọa và giả định

Yêu cầu chức năng được xem xét cùng tài sản, tác nhân và ranh giới tin cậy. Việc đặt chúng trong cùng mục giúp lý giải vì sao tự động hóa phải có điều kiện và vì sao khả năng truy vết là một yêu cầu thiết kế.

Thu nhận và truy xuất sự kiện. Hệ thống phải thu nhận được access log của Nginx, bản ghi EVE JSON của Suricata, cảnh báo runtime của Falco và dữ liệu cảnh báo từ Wazuh. Các nguồn này quan sát những mặt khác nhau của cùng hoạt động. Một request có thể sinh nhiều sự kiện, vì vậy không được đồng nhất số record thu nhận với số lần tấn công. Collector cần giữ nguồn, thời gian và định danh ổn định để đối chiếu khi cảm biến gửi lại dữ liệu.

Sau thu nhận, người vận hành phải tìm được sự kiện theo khoảng thời gian, nguồn, URI và định danh. Khi xem một sự cố, dashboard cần liên kết ngược về các sự kiện thành phần thay vì chỉ hiển thị một câu mô tả tổng hợp. Dữ liệu lỗi schema phải được nhận diện và ghi nhận; việc tự chuyển dữ liệu thiếu trường thành benign sẽ làm sai cả đánh giá phát hiện và quy trình sinh rule.

Phát hiện và sinh quy tắc. Hệ thống cần duy trì hai loại đầu ra AI có ý nghĩa khác nhau. Network, Log, Graph và Fusion tạo điểm hoặc nhãn phục vụ phát hiện và điều tra. GenTI sử dụng LLM để tạo một điều kiện phát hiện có thể đưa cho engine

kiểm tra. Điểm rủi ro cao không thay thế một quy tắc hợp lệ, còn một rule đúng cú pháp chưa chứng minh nó phát hiện đúng hành vi cần tìm.

Đối với luồng GenTI chính, hệ thống phải tự chọn các quan sát phù hợp, yêu cầu model phân vai evidence và phản ví dụ, sinh yêu cầu phát hiện rồi tạo Candidate. Kết quả phải giữ nội dung thực do model của đề tài tạo ra, gắn model hash và dấu vết đầu vào. Nếu model không xác định được cả hai nhóm, chu kỳ cần dừng ở trạng thái chưa đủ bằng chứng; không tự tạo phản ví dụ giả để hoàn thành một lượt sinh.

Tự động hóa có điều kiện. Bảng 3.1. Yêu cầu chức năng và bằng chứng nghiệm thu

Mã yêu cầu	Chức năng cần có	Bằng chứng nghiệm thu
FR01	Thu nhận đa nguồn và chống đọc lặp	Event ID, checkpoint và số record theo nguồn
FR02	Chuẩn hóa và truy vấn sự kiện	Schema thống nhất, raw reference và bộ lọc API
FR03	Phát hiện theo chữ ký và AI	SID hoặc model identity, điểm và lý do
FR04	Tạo sự cố từ các sự kiện liên quan	Incident ID, timeline và liên kết event
FR05	AI chọn mẫu và sinh Candidate	Slot đã kiểm tra, prompt hash, rule và model hash
FR06	Kiểm định bằng engine native	Engine version, rule hash và receipt có TP TN FP FN
FR07	Tự động Shadow rồi Active	Điều kiện chuyển trạng thái, bộ đếm và lịch sử
FR08	Chặn HTTP trong profile cho phép	Response trước DVWA và audit applied=true
FR09	Dừng, vô hiệu hóa và hết hạn	Chính sách được lưu, lease và bản ghi gỡ thực thi

Mã yêu cầu	Chức năng cần có	Bằng chứng nghiệm thu
FR10	Lưu kết quả và thông báo dashboard	Rule còn sau tải lại trang, danh sách hoạt động

Mỗi yêu cầu phải có cách kiểm chứng độc lập với phần giao diện. Ví dụ, một nhãn Active trên dashboard chỉ phản ánh trạng thái registry; nghiệm thu FR08 còn cần gửi request, xác nhận HTTP 403 do rule và kiểm tra request chưa đến ứng dụng. Tương tự, việc mở được trang AI không chứng minh mô hình đã được nạp đúng artifact hay smoke test đã đạt.

Tính ổn định và nhất quán

Các container có thời gian khởi động khác nhau, đặc biệt là dịch vụ suy luận LLM trên CPU. Vì vậy tiến trình khởi động phải dựa vào health gate, không chỉ dựa vào thứ tự gọi lệnh. Cơ sở dữ liệu và Redis cần sẵn sàng trước khi worker xử lý chu kỳ; model cần xác nhận identity và kiểm thử phiên trước khi kết quả được dùng để tự động hóa. Trong cấu hình Compose nền đã kiểm tra, chính sách restart mặc định là no; không được mặc định rằng Docker sẽ tự phục hồi mọi service sau khi nó thoát.

Trạng thái vận hành phải được lưu ngoài bộ nhớ của trình duyệt. Việc tải lại dashboard không được xóa rule đã sinh, quyết định dừng tự động hoặc lịch sử kiểm định. Ngược lại, heartbeat thể hiện sức khỏe của phiên hiện tại cần có thời hạn; không tái sử dụng vô hạn một trạng thái ready của lần chạy trước. Hai loại dữ liệu bền vững và tín hiệu tạm thời phải được phân biệt trong thiết kế.

Khả năng tái lập và quan sát

Một kết quả chỉ có thể được tái lập khi giữ được phiên bản engine, nội dung rule, model artifact, cấu hình, tập case và đơn vị đánh giá. Hash của file trọng số, hash của artifact triển khai và hash của ZIP có thể khác nhau; báo cáo và giao diện phải ghi rõ loại hash. Khi kiểm định lại rule sau cấp SID, receipt phải gắn nội dung cuối cùng chứ không chỉ gắn tên Candidate.

Yêu cầu hiệu năng cần được trình bày dưới dạng đại lượng có thể đo: độ trễ từ request đến event, từ event đến incident, thời gian review, thời gian sinh rule, độ trễ kiểm tra gateway, tốc độ xử lý và mức dùng tài nguyên. Đề tài chưa có chuỗi đo tải đầy đủ cho mọi đại lượng này. Do đó các ngưỡng chấp nhận cho vận hành mở rộng là tiêu chí thiết kế cần nghiệm thu tiếp, không được ghi thành thành tích đã đạt.

Bảng 3.2. Đối chiếu thuộc tính, cách thiết kế và đo

Thuộc tính	Cách thiết kế và đo	Điều kiện diễn giải
Tái lập	Pin artifact và engine, lưu manifest	Cùng tên model chưa đủ chứng minh cùng trọng số
Truy vết	Event ID, rule hash, receipt và audit	Có thể đi từ phản ứng về nguồn dữ liệu
Cô lập	Mạng Docker, mount chỉ đọc khi phù hợp	Worker không có quyền quản trị host mặc định
Khả dụng	Health gate, timeout, heartbeat	Hạ tầng lỗi phải được phân loại riêng
Hiệu năng	Đo p50 p95 p99 và throughput	Chỉ công bố sau khi có phép đo cùng cấu hình
Phục hồi	Giữ volume, lease và lịch sử	Khởi động lại không đồng nghĩa phê duyệt lại rule

Mô hình đe dọa và giả định. Tài sản và tác nhân. Tài sản cần bảo vệ gồm tính sẵn sàng của DVWA trong lab, dữ liệu sự kiện, tính toàn vẹn của rule registry, trọng số model và thông tin xác thực quản trị. Tác nhân kiểm thử được phép gửi traffic tới mục tiêu lab trong phạm vi đã xác định. Một tác nhân không tin cậy cũng có thể kiểm soát chuỗi URI, header hoặc nội dung log phát sinh từ request; vì vậy dữ liệu đưa vào LLM phải được xem là dữ liệu quan sát, không phải chỉ dẫn quản trị.

Các ranh giới tin cậy nằm giữa client và gateway, gateway và backend, cảm biến và pipeline, dữ liệu quan sát và prompt, Candidate và worker kiểm định, registry và điểm thực thi. Một lỗi ở ranh giới có thể làm model chọn sai benign, dùng nhầm rule đã kiểm định hoặc ghi trạng thái Active mà không có tác động tương ứng. Thiết kế tập trung vào việc thu hẹp và xác minh từng ranh giới này.

Các tình huống đe dọa chính. Prompt injection có thể xuất hiện dưới dạng chuỗi trong log yêu cầu bỏ qua chỉ dẫn hoặc xuất một rule quá rộng. Biện pháp là giới hạn schema, giới hạn trường đầu vào, không cho model gọi công cụ thực thi tùy ý và kiểm định độc lập đầu ra. Sanitizer cần loại dữ liệu nhạy cảm nhưng vẫn giữ thông tin

cần thiết để mô tả điều kiện phát hiện; nếu xóa mất payload phân biệt attack với benign thì mẫu đó không còn thích hợp để sinh rule.

Một rủi ro khác là vòng lặp tự củng cố nhãn. Request nhận HTTP 403 vì đã bị IPS chặn không tự trở thành mẫu attack đã được xác nhận. Nếu dùng status do chính hệ thống tạo làm nhãn để train hoặc chọn evidence, model có thể học lại quyết định cũ thay vì ý nghĩa hành vi. Vì thế nhãn oracle, trạng thái HTTP và điểm AI cần được lưu tách biệt, đồng thời thử ablation khi đánh giá độ phụ thuộc vào những trường này.

Rule quá rộng có thể chặn truy cập hợp lệ. Rule không tương thích với engine có thể không tạo cảnh báo. Rule hợp lệ nhưng sai profile có thể không tương đương với matcher gateway. Các rủi ro này được xử lý bằng grammar giới hạn, kiểm native, parity trên cùng case, Shadow, lease và thao tác dừng. Các cơ chế trên giảm phạm vi tác động trong lab; chúng chưa thay thế đánh giá an toàn cho production.

Giả định của thí nghiệm. Thí nghiệm giả định người thực hiện có quyền quản lý lab và các nguồn log trong lab không bị sửa ngoài quy trình. Đồng hồ các thành phần cần được đối chiếu theo UTC hoặc cùng quy ước trước khi tính latency. Dữ liệu cảm biến không đầy đủ phải được ghi nhận là thiếu quan sát; việc Falco không báo một request web không có syscall đáng chú ý không tự là false negative của mọi tầng hệ thống.

Gateway mặc định bind 127.0.0.1 trên Windows. Kali trong VMware không truy cập được loopback của Windows bằng địa chỉ loopback của Kali. Mọi mở rộng sang VMnet cần thiết kế và nghiệm thu mạng riêng; báo cáo không giả định môi trường đó đã kết nối hoặc đã có kết quả pentest từ Kali khi hồ sơ hiện tại chưa chứng minh.

3.2. Kiến trúc tổng thể và trách nhiệm các thành phần

Người kiểm thử hoặc client gửi request qua Nginx tới DVWA; MariaDB là cơ sở dữ liệu của ứng dụng lab. Nginx access log cung cấp method, URI, status, upstream và dấu vết request. Suricata quan sát mạng để tạo HTTP/flow/alert; Falco theo dõi runtime; Wazuh tổng hợp các tín hiệu SIEM. Các sensor có nhiệm vụ thu thập và cảnh báo theo cấu hình, không tự nhận mọi trạng thái rule trong registry làm lệnh nạp mới.

Collector đọc các nguồn log, giữ checkpoint để giảm đọc lặp, rồi đưa sự kiện thô lên Redis Streams. Normalizer chuyển về schema chung, bảo toàn raw event và provenance, khử trùng theo định danh sự kiện và lưu PostgreSQL. Redis phục vụ

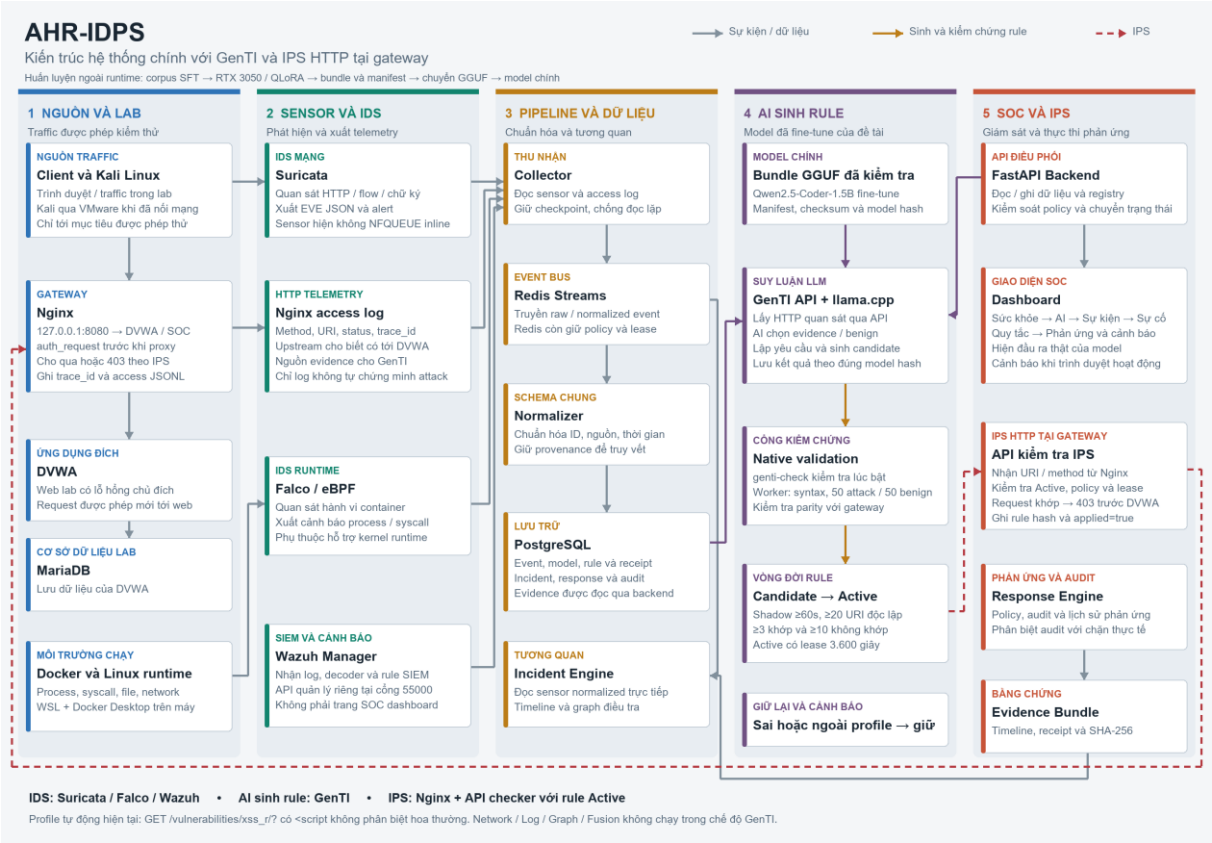
luồng trao đổi và trạng thái ngắn hạn; PostgreSQL giữ event, incident, rule version, validation, audit và response.

Backend FastAPI cung cấp REST/WebSocket cho dashboard. Incident Engine gom sự kiện có căn cứ theo phiên hoặc đối tượng. Graph trong điều tra biểu diễn quan hệ giữa các bằng chứng; nó không bắt buộc phải là một model Graph AI để GenTI hoạt động. Dashboard tách Sức khỏe, Mô hình AI, Sự kiện, Sự cố, Quy tắc và Phản ứng theo công việc của người vận hành.

IDS phân tích hoạt động và tạo tín hiệu phát hiện; IPS tác động lên đường đi của request hoặc lưu lượng. AI sinh rule tạo một điều kiện phát hiện mới từ đầu vào có cấu trúc. Ba vai trò có quan hệ nhưng không thay thế lẫn nhau: model có thể sinh chuỗi đúng JSON mà rule vẫn sai; rule có thể đạt cú pháp mà không khớp attack; registry có thể ghi Active mà chưa có thành phần nào thực thi chặn.

Bảng 3.3. Phân định IDS, AI sinh rule, kiểm định và IPS trong hệ thống chính

Thành phần	**Vai trò**	**Dấu hiệu**
Suricata	IDS mạng	EVE HTTP/flow/alert; SID và URI của request thực.
Falco	IDS runtime	Sự kiện syscall/process/container từ cảm biến.
Wazuh Manager	SIEM/IDS	Nhận log và sinh alert theo luật Wazuh.
GenTI + model Qwen	AI sinh rule	Candidate, nội dung rule, prompt hash, model hash, evidence và benign.
Worker tự động	Kiểm định và vòng đời	Receipt native, bộ đếm Shadow, lịch sử chuyển trạng thái.
Nginx + bộ kiểm tra IPS	IPS HTTP	HTTP 403 trước DVWA, rule header và bản ghi phản ứng có applied=true.
Incident Engine	Tương quan	Event ID liên kết Incident ID, timeline và quan hệ điều tra.



Hình 3.1. Kiến trúc toàn bộ AHR-IDPS và luồng IDS, AI sinh rule, Shadow Active và IPS HTTP

Chức năng của từng node. Nhóm ứng dụng

Bảng 3.4. Đối chiếu node, đầu vào và xử lý

Node	Đầu vào và xử lý	Đầu ra
Nginx gateway	Nhận HTTP, kiểm subrequest IPS, proxy	Access JSONL, status và dấu vết rule
DVWA	Xử lý request và phiên đăng nhập	Trang ứng dụng, hành vi web lab
MariaDB	Lưu dữ liệu phục vụ DVWA	Health DB và dữ liệu ứng dụng
Suricata	Giải mã flow, HTTP, kiểm chữ ký	EVE flow HTTP alert với SID
Falco	Quan sát syscall và hành vi container	Cảnh báo runtime khi điều kiện khớp

Node	Đầu vào và xử lý	Đầu ra
Wazuh Manager	Phân tích log và luật SIEM	Alert, trạng thái daemon và API

Nhóm pipeline và điều tra

Bảng 3.5. Đối chiếu node, đầu vào và xử lý

Node	Đầu vào và xử lý	Đầu ra cần quan sát
Collector	Tail log theo checkpoint, tạo envelope	Raw event ổn định và lỗi nguồn
Redis Streams	Chuyển raw và normalized event	Stream ID, consumer và pending
Normalizer	Ánh xạ schema từng nguồn	Canonical event hoặc lỗi chuẩn hóa
PostgreSQL	Lưu event, rule và lịch sử	Truy vấn, ràng buộc định danh, audit
Incident Engine	Gom bằng chứng theo ngữ cảnh	Incident, timeline, quan hệ thực thể
FastAPI backend	Cung cấp dữ liệu và thao tác quản trị	REST, trạng thái, kiểm tra quyền
React dashboard	Hiển thị và cho phép điều tra	Trang sức khỏe, AI, sự kiện, sự cố, rule

Redis phục vụ luồng xử lý, còn PostgreSQL là nơi truy vấn dữ liệu bền vững theo thiết kế ứng dụng. Không dùng việc có message trong stream để kết luận event đã được lưu thành công. Khi phân tích một khoảng trống dữ liệu, cần kiểm tra lần lượt source file, checkpoint, stream, normalizer và bản ghi DB; cách này xác định được tầng mất dữ liệu thay vì khởi động lại toàn bộ stack ngay từ đầu.

Nhóm AI và phản ứng

Bảng 3.6. Đối chiếu node, chức năng

Node	Chức năng	Ràng buộc chính
Network Log Graph	Chấm điểm theo biểu diễn riêng	Kết quả thực nghiệm không mặc nhiên đang bật
Fusion	Tổng hợp điểm và context	Không là cổng bắt buộc của GenTI
GenTI API và llama.cpp	Review và sinh JSON rule	Model identity, schema và một yêu cầu mỗi lúc
genti-check	Smoke test khi bật model	Gắn startup ID và đúng artifact
genti-automation	Kiểm định và quản lý vòng đời	Grammar, receipt, Shadow và lease
Gateway IPS checker	Quyết định trước DVWA	Rule đúng hash và chính sách còn hiệu lực
Response Engine	Chính sách phản ứng và audit	Audit wrapper không chứng minh chặn thật
Evidence bundle	Gom tài liệu và checksum	Hash bảo vệ toàn vẹn, không xác nhận nhãn

3.3. Luồng xử lý dữ liệu và hình thành quyết định

Luồng dữ liệu đi từ sensor đến kho lưu và giao diện. Luồng điều khiển đi từ rule đã kiểm định tới vòng đời và quyết định chặn tại Nginx. Việc tách hai luồng cho phép xem lại bằng chứng ngay cả khi Candidate chưa được thực thi. Nó cũng ngăn kết luận sai rằng một alert Suricata hoặc một bản ghi Response Engine luôn đồng nghĩa lưu lượng đã bị chặn.

Collector chuẩn hóa timestamp, đọc checkpoint của nguồn rồi tạo định danh từ source và source event ID ổn định. Nếu không có định danh nguồn, canonical JSON của payload được hash để tạo khóa thay thế.

$$e_{id} = SHA256(source \parallel 0x00 \parallel stable_source_id).$$

Normalizer chọn mapper theo sensor, đưa field về schema chung và giữ raw/provenance. Dữ liệu không hợp lệ không được tự đổi nhãn thành benign để đủ mẫu. Redis Streams phục vụ chuyển tiếp, PostgreSQL lưu bản chuẩn và audit.

Thuật toán A Thu nhận và chuẩn hóa

Với mỗi bản ghi sau checkpoint của nguồn:
Lấy định danh nguồn ổn định hoặc hash payload chuẩn.
Tạo event_id từ tên nguồn và định danh ổn định.
Phát envelope thô kèm thời gian và provenance.
Chọn mapper theo nguồn và chuẩn hóa trường.
Nếu sai schema: ghi lỗi hoặc cách ly dữ liệu.
Nếu hợp lệ: lưu theo event_id và phát event chuẩn.
Cập nhật checkpoint theo hợp đồng thu nhận.

Trong bước chuẩn bị dữ liệu học, learning_event loại marker chiến dịch, nhãn, điểm AI cũ và query nhạy cảm khỏi biểu diễn học. Group ID chỉ dùng chia tập/đánh giá. Không dùng ground truth làm đặc trưng cho model.

Request đi vào Nginx, tạo access record và có thể tạo HTTP/flow/alert ở Suricata. Nếu hoạt động gây dấu hiệu runtime phù hợp, Falco cũng có thể phát sinh cảnh báo; Wazuh nhận và phân tích các log đã cấu hình. Sau chuẩn hóa, Incident Engine sử dụng bằng chứng đủ điều kiện và ngữ cảnh để liên kết sự kiện. Một request hợp lệ vẫn cần được lưu để phục vụ benign control, nhưng không nhất thiết tạo sự cố nghiêm trọng.

Khóa tương quan và cửa sổ thời gian giúp nối các sự kiện gần nhau, song cũng có thể gộp nhầm nhiều hoạt động của cùng client hoặc tách một chiến dịch dài thành nhiều incident. Trong mã đã đối chiếu, bucket mặc định là 300 giây. Đánh giá incident vì vậy cần xem cả event thành phần và điều kiện gom, không dùng số lượng incident như số lượng attack chính xác.

Chu kỳ GenTI chọn dữ liệu mới, làm sạch, yêu cầu model review rồi sinh Candidate. Worker lấy Candidate đúng identity để kiểm định. Nếu đạt, rule được đưa vào Shadow nhằm đo mức phơi nhiễm trước khi chặn. Khi các cổng tự động đều đạt, gateway sử dụng rule Active trong profile đã cho phép. Dữ liệu phản ứng đi ngược về dashboard và evidence, tạo chuỗi truy vết khép kín.

Điểm quan trọng là worker điều phối trạng thái nhưng không có quyền tự mở rộng grammar. Một rule SQLi do model sinh vẫn có thể được hiển thị như kết quả nghiên cứu, song nó không được tự chuyển sang chặn HTTP nếu cơ chế thực thi hiện chỉ được kiểm chứng cho profile GET XSS. Việc hiển thị đầu ra model và quyền thực thi của đầu ra là hai quyết định khác nhau.

3.4. Mô hình dữ liệu nhãn và khả năng truy vết

Schema không chỉ thống nhất tên trường mà còn phân biệt dữ liệu gốc, thông tin suy diễn và nhãn oracle. Các liên kết định danh bảo đảm một kết quả được nối đúng với nguồn, model và nội dung rule đã kiểm.

Trường nhận diện và provenance. Bảng 3.7. Đối chiếu nhóm trường, ý nghĩa thiết kế

Nhóm trường	Ý nghĩa thiết kế	Cách sử dụng
event_id	Khóa ổn định của bản ghi chuẩn	Chống lặp và nối evidence
timestamp source	Thời điểm và nguồn quan sát	Timeline, chọn mapper và truy nguyên
scenario_id session_id	Ngữ cảnh chiến dịch hoặc phiên	Chia tập và tương quan khi có
HTTP flow process	Thuộc tính chuyên biệt từng nguồn	Đặc trưng và truy vấn điều tra
severity features	Mức độ và biểu diễn tính toán	Ưu tiên, scoring và kiểm tra drift
label oracle_source	Nhãn đánh giá và căn cứ xác nhận	Lưu riêng khỏi input suy luận
model identity	Artifact tạo điểm hoặc rule	So khớp khi kiểm định và triển khai
evidence_ref raw	Liên kết bản ghi nguồn	Đối chiếu nội dung và checksum

Các trường trong bảng thể hiện hợp đồng khái niệm xuyên suốt báo cáo; từng cảm biến có thể không cung cấp đủ mọi trường. Không nên tạo session_id giả để biến một event runtime thành request HTTP. Khi một trường suy diễn được bổ sung, cần giữ phương pháp suy diễn và không trình bày như dữ liệu gốc của sensor. Schema cần phân biệt missing, unknown và giá trị bằng không.

Nhãn và đặc trưng học. Label dùng đánh giá phải có provenance và phạm vi. Nhãn attack trong chiến dịch thể hiện ý định gửi request tấn công; nó không mặc nhiên xác nhận exploit thành công. Việc sao chép nhãn chiến dịch sang mọi record của nhiều sensor có thể làm các benign control trong cùng chiến dịch bị gán sai. Quy trình chuẩn bị cần nêu rõ đơn vị gán nhãn và cách liên kết marker.

Các trường scenario, marker, nhãn, điểm model cũ và kết quả phản ứng không được đưa trực tiếp vào vector học như dấu hiệu nhận biết attack. Chúng hữu ích cho chia tập và đối chiếu sau suy luận. Kiểm tra leakage phải được thực hiện ở cả bước chọn cột lẫn bước dựng văn bản log, vì một chuỗi marker bị lẫn vào text vẫn có thể trở thành đặc trưng mạnh.

Lưu trữ và nhất quán. Event, incident, rule và receipt có vòng đời khác nhau nhưng phải liên kết qua định danh. Một receipt phải xác nhận rule hash cụ thể; nếu nội dung thay đổi, kết quả kiểm định cũ không còn là bằng chứng cho nội dung mới. Tương tự, Candidate gắn model hash phải giữ nguyên identity ngay cả khi model chính được thay trong tương lai.

Idempotency giúp tránh tạo lặp khi worker retry, nhưng không được che mất một lượt thất bại. Hệ thống cần lưu lý do hold hoặc generation failed, thời điểm và identity để phân biệt lỗi dữ liệu, lỗi inference và lỗi kiểm định. Khi truy vết một phản ứng, chuỗi mong muốn là response → rule version và hash → receipt → Candidate → model và các event được chọn.

3.5. Vòng đời quy tắc và kiểm soát tác động

Candidate, Shadow và Active biểu diễn những mức tin cậy và quyền thực thi khác nhau. Cổng kiểm định, thời hạn và cách xử lý lỗi phối hợp để giới hạn tác động, đồng thời giữ bằng chứng phục vụ điều tra và tái lập.

Thuật toán E Kiểm định và thực thi HTTP có giới hạn

Kiểm artifact, version, rule hash và grammar hỗ trợ.
Chạy native syntax và PCAP có nhãn đúng task.
Yêu cầu 50 attack, 50 benign, FP=FN=0 và parity.
Lưu receipt; chuyển Candidate sang Shadow có điều kiện.
Ghi mọi Shadow trước khi xét chặn bởi rule Active.
Đếm duy nhất method và URI ở hai nhóm khớp, không khớp.
Nếu tuổi ≥60 giây, khớp ≥3, không khớp ≥10, tổng ≥20:
  Kiểm lại receipt và identity; Active với lease 3.600 giây.
Với request thuộc phạm vi bảo vệ:
  Kiểm policy, heartbeat và đúng identity thực thi.
  Nếu rule Active còn lease và khớp: trả 403, ghi audit.
  Nếu checker lỗi: fail closed và ghi lỗi hạ tầng.
  Nếu không có điều kiện chặn: cho chuyển tiếp.
Khi disable hoặc hết hạn: gỡ thực thi, giữ evidence.

Incident Engine lấy event có rule hoặc mức rủi ro đủ điều kiện, tạo khóa tương quan theo scenario/flow/ngữ cảnh và bucket thời gian mặc định 300 giây, rồi upsert timeline và quan hệ event–incident. Trong chế độ sensor-only, severity là căn cứ rủi ro chính; không cần Fusion để GenTI tạo rule. Graph điều tra và Graph AI được giữ phân biệt.

Định nghĩa các trạng thái. Candidate là đầu ra còn cần kiểm định. Shadow là quan sát quyết định giả định mà chưa chặn. Active là được gateway thực thi trong profile và thời hạn tương ứng. Disabled hoặc hết lease làm ngừng thực thi nhưng không xóa evidence. Những trạng thái này giúp tách rõ đề xuất, kiểm tra và tác động; tên trạng thái phải được đọc cùng điều kiện chuyển chứ không chỉ cùng màu hiển thị.

Rule ngoài profile, receipt lỗi, model hash thay đổi hoặc bằng chứng chưa đủ không được tự bỏ qua. Hệ thống giữ Candidate với lý do cụ thể để người vận hành biết còn thiếu gì. Tự động hoàn toàn có nghĩa là tự thực hiện các bước được cho phép khi đủ điều kiện, đồng thời tự dừng ở ranh giới chưa chứng minh; không có nghĩa mọi đầu ra AI đều được áp dụng.

Nguyên tắc lỗi và gỡ thực thi. Khi chính sách bảo vệ đang bật nhưng worker hoặc checker mất trạng thái sẵn sàng, request thuộc phạm vi có thể bị từ chối do lỗi hạ

tầng. Đây là lựa chọn fail closed trong lab và phải được phân loại riêng với HTTP 403 do rule khớp. Nếu cộng cả lỗi hạ tầng vào số tấn công bị chặn, kết quả IPS sẽ bị thổi phồng.

Nút dừng tự động cần đồng thời làm rõ tác động tới enforcement. Vô hiệu hóa một rule chỉ gỡ rule đó; nếu worker vẫn hoạt động, evidence mới có thể tạo một Candidate khác. Khi cần ngừng toàn bộ chu kỳ, người vận hành dùng chính sách dừng và gỡ chặn toàn cục, giữ lại các bản ghi để điều tra nguyên nhân.

Tổng kết thiết kế. Thiết kế AHR-IDPS đặt khả năng kiểm chứng ở giữa mô hình AI và điểm thực thi. Cảm biến tạo bằng chứng; pipeline bảo đảm truy xuất; LLM tạo đề xuất; engine native xác minh hành vi trên case; Shadow đo mức quan sát; gateway thực thi có giới hạn. Sự phân chia này cho phép giải thích một kết quả theo đúng tầng chịu trách nhiệm và giảm nguy cơ gán thành công của một tầng cho toàn hệ thống.

CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG

Chương này trình bày triển khai theo mã nguồn và hồ sơ vận hành đã lưu của AHR-IDPS. Môi trường chính là Windows với Docker Desktop Linux Engine và WSL, ứng dụng DVWA đi qua Nginx, model GenTI chạy bằng llama.cpp trong Docker. Những ảnh giao diện và kết quả thực nghiệm trong chương phản ánh hồ sơ đã lưu, không phải xác nhận rằng stack đang chạy tại thời điểm người đọc mở tài liệu.

4.1. Nền tảng triển khai và môi trường ứng dụng web

Điều kiện phần cứng, container và đường vào DVWA phải được xác định trước khi đánh giá AI hoặc IPS. Sự khác nhau giữa môi trường huấn luyện và môi trường suy luận ảnh hưởng trực tiếp đến thời gian chờ và cách vận hành.

Huấn luyện LLM được thực hiện trên NVIDIA L4 trong Google Colab, trong khi suy luận tại lab sử dụng CPU và trọng số GGUF Q4_K_M. Hai môi trường không có cùng đặc tính thời gian hoặc bộ nhớ. Kết quả train thành công trên L4 không bảo đảm tốc độ sinh rule tức thời trên CPU. Vì vậy giao diện cần phản ánh trạng thái bận, lỗi và thời gian suy luận thực được ghi thay vì suy từ cấu hình GPU lúc train.

Compose nền tham chiếu các image thông qua biến môi trường, sử dụng named volume cho dữ liệu và health check cho các thành phần. Cấu hình Nginx được mount chỉ đọc; các service áp dụng giới hạn tài nguyên và no-new-privileges theo từng vai trò. Suricata native test cần giữ image digest tương ứng với engine đích để tránh kết quả bị đổi vì tag được cập nhật.

Tệp môi trường và secrets được quản lý riêng khỏi báo cáo. Khi tái lập, cần lưu manifest tên image và digest đã sử dụng nhưng không xuất giá trị mật khẩu. Bản cấu hình đưa vào phụ lục phải được che dữ liệu nhạy cảm và giữ cấu trúc đủ để người đọc hiểu port bind, network và volume; một ảnh chụp chứa secret làm giảm chất lượng triển khai dù chức năng có hoạt động.

Triển khai DVWA Nginx và MariaDB. DVWA là mục tiêu web có chủ đích phục vụ lab, không phải ứng dụng để công bố trực tiếp ra Internet. MariaDB chỉ phục vụ mạng cơ sở dữ liệu của lab; DVWA sử dụng thông tin kết nối qua biến môi trường. Health check của DVWA kiểm tra khả năng phản hồi trang setup, còn kiểm thử nghiệp vụ cần thêm đăng nhập, khởi tạo dữ liệu và truy cập các endpoint trong kịch bản.

Nginx là điểm vào duy nhất cho đường HTTP đã nghiệm thu, mặc định 127.0.0.1:8080. Gateway vừa chuyển request tới DVWA vừa phục vụ dashboard và API theo đường dẫn. Thiết kế này tạo một vị trí chung để ghi access log và áp dụng kiểm tra IPS trước ứng dụng. Nếu gửi request trực tiếp tới container DVWA bằng đường khác, phép thử đó đã bỏ qua gateway và không thể dùng để đánh giá enforcement của đường chính.

Access log dạng JSONL cần giữ method, URI, status, thời gian xử lý và thông tin liên kết sẵn có. Status là kết quả xử lý, không phải nhãn ý định độc lập. Khi kiểm chứng chặn trước DVWA, phải đối chiếu response với audit và dấu vết upstream, không chỉ tìm một dòng access log có status khác 200.

4.2. Cảm biến mạng runtime và tích hợp Wazuh

Các cảm biến được triển khai để bổ sung visibility chứ không cùng phát hiện mọi hành vi. Kiểm tra process, kiểm tra dữ liệu và kiểm API quản trị cần được thực hiện riêng để xác định đúng phạm vi quan sát.

Suricata quan sát lưu lượng ở vị trí đã cấu hình và tạo các loại EVE event như flow, HTTP và alert. Đối với rule HTTP, engine cần nhìn thấy dữ liệu ứng dụng phù hợp và reassembly đủ điều kiện. Một dòng access log không chứng minh sensor mạng đã thấy cùng request; việc kiểm tra phải tìm URI, flow hoặc SID tương ứng trong EVE.

Suricata của Colab trong hồ sơ là 7.0.3, trong khi native Docker đích là 8.0.6. Việc kiểm lại trên đích là bắt buộc về mặt phương pháp khi kết luận rule sẽ dùng tại runtime. Trong cấu hình sensor hiện mô tả, Suricata thực hiện IDS; không có cơ sở gọi nó là đường NFQUEUE inline IPS chỉ vì rule được chấp nhận.

Triển khai Falco và kiểm tra runtime event. Falco bổ sung góc nhìn syscall, process và container mà một cảm biến HTTP không cung cấp. Khả năng quan sát phụ thuộc kernel và driver trong môi trường Linux của Docker. Khi driver không nạp hoặc không tương thích, cần ghi nhận trạng thái giảm coverage; không tạo dữ liệu mô phỏng rồi trình bày như log Falco thật.

Một kiểm tra Falco hợp lệ cần một hoạt động runtime có chủ đích trong lab và một cảnh báo có container hoặc process liên quan. Không kỳ vọng mọi SQLi hoặc XSS ở mức chuỗi URI đều tạo syscall khác thường. Do đó coverage Falco phải tính

trên những kịch bản có điều kiện runtime phù hợp, thay vì dùng toàn bộ request web làm mẫu số.

Triển khai Wazuh Manager và API loopback. Wazuh Manager nhận log, áp dụng decoder/rule và cung cấp API quản trị. Quy trình kiểm tra gồm xác nhận port loopback đang lắng nghe, xác thực bằng tài khoản đọc riêng và gọi trạng thái manager. Script wazuh_manager_console.ps1 đã có cơ chế đọc thông tin xác thực cục bộ, lấy token và kiểm daemon. Tùy chọn SetupAccess dùng cho lần thiết lập tài khoản đọc theo quy trình; không chép token hoặc password vào terminal transcript của báo cáo.

Trong source đã có fragment bind Manager API tại 127.0.0.1:55000 và script start_wazuh_api.sh. Chỉ bật khi cần quản trị API, rồi kiểm tra bằng console/auth probe của dự án. Tự động hóa GenTI không phụ thuộc việc người vận hành mở URL 55000 để thao tác dashboard. Wazuh Dashboard là một dịch vụ giao diện riêng, không được chứng nhận đã triển khai chỉ vì Manager API hoạt động.

Khởi tạo cảm biến và phân loại các quy tắc có sẵn

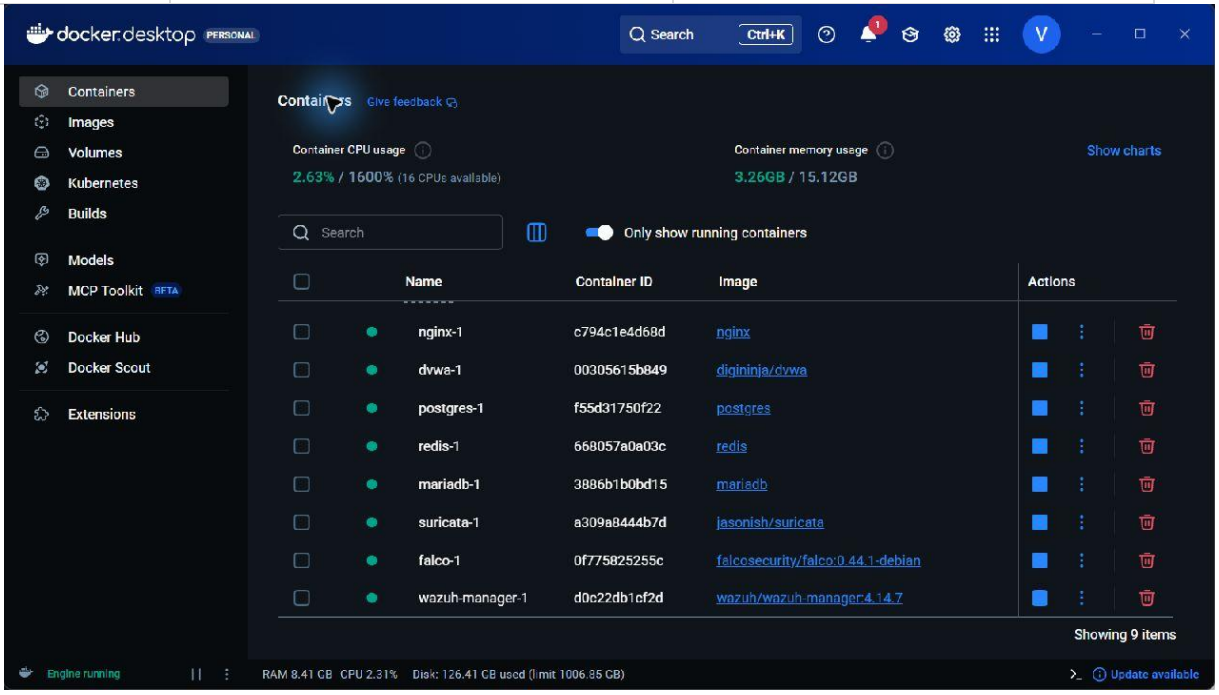
Bộ quy tắc cảm biến được kiểm kê và mở rộng trong lab ngày 09/11/2026. Trạng thái sau mở rộng gồm 33 luật Suricata, 41 luật Falco đang bật và 31 luật AHR của Wazuh. Các ảnh khởi tạo dưới đây ghi lại bước dựng dịch vụ trước khi bổ sung luật; số lượng hiện hành được trình bày trong bảng và danh mục đã cập nhật.

Thứ tự khởi tạo bắt đầu từ MariaDB, PostgreSQL và Redis, tiếp theo là DVWA cùng Nginx. Suricata cần namespace mạng của Nginx; Falco cần quyền quan sát runtime Linux và metadata container. Hai cảm biến tạo file đầu ra trên named volume trước khi Wazuh bắt đầu đọc. Wazuh sau đó nạp decoder, luật chuẩn và ánh xạ Falco riêng. Các container được kiểm tra health, đồng thời đối chiếu file luật với cấu hình để tránh nhầm giữa "file tồn tại" và "engine đã nạp".

Bảng 4.1. Bộ quy tắc cảm biến sau mở rộng

Thành phần	Bộ quy tắc kiểm kê khi khởi tạo	Ý nghĩa và giới hạn
Suricata 8.0.6	33 luật nạp, 0 lỗi, 0 bỏ qua	1 probe và 32 chữ ký HTTP; hành động alert
Falco 0.44.1	41 luật bật: 25 chuẩn + 16 AHR	Quan sát syscall trong DVWA; không tự chặn

Thành phần	Bộ quy tắc kiểm kê khi khởi tạo	Ý nghĩa và giới hạn
Wazuh 4.14.7	31 luật AHR; giữ bộ chuẩn	Ánh xạ cảnh báo và tương quan, không trực tiếp phân tích payload gói mạng



Hình 4.1. Docker Desktop hiển thị hạ tầng lab và ba cảm biến đang chạy

Suricata khởi tạo chế độ IDS mạng

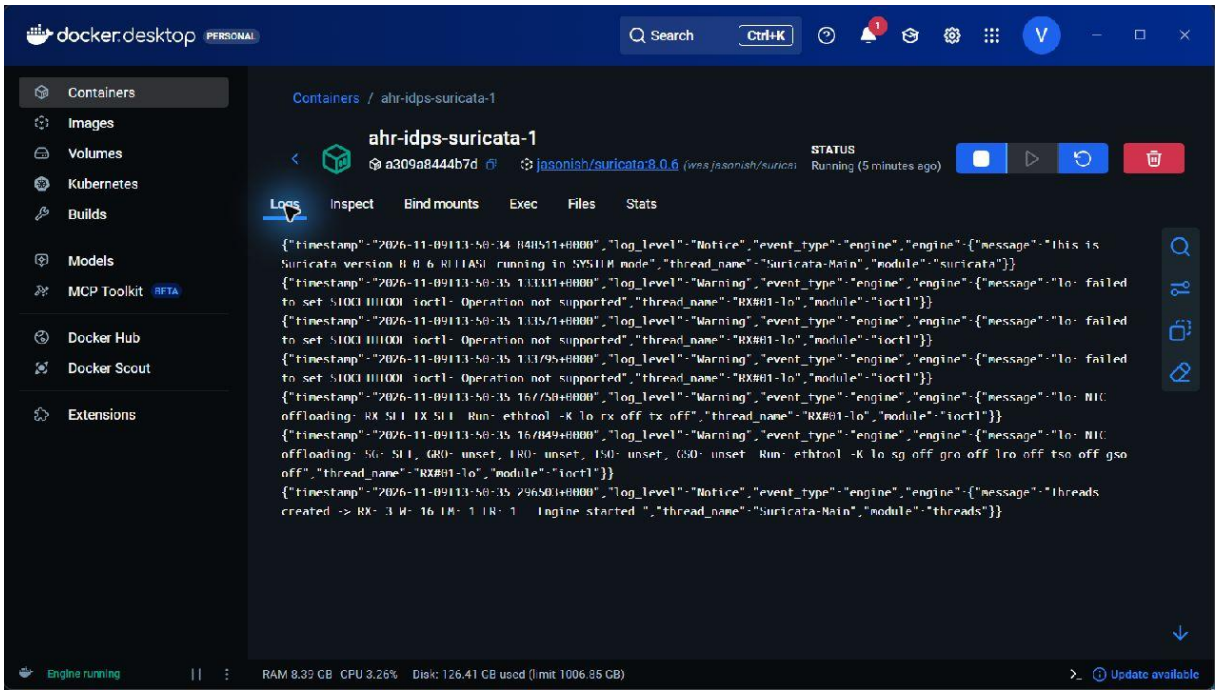
Image Suricata được cố định bằng digest. Container dùng network_mode theo dịch vụ Nginx để quan sát traffic đi tới gateway và traffic từ gateway sang DVWA. Entrypoint tạo các thư mục ghi log và socket điều khiển, thiết lập quyền, sau đó chạy engine với cấu hình được mount chỉ đọc. File suricata.yaml đặt host-mode là sniffer-only và lệnh khởi động chọn PCAP; cơ chế hiện tại quan sát và phát alert, không phải đường NFQUEUE inline để tự drop gói tin.

Suricata nạp base.rules, ahr-web.rules, generated.rules và shadow.rules. SID 9000001 kiểm tra đường bắt HTTP; 32 chữ ký 9000101–9000132 bổ sung phát hiện SQLi, XSS, file inclusion, command injection, upload và reconnaissance trong phạm vi HTTP đã định nghĩa. Hai file dành cho luật generated và shadow vẫn tách khỏi bộ luật nền.

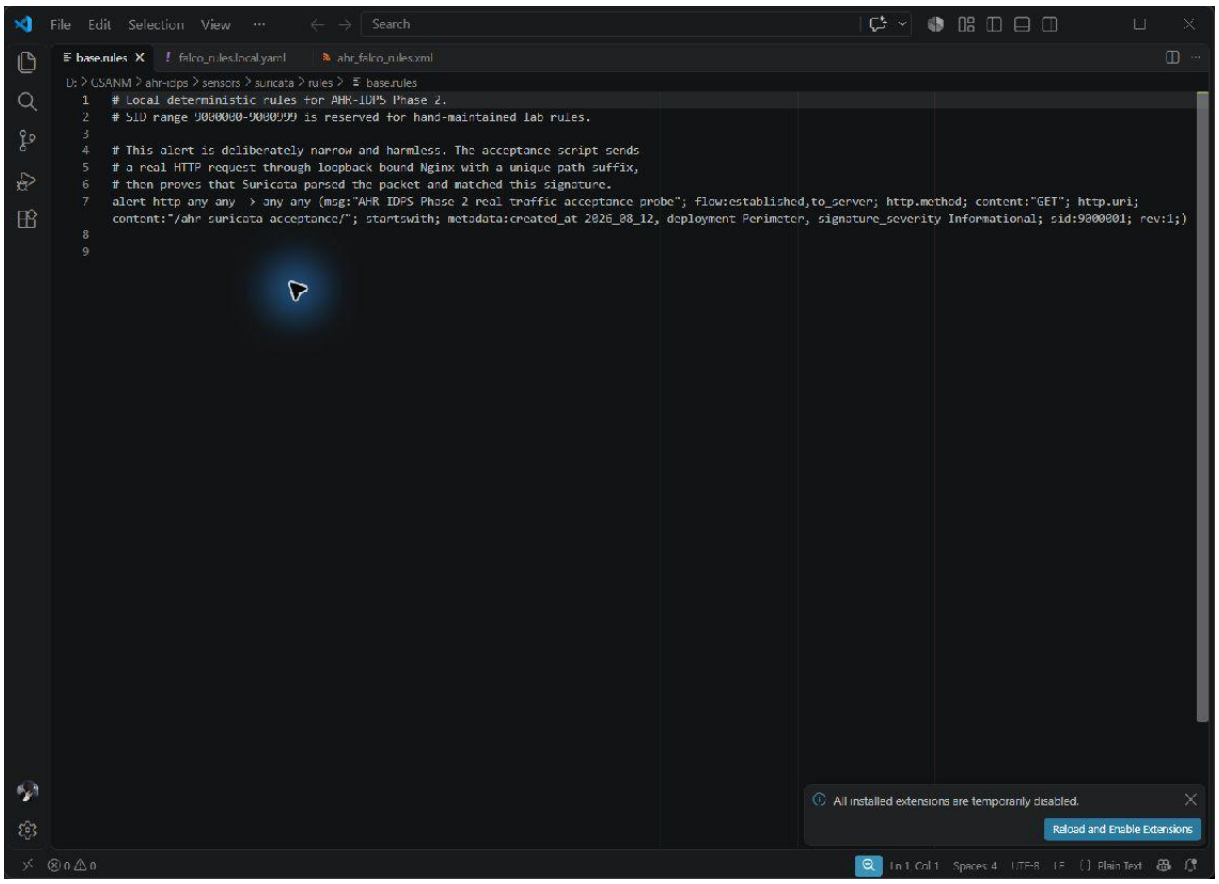
Thống kê ruleset ghi nhận 33 luật nạp, không có luật lỗi hoặc bị bỏ qua. Đây là chữ ký IDS phát alert. Nhãn Active của candidate trên dashboard không tự chứng

minh Suricata đã reload luật; trạng thái cảm biến phải được đối chiếu riêng với cấu hình và EVE.

EVE JSON được ghi vào volume dùng chung cho Collector và Wazuh. Healthcheck kiểm file đầu ra và uptime qua socket. Cảnh báo loopback/offloading trong WSL được giữ nguyên trong ảnh khởi tạo; chất lượng phát hiện được kiểm riêng bằng replay có nhãn.



Hình 4.2. Log Suricata 8.0.6 với thông báo Engine started và cảnh báo loopback được giữ nguyên



Hình 4.3. Luật nền Suricata SID 9000001 trong file được mount vào container

Falco khởi tạo giám sát runtime và bộ luật cho DVWA

Falco sử dụng modern_ebpf, quan sát syscall của kernel Linux phía Docker/WSL và dùng plugin container để bổ sung metadata. Cấu hình AHR bật JSON output, ghi stdout và events.jsonl, đặt mức priority tối thiểu debug, bật rule_matching: all và health endpoint tại 127.0.0.1:8765. Chế độ all cho phép một syscall thỏa nhiều luật, thay vì một luật chuẩn che mất kết quả luật riêng của đề tài. Điều kiện quan sát vẫn phụ thuộc kernel, khả năng nạp BPF và quyền được cấp cho container.

Falco nạp luật chuẩn và file riêng của AHR. Bộ hiện hành có 41 luật được bật: 25 luật chuẩn và 16 luật AHR giới hạn theo DVWA; macro và list không được cộng thành luật. JSON output giữ tên rule, process, parent, lệnh, container và đối tượng file để đối chiếu runtime. File dành cho luật sinh tự động hiện rỗng.

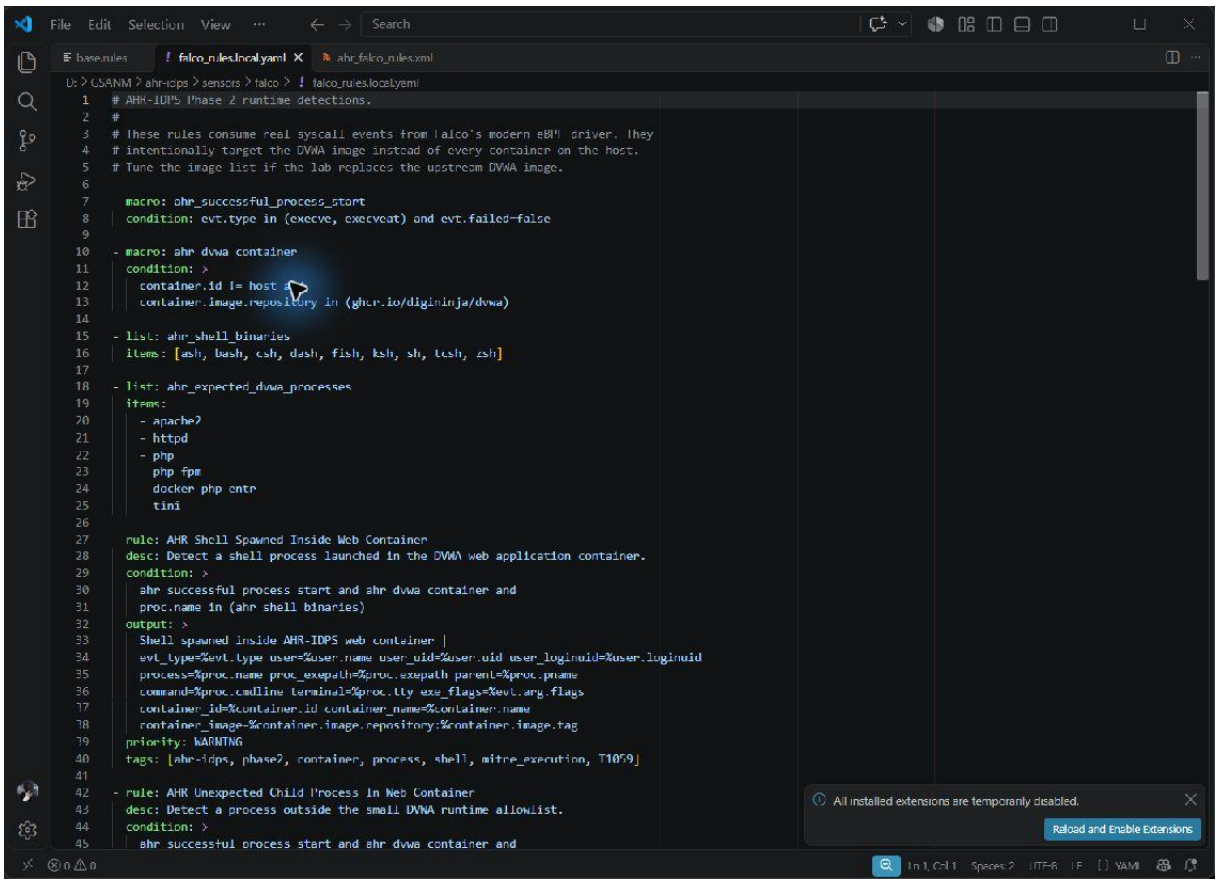
Bảng 4.2. Mười sáu luật runtime AHR dành cho DVWA

Luật riêng của AHR-IDPS	**Điều kiện trọng tâm**	**Mức cảnh báo**

Luật riêng của AHR-IDPS	Điều kiện trọng tâm	Mức cảnh báo
AHR Shell Spawned Inside Web Container	Process khởi chạy thành công và tên shell thuộc danh sách theo dõi trong container DVWA	WARNING
AHR Unexpected Child Process In Web Container	Process nằm ngoài allowlist runtime và ngoài nhóm shell đã theo dõi riêng	WARNING
AHR Suspicious Executable In Web Container	Thực thi từ /tmp, /var/tmp hoặc /dev/shm	ERROR
AHR Sensitive Path Write In Web Container	Mở ghi thành công vào đường dẫn hệ thống nhạy cảm	ERROR
AHR Unexpected Outbound Connection From Web Container	Kết nối ra ngoài, trừ đường Apache tới MariaDB cổng 3306 đã cho phép	WARNING
AHR Privilege Activity In Web Container	Thay đổi identity, capability, namespace hoặc bit setuid/setgid	ERROR
AHR Web Parent Shell Execution	Theo điều kiện process, parent, file hoặc socket trong phạm vi DVWA	ERROR
AHR Web Parent Downloader Execution	Theo điều kiện process, parent, file hoặc socket trong phạm vi DVWA	WARNING
AHR Network Utility In Web Container	Theo điều kiện process, parent, file hoặc socket trong phạm vi DVWA	WARNING
AHR Web Parent Script Interpreter Execution	Theo điều kiện process, parent, file hoặc socket trong phạm vi DVWA	ERROR
AHR Web Process Credential File Read	Theo điều kiện process, parent, file hoặc socket trong phạm vi DVWA	ERROR
AHR PHP Upload File Write	Theo điều kiện process, parent, file hoặc socket trong phạm vi DVWA	ERROR
AHR Scheduled Task File Write	Theo điều kiện process, parent, file hoặc socket trong phạm vi DVWA	ERROR

Luật riêng của AHR-IDPS	Điều kiện trọng tâm	Mức cảnh báo
AHR Docker Socket Connection In Web Container	Theo điều kiện process, parent, file hoặc socket trong phạm vi DVWA	CRITICAL
AHR Namespace Utility In Web Container	Theo điều kiện process, parent, file hoặc socket trong phạm vi DVWA	ERROR
AHR Web Parent Reconnaissance Tool Execution	Theo điều kiện process, parent, file hoặc socket trong phạm vi DVWA	WARNING

Các luật AHR giới hạn theo repository image ghcr.io/digininja/dvwa và container.id khác host. Điều kiện này giảm nhiễu từ các dịch vụ còn lại nhưng cũng giới hạn phạm vi: khi đổi image web, phải rà lại macro, không mặc định mọi container mới đều được bao phủ. Một shell khởi chạy do thao tác quản trị hợp lệ vẫn có thể thỏa điều kiện; cảnh báo Falco cần được đọc cùng process, parent, command và bối cảnh phiên. Trong cấu hình này Falco phát hiện và ghi evidence, không tự chặn hay cô lập container.



Hình 4.4. Macro giới hạn DVWA và luật Falco phát hiện shell trong cấu hình khởi tạo

Wazuh khởi tạo SIEM và ánh xạ luật cảm biến

Wazuh chạy topology manager-only. Cấu hình ossec.conf bật JSON alerts, nhận log EVE của Suricata và events.jsonl của Falco trên volume dùng chung, đồng thời bật Docker listener. Decoder chuẩn được lấy từ ruleset/decoders, rule chuẩn từ ruleset/rules; thư mục etc/decoders và etc/rules dành cho mở rộng của dự án. Kiểm tra wazuh-analysisd -t kết thúc thành công, và các tiến trình analysisd, logcollector, remoted, modulesd, authd cùng database được healthcheck theo dõi.

Wazuh giữ bộ luật chuẩn của image: kiểm kê 168 file XML với 4.512 định nghĩa trước khi áp dụng loại trừ và dependency. Phần mở rộng AHR có 31 luật; không đồng nhất số định nghĩa phân phối với số rule thực sự tạo cảnh báo. Rule mẫu SSH 100001 vẫn là cấu hình minh họa riêng của image.

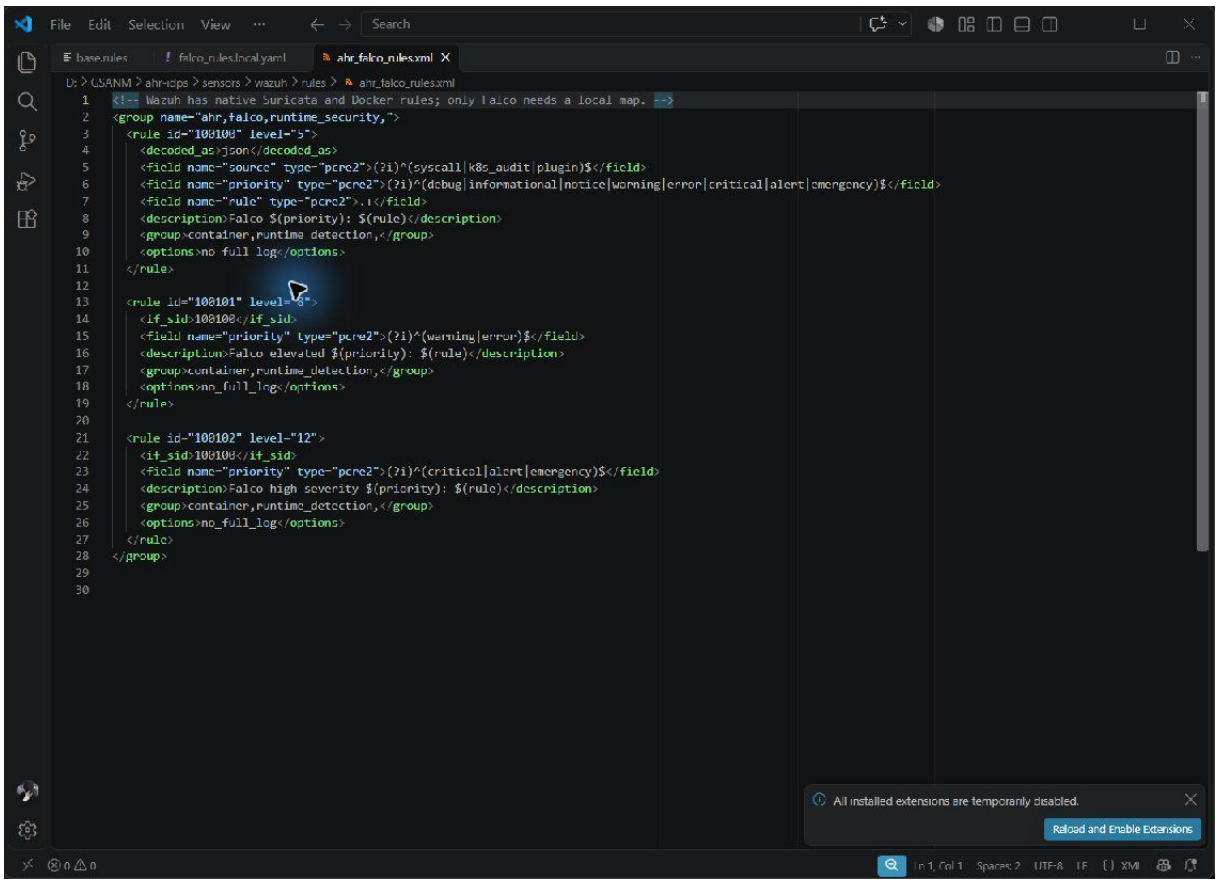
Wazuh có sẵn nhóm Suricata trong 0475-suricata_rules.xml: 86600 ở level 0 phân loại thông điệp JSON, 86601 ở level 3 nhận event_type alert, còn 86602–86604 xử lý HTTP, DNS và TLS ở level 0. Nhóm Docker nằm trong các file 0455-docker_rules.xml và 0560-docker_integration_rules.xml. Đây là logic SIEM xử lý sự kiện do nguồn khác cung cấp, không phải chữ ký payload mà Wazuh trực tiếp áp lên

gói mạng. Cũng không được cộng số rule Wazuh vào số signature Suricata để tính độ bao phủ tấn công.

Phần AHR gồm ba ánh xạ priority Falco, 16 ánh xạ theo tên luật runtime, chín luật phân loại HTTP và ba luật tương quan lặp. Các luật tương quan 100250–100252 dùng cùng src_ip, dest_ip, frequency 5 trong 60 giây; IP trên leg upstream có thể là IP proxy, cần đối chiếu access log trước phản ứng.

Cổng 1514 phục vụ nhận sự kiện agent và 1515 phục vụ enrollment, đều giới hạn loopback của máy. API Wazuh là dịch vụ HTTPS cổng 55000, chỉ được publish khi ghép thêm compose.api.fragment.yml bằng script start_wazuh_api.sh. Trong phiên kiểm tra, overlay API đã được ghép, cổng 55000 publish trên loopback và truy vấn HTTPS không kèm xác thực nhận HTTP 401. Kết quả xác nhận API có phản hồi và yêu cầu xác thực; không có token hay mật khẩu được đưa vào ảnh. Đây là API quản lý, không phải giao diện Wazuh Dashboard. Topology hiện tại không khởi tạo indexer và Wazuh Dashboard; màn hình SOC của đề tài là dashboard riêng do backend AHR-IDPS cung cấp.

Một số chức năng như rootcheck, FIM/syscheck, SCA, syscollector và vulnerability detection được cấu hình tắt trong manager-only lab. Không được mô tả chúng đã giám sát đầy đủ chỉ vì tiến trình container đang chạy. Tương tự, đăng ký agent có cơ chế mật khẩu riêng nhưng giá trị bí mật không xuất hiện trong ảnh hay báo cáo.



Hình 4.5. Ba luật ánh xạ Falco của Wazuh với ID 100100 đến 100102

Phạm vi phát hiện của bộ luật nền

Bộ mở rộng bổ sung 70 luật cấu hình: 32 Suricata, 10 Falco và 28 Wazuh. Các luật này được soạn cho lab và kiểm bằng engine; chúng không phải đầu ra của model GenTI fine-tune. Suricata và Falco phát hiện; Wazuh phân loại và tương quan. Quyết định chặn HTTP thuộc điểm thực thi IPS riêng. Danh mục sau cho phép truy ngược SID hoặc ID từ cảnh báo về điều kiện và phạm vi quan sát.

Entrypoint Wazuh đồng bộ file luật AHR từ cấu hình nguồn vào volume etc mỗi lần khởi động, với quyền root:wazuh 0640. Cơ chế này tránh nạp bản cũ còn trong volume. Việc sửa luật thực hiện trên file nguồn rồi kiểm syntax và reload/restart; không chỉ sửa bản sao tạm trong container.

Bảng 4.3. Danh mục 32 chữ ký HTTP bổ sung

SID	Dấu hiệu phát hiện	Phạm vi
9000101	WEB SQLi UNION SELECT	GET; URI/body và endpoint theo điều kiện chữ ký

SID	Dấu hiệu phát hiện	Phạm vi
9000102	WEB SQLi quoted boolean condition	GET; URI/body và endpoint theo điều kiện chữ ký
9000103	WEB SQLi time delay SLEEP	GET; URI/body và endpoint theo điều kiện chữ ký
9000104	WEB SQLi time delay BENCHMARK	GET; URI/body và endpoint theo điều kiện chữ ký
9000105	WEB SQLi schema discovery	GET; URI/body và endpoint theo điều kiện chữ ký
9000106	WEB SQLi server file read	GET; URI/body và endpoint theo điều kiện chữ ký
9000107	WEB SQLi server file write	GET; URI/body và endpoint theo điều kiện chữ ký
9000108	WEB reflected XSS script tag	GET; URI/body và endpoint theo điều kiện chữ ký
9000109	WEB reflected XSS event handler	GET; URI/body và endpoint theo điều kiện chữ ký
9000110	WEB reflected XSS javascript URI	GET; URI/body và endpoint theo điều kiện chữ ký
9000111	WEB reflected XSS cookie access	GET; URI/body và endpoint theo điều kiện chữ ký
9000112	WEB stored XSS script in form body	POST; URI/body và endpoint theo điều kiện chữ ký
9000113	WEB stored XSS event handler in form body	POST; URI/body và endpoint theo điều kiện chữ ký
9000114	WEB file inclusion directory traversal	GET; URI/body và endpoint theo điều kiện chữ ký
9000115	WEB file inclusion passwd path	GET; URI/body và endpoint theo điều kiện chữ ký

SID	Dấu hiệu phát hiện	Phạm vi
9000116	WEB file inclusion PHP filter wrapper	GET; URI/body và endpoint theo điều kiện chữ ký
9000117	WEB file inclusion data wrapper	GET; URI/body và endpoint theo điều kiện chữ ký
9000118	WEB command injection shell separator and identity command	POST; URI/body và endpoint theo điều kiện chữ ký
9000119	WEB command injection shell separator and downloader	POST; URI/body và endpoint theo điều kiện chữ ký
9000120	WEB command injection shell executable path	POST; URI/body và endpoint theo điều kiện chữ ký
9000121	WEB command injection command substitution	POST; URI/body và endpoint theo điều kiện chữ ký
9000122	WEB recon environment file	GET; URI/body và endpoint theo điều kiện chữ ký
9000123	WEB recon Git configuration	GET; URI/body và endpoint theo điều kiện chữ ký
9000124	WEB recon Subversion entries	GET; URI/body và endpoint theo điều kiện chữ ký
9000125	WEB recon Apache password file	GET; URI/body và endpoint theo điều kiện chữ ký
9000126	WEB recon SQL backup	GET; URI/body và endpoint theo điều kiện chữ ký
9000127	WEB upload server-side PHP extension	POST; URI/body và endpoint theo điều kiện chữ ký
9000128	WEB upload PHP command execution code	POST; URI/body và endpoint theo điều kiện chữ ký
9000129	WEB file inclusion metadata target	GET; URI/body và endpoint theo điều kiện chữ ký

SID	Dấu hiệu phát hiện	Phạm vi
9000130	WEB remote HTTP file inclusion	GET; URI/body và endpoint theo điều kiện chữ ký
9000131	WEB sqlmap user agent on DVWA	GET; URI/body và endpoint theo điều kiện chữ ký
9000132	WEB diagnostic HTTP method on DVWA	TRACE; URI/body và endpoint theo điều kiện chữ ký

Bảng 4.4. Nhóm ID Wazuh và quan hệ với cảm biến

ID	Vai trò	Nguồn hoặc điều kiện
100100–100102	3 ánh xạ priority	Falco JSON
100110–100125	16 ánh xạ runtime cụ thể	Tên luật AHR Falco
100200	Cha phân loại HTTP	SID 9000101–9000132
100201	SQLi	9000101–9000107
100202	XSS	9000108–9000113
100203	LFI/RFI	9000114–9000117; 9000130
100204	Command injection	9000118–9000121
100205	Dò file nhạy cảm	9000122–9000126
100206	Upload	9000127–9000128
100207	URL metadata	9000129
100208	Công cụ/method HTTP	9000131–9000132
100250–100252	Tương quan SQLi, dò file, XSS	5 sự kiện / 60 giây, cùng src_ip và dest_ip

Phân biệt luật khởi tạo và quy tắc do AI tạo

Luật chuẩn đi cùng image là điểm xuất phát của nhà cung cấp; luật AHR là cấu hình riêng được người xây dựng hệ thống viết và mount vào cảm biến; candidate GenTI là kết quả của model fine-tune, có model hash, yêu cầu, evidence và receipt riêng. Ba nguồn này cần được thể hiện tách biệt khi báo cáo kết quả. Số lượng rule có

sẵn không phải số rule AI đã sinh, và số rule AI đã sinh không phải số rule đang có hiệu lực chặn.

4.3. Pipeline dữ liệu backend và dashboard

Thu nhận, chuẩn hóa, lưu trữ và hiển thị tạo một đường liên tục phục vụ điều tra. Trạng thái trên giao diện phải được đối chiếu với dữ liệu backend để tránh coi một trang trống hoặc dữ liệu cũ là hệ thống an toàn.

Collector theo dõi vị trí đọc của từng nguồn, đóng gói bản ghi thô và gửi vào luồng. Việc dùng định danh ổn định giúp normalizer hoặc tầng lưu trữ nhận ra bản ghi đọc lại sau restart. Với tệp log xoay vòng, checkpoint phải được kiểm tra cùng danh tính nguồn; nếu chỉ lưu offset mà không nhận diện file, hệ thống có thể bỏ dữ liệu hoặc đọc trùng.

Normalizer chọn mapper theo sensor, chuyển thời gian, thuộc tính HTTP, severity và raw reference về dạng chung. Không gom mọi nguồn thành một chuỗi văn bản duy nhất rồi bỏ provenance. Khi gặp dữ liệu không hợp lệ, hệ thống cần lưu lỗi để điều tra và không tiếp tục gán nhãn mặc định thuận tiện cho train.

Redis Streams vận chuyển các bước xử lý và PostgreSQL lưu dữ liệu dùng cho API. Khi service bị gián đoạn, cần quan sát consumer pending, lỗi DB và số event được lưu theo cùng khoảng thời gian. Kiểm thử restart nên dùng một tập request có marker để so sánh số record trước sau, tránh nhầm traffic nền của dashboard với dữ liệu bị lặp.

FastAPI backend WebSocket và React dashboard. Backend cung cấp các tài nguyên health, events, incidents, rules, models và responses. Dashboard sử dụng API cùng cơ chế cập nhật hiện có để hiển thị trạng thái. Các trang cần có tình huống thiếu dữ liệu, tải lỗi hoặc model chưa sẵn sàng; một danh sách trống không được tự hiểu là hệ thống an toàn nếu kết nối đang lỗi.

Người vận hành mở Sức khỏe để biết dịch vụ và ingestion có hoạt động; Mô hình AI để biết artifact và boot check; Sự kiện để xem request nguồn; Sự cố để kiểm timeline; Quy tắc để xem vòng đời; cuối cùng Phản ứng để xác nhận tác động. Đây là trình tự đọc bằng chứng, không phải yêu cầu phải bấm tay từng bước cho mỗi rule.

Bảng 4.5. Quy trình đọc bằng chứng trên các trang dashboard

Trang	Cần xem	Cách diễn giải
Sức khỏe	Container/sensor, ingestion gần nhất	Process healthy không đồng nghĩa vừa có event mới.
Mô hình AI	Base, artifact, stage, boot check, review và kết quả	Đúng model fine-tune; đầu ra giữ sau refresh.
Sự kiện	Source, URI, status, rule, raw event	Nối request với tín hiệu sensor, tránh chỉ đọc điểm.
Sự cố	Timeline và event liên quan	Xác định chuỗi và nguồn, không coi mọi request là sự cố.
Quy tắc	Candidate/Shadow/Active, lý do chờ, native, lease	Chỉ cột thực thi IPS mới nói đang chặn bằng rule.
Phản ứng	http_block, applied, target, rule và audit	Đối chiếu upstream để xác nhận chặn trước DVWA.

Khi kiểm tra WebSocket hoặc polling, cần đối chiếu số hoạt động mới với nguồn API và thử mất kết nối rồi phục hồi. Việc tạm dừng làm mới ở dashboard cũng dừng cập nhật thông báo giao diện; người vận hành cần hiểu đây là trạng thái trình bày, không phải chứng cứ worker đã ngừng. Các quyết định thay đổi chính sách phải được xác nhận ở backend và có audit, không chỉ đổi trạng thái nút trong React.

4.4. Tích hợp mô hình AI và tự động sinh quy tắc

Việc nạp đúng artifact, chọn bằng chứng và giữ đầu ra model là các phần liên quan của một chu kỳ. Các nhánh phân loại và GenTI có vai trò khác nhau; trạng thái vận hành phải phản ánh đúng nhánh đang được sử dụng.

Thư mục Drive lưu các model phân loại đã được người thực hiện huấn luyện, gồm Network, Log, Graph và Fusion, cùng mô hình đối chứng. Chúng trả điểm rủi ro hoặc xác suất nhãn; GenTI trả nội dung quy tắc. Trong luồng chính đang mô tả, GenTI lấy evidence trực tiếp từ sensor và không cần điểm Fusion để bắt đầu sinh rule. Kết quả phân loại vẫn là một phần công việc của đề tài, nhưng không được dùng thay cho chất lượng rule sinh hoặc hiệu quả IPS.

Bảng 4.6. Vai trò các nhánh phân loại trong nghiên cứu hệ thống

Khối	Đầu vào và chức năng	Quan hệ với luồng chính
Network	Đặc trưng mạng/flow; XGBoost chấm điểm	Nghiên cứu phát hiện; không là điều kiện bắt buộc của GenTI.
Log	Biểu diễn event/log; phân loại nội dung	Có thể giúp ưu tiên điều tra; không thay thế bằng chứng native.
Graph	Quan hệ thực thể trong graph sự kiện	Phụ thuộc biểu diễn và cấu trúc graph; không mặc nhiên là temporal GNN đầy đủ.
Fusion	Kết hợp điểm rule và các model; có OOF	Thử nghiệm tổng hợp rủi ro; không tự cấp quyền chặn cho rule LLM.

Trong luồng GenTI chính, script điều khiển dừng các worker phân loại không cần thiết để dành tài nguyên cho LLM và giữ đường vận hành rõ ràng. Điều này không xóa các model và kết quả nghiên cứu trong Drive. Dashboard cần phản ánh nhánh nào đang hoạt động, nhánh nào chỉ có artifact đã đăng ký, tránh suy rằng mọi khối xuất hiện trong sơ đồ đều đang xử lý live.

Incident Engine vẫn phục vụ việc gom bằng chứng từ sensor trong chế độ phù hợp. Graph điều tra biểu diễn quan hệ giữa sự kiện và thực thể; nó khác với Graph AI tạo vector graph để phân loại. Một sơ đồ đẹp trên dashboard không tự chứng minh GNN đã chạy, còn một model Graph đã train không tự bảo đảm incident graph hiển thị đúng.

Mô hình chính đã được thay bằng checkpoint stage 3 của lượt fine-tune cục bộ Qwen2.5-Coder-1.5B. Bản merged được kiểm checksum rồi chuyển offline sang GGUF Q4_K_M bằng image llama.cpp cố định digest. Runtime, adapter API và backend giữ cùng hợp đồng; model 7B trước đó được lưu nguyên vẹn tại thư mục rollback. Không dùng kết quả benchmark trước lượng tử hóa để thay cho kiểm chứng runtime mới.

Bảng 4.7. Định danh và chuỗi truy vết model triển khai

Định danh	Giá trị hoặc ý nghĩa
Base model	Qwen/Qwen2.5-Coder-1.5B-Instruct
Base revision	2e1fd397ee46e1388853d2af2c9931 45b0f1098a
Stage	3; chọn theo validation; corpus 50.000 prompt
Runtime contract	ahr-genti-gguf, Q4_K_M
Hash file GGUF	41e71e31466ac8a63204836c04057e 285ae8eb664778c1de770f886b95fad07f
Hash artifact runtime theo verifier	ecc4941e369a49a2833907a2080292 45e183708234ee2e21e4d18b3ddca96697
Nguồn merged manifest	4f4f54774a8b65e8fe3070932c9622 536e13f7936134da7dfc4f362c932ee818

Nguồn: Manifest runtime cục bộ và identity trong các receipt vận hành; các hash có vai trò khác nhau, không phải checksum toàn ZIP.

Compose cấu hình llama server CPU 4 thread, context 2.048, một slot, tối đa 512 token đầu ra và giới hạn RAM container 8 GiB. Adapter giới hạn prompt ở 1.536 token, gọi nội bộ ở nhiệt độ 0, seed 42, yêu cầu JSON và từ chối đầu ra chưa kết thúc trong giới hạn token. Không có fallback tự động sang model bên ngoài khi model chính lỗi.

Mỗi lần bật model, checker chạy smoke native theo startup ID và artifact hiện tại. Kết quả cũ không được dùng để chứng nhận một lượt boot mới hoặc trọng số khác. Smoke nhỏ kiểm tra đường inference và công cụ hoạt động; native 50/50 ở mức rule là phép kiểm khác.

Worker tìm event Nginx GET DVWA mới trong cửa sổ 30 phút, lọc bỏ API, dashboard, tài nguyên tĩnh, khử URI lặp và lấy một pool tối đa sáu quan sát phù hợp. Bộ lọc giúp giới hạn dữ liệu và tăng độ đa dạng; vai trò evidence/benign cuối cùng do

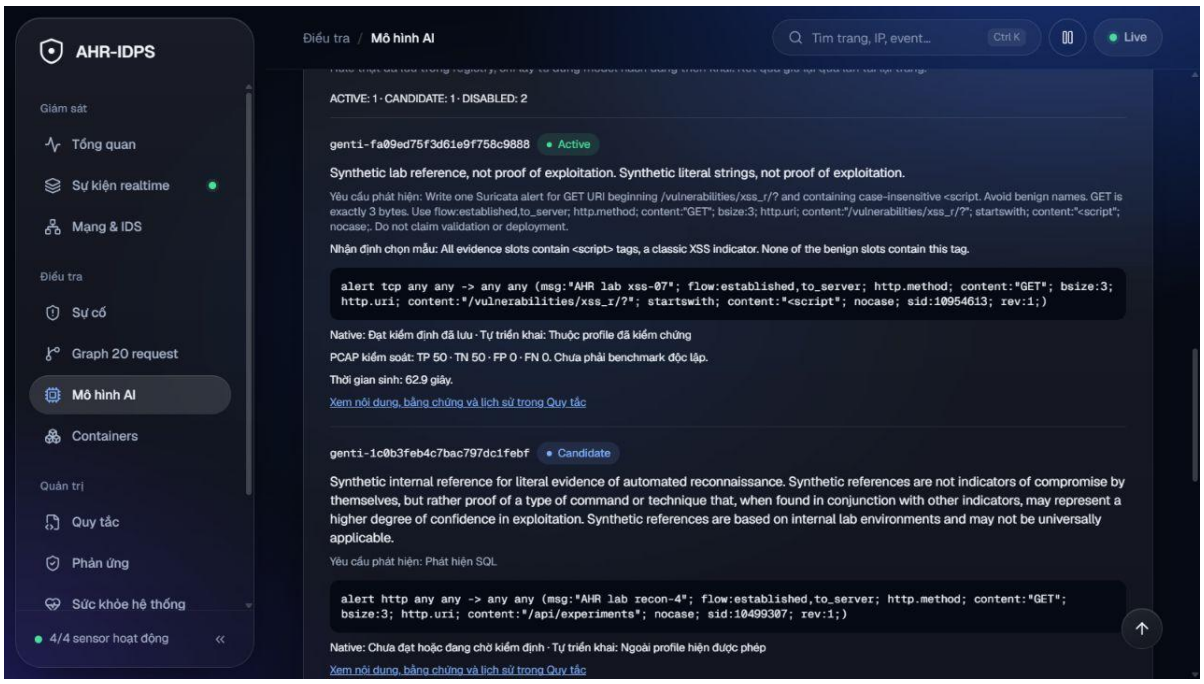
cùng model fine-tune chọn bằng slot. Model viết task và reason, có thể để nhóm rỗng nếu không đủ căn cứ.

Backend kiểm schema, slot nguyên trong phạm vi, không trùng hai vai trò và cùng endpoint. Nó không chấp nhận Event ID do model tự bịa. Chỉ khi có cả evidence lẫn benign hợp lệ, worker mới sinh Candidate. Trong hồ sơ GET XSS được phép, task được ràng buộc theo điều kiện đã kiểm chứng để rule và gateway cùng diễn giải một nghĩa.

Trong lượt nghiệm thu model mới, cùng LLM đang cài thực hiện chọn evidence và benign từ event Nginx, rồi sinh Candidate. Rule lưu artifact mới, prompt hash và các Event ID đã chọn; không gán cờ human-approved. Kiểm định native, Shadow, Active và chặn HTTP được ghi thành các bằng chứng riêng ở Chương 7.

Rule được lưu trong PostgreSQL cùng nội dung sau cấp SID, version, model hash, prompt hash, ID bằng chứng, ID phản ví dụ, lý do chọn mẫu và thời gian suy luận nếu đã đo. Trang AI lấy kết quả đã lưu theo đúng model đang chạy, vì vậy tải lại trang không làm mất Candidate. Không thay đầu ra thực bằng chuỗi rule minh họa rồi gọi đó là thành tích của model.

Ảnh dưới là kết quả thực tế trong hồ sơ ngày 02/11. Nó cho phép xem đúng rule do model sinh, trạng thái Active và bộ đếm native, đồng thời giữ phân biệt giữa kết quả được lưu và trạng thái trực tiếp của dịch vụ sau khi tắt Docker.



Hình 4.6. Kết quả model trên dashboard: rule, giải thích chọn mẫu, thời gian sinh và kết quả native được giữ lại. Ảnh lưu ngày 02/11/2026.

Shortlist dùng nguồn, endpoint và một số marker để tạo pool đa dạng, tối đa sáu quan sát. Model chọn các slot evidence/benign và được phép trả uncertain/nhóm rỗng. Backend kiểm slot nguyên, không trùng, cùng endpoint, đủ hai nhóm và đúng artifact. Generation nhận yêu cầu theo contract, decode JSON, kiểm rule một dòng và các trường ngoài không được phép, cấp SID rồi ghi nội dung/hash/model vào registry.

Thuật toán D Tự chọn mẫu và sinh quy tắc

Nếu policy tắt hoặc boot/model chưa ready: chờ.

Lấy pool HTTP DVWA mới, làm sạch và giới hạn.

Gọi model chính để review các slot quan sát.

Nếu slot sai, thiếu một vai trò hoặc khác endpoint: giữ.

Tạo task từ nhận định đã kiểm hợp đồng.

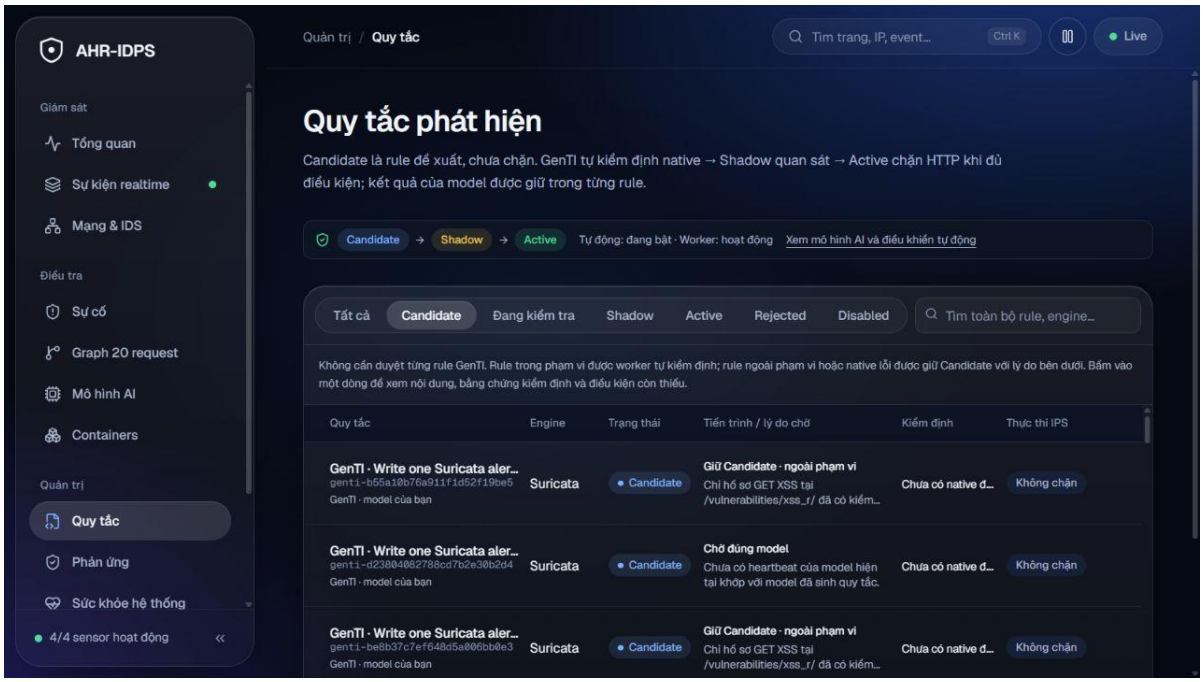
Gọi model chính sinh một JSON quy tắc phòng vệ.

Nếu output sai hoặc model đổi: ghi lỗi, không triển khai.

Lưu Candidate, nội dung, artifact và event đã chọn.

Hash khóa đầu ra gắn artifact và prompt; idempotency/cooldown giảm tạo lặp. Nhãn status do IPS tạo ra có thể là hậu quả quyết định chặn trước đó, nên không dùng status=403 làm ground truth của ý đồ tấn công. Đây là một nguồn feedback bias cần loại/ablate trong tập review tiếp theo.

Rule mới phải gắn đúng model, đúng nội dung, đúng version và thuộc grammar tự động được hỗ trợ. Candidate ngoài profile được giữ lại kèm lý do; model cũ hoặc native lỗi cũng không được đẩy Active. Hàng đợi native không chỉ nhìn 30 dòng đầu; biên nhận lỗi được gắn version/hash để tránh chặn nhầm một nội dung mới bằng lỗi cũ.



Hình 4.7. Trang Candidate: hiển thị tiến trình, lý do chờ, kiểm định và trạng thái chặn. Các rule sai model hoặc ngoài phạm vi được giữ lại.

Worker chạy Suricata với nội dung sau cấp SID, tạo receipt chứa rule hash, rule version, model artifact, engine version và đường dẫn kết quả. Bộ kiểm có ít nhất 50 attack + 50 benign tổng hợp và yêu cầu FP=FN=0. Quyết định gateway được đối chiếu trên cùng các case để bảo đảm không dịch gần đúng semantics của rule. Một receipt cũ hoặc sai hash không được phép chuyển trạng thái.

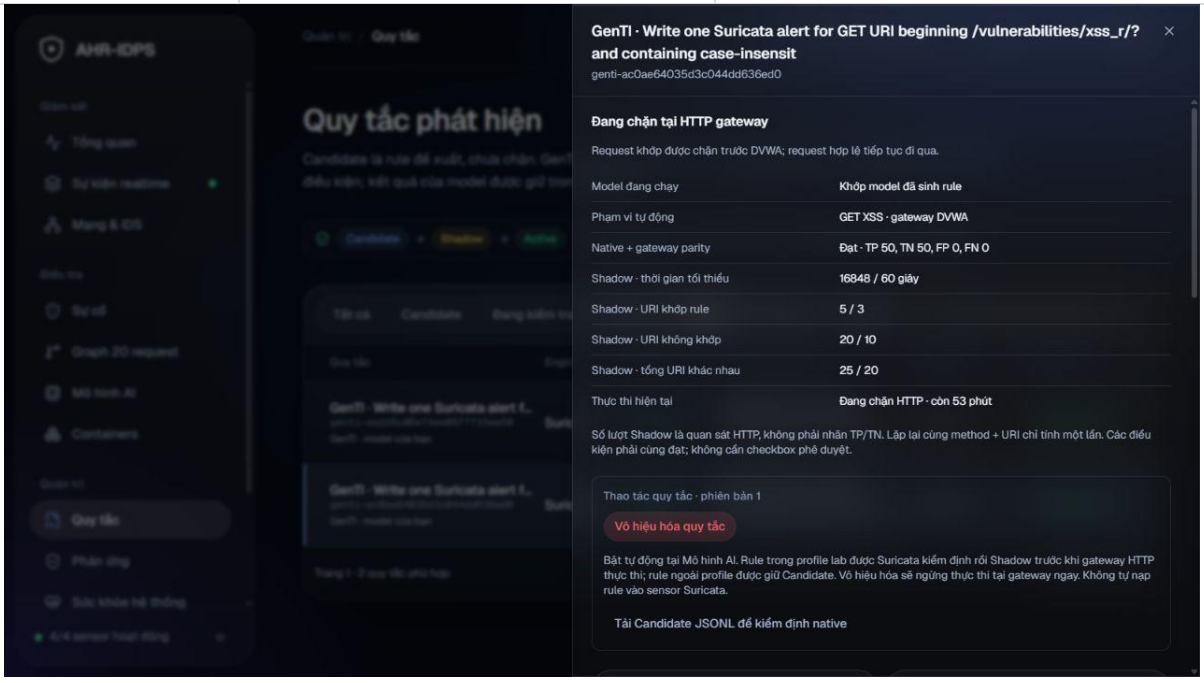
Nếu native không đạt, rule giữ Candidate và có audit auto-native-failed. Nếu đạt, worker tạo validation run và cập nhật Shadow có điều kiện theo status/version. Thao tác registry thủ công không được dùng làm đường tắt Active cho GenTI.

Shadow không tự chặn nhưng ghi nhận URI khớp và không khớp. Ngưỡng đồng thời là ít nhất 60 giây, ít nhất 3 URI khớp, ít nhất 10 URI không khớp và tổng ít nhất 20 URI khác nhau. Method+URI lặp chỉ tính một lần. Mọi Shadow được ghi nhận trước khi một rule Active bất kỳ trả quyết định chặn, khắc phục tình huống rule Active đứng đầu khiến các Shadow phía sau không đủ dữ liệu.

Active có lease một giờ và ghi cơ chế deployment là nginx-http-auth-request. Request GET khớp chữ ký được trả 403 trước DVWA; request không khớp tiếp tục đi qua. Worker heartbeat và hàm advance chạy riêng để vòng đời không bị đứng trong lúc model đang suy luận lâu. Hết lease, worker tự vô hiệu hóa và ghi audit; thao tác Disable cũng ngừng thực thi rule.

Bảng 4.8. Điều kiện chuyển trạng thái quy tắc tự động

Bước	**Điều kiện**	**Nếu chưa đạt**
Candidate	Đúng model, grammar và trạng thái	Giữ và giải thích lý do.
Native	Syntax, 50/50 case, FP=FN=0, parity	Giữ Candidate; lưu lỗi.
Shadow	60s, 20 URI, ≥3 khớp, ≥10 không khớp	Quan sát tiếp, không tự chặn.
Active	Native còn khớp, Shadow đủ, policy bật	Chặn HTTP theo lease.
Disable/hết hạn	Người dừng hoặc lease hết	Ngừng chặn, giữ audit.



Hình 4.8. Chi tiết rule Active: model, native, bộ đếm Shadow và thời hạn thực thi. Không yêu cầu checkbox phê duyệt GenTI.

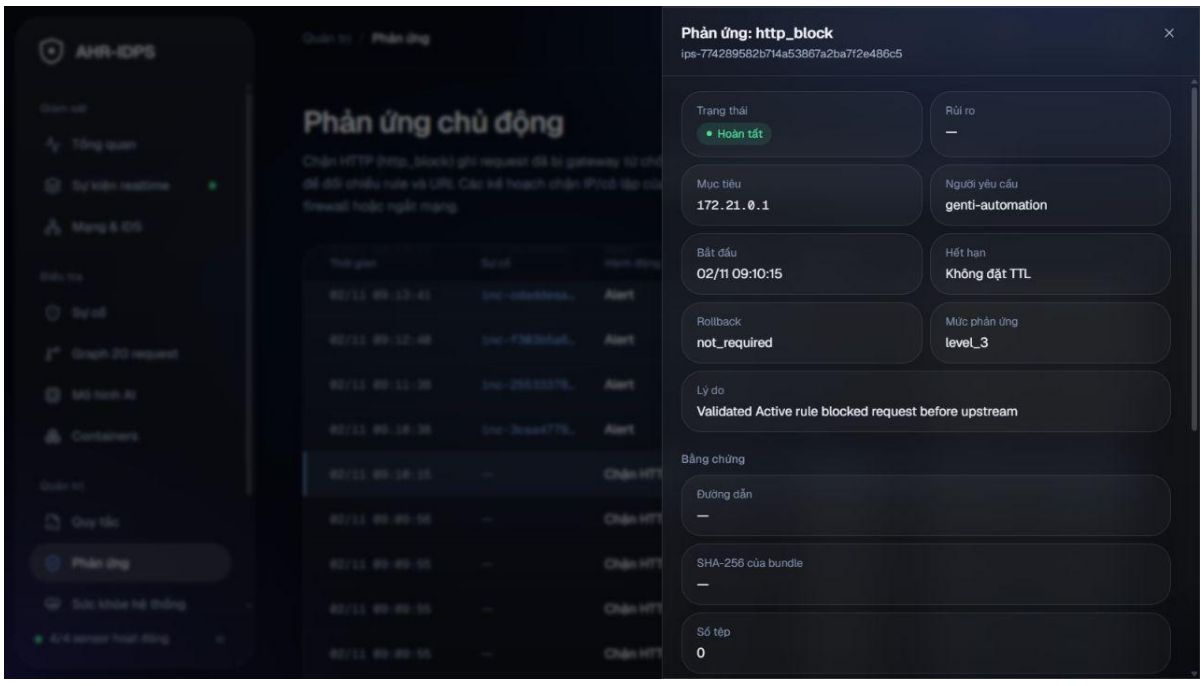
Các ngưỡng Shadow là điều kiện quan sát đã cấu hình, không phải nhãn xác nhận ý định của người dùng. Ba URI khớp chỉ nói matcher nhận ra ba URI khác nhau; chúng không tự chứng minh ba true positive. Để đo DR hoặc FPR live, cần một lớp nhãn độc lập ngoài các bộ đếm này.

4.5. Enforcement IPS evidence và điều khiển vận hành

Quyết định chặn cần được nối với response, dấu vết upstream và audit của đúng rule. Bảo toàn evidence, thông báo và khả năng dừng tạo điều kiện để kiểm chứng tác động và xử lý ngoại lệ trong cùng luồng vận hành.

Response Engine mặc định có đường audit hoặc alert_only cho các hành động không được cho phép. Trong triển khai chính đã kiểm chứng, đường chặn thật là Nginx gọi API kiểm tra nội bộ trước khi proxy tới DVWA. Hai loại phản ứng cần có cách đọc riêng trên dashboard để người vận hành không coi một record audit là đã block máy hoặc container.

Luồng IPS 12 bước kiểm tra các request attack bị 403 và đối chứng được chuyển tiếp. Log Nginx của mẫu chặn không có upstream tới DVWA; mẫu đối chứng có upstream. Header X-AHR-Rule, response http_block và audit ips-block nối quyết định với rule thực thi. Đây là bằng chứng trực tiếp hơn một thông báo “đã Active” trên registry.



Hình 4.9. Ảnh Phản ứng đã lưu trong hồ sơ vận hành ngày 02/11. Việc chặn được kết luận từ response cùng log upstream, không chỉ từ ảnh giao diện.

Khi policy bật, checker xác nhận worker và model có trạng thái hợp lệ, rule Active còn lease và đúng identity. Request thuộc profile nếu khớp thì trả 403 kèm dấu vết kiểm tra; request ngoài phạm vi không được diễn giải gần đúng theo rule. Khi hạ tầng lỗi, status 5xx do subrequest phải được báo là lỗi checker. Thống kê hiệu quả phòng vệ cần loại nhóm này khỏi số chặn đúng và báo riêng mức ảnh hưởng khả dụng.

Evidence bundle manifest và SHA 256. Evidence cần bao gồm raw event, event chuẩn, Candidate nguyên văn, rule sau cấp SID, model identity, receipt native, điều kiện Shadow và response. Mỗi phép thử có thư mục đầu ra mới theo mốc thời gian để không ghi đè hồ sơ cũ. Khi chỉ lưu ảnh dashboard, người đọc không thể tự tính lại confusion count hoặc xác minh rule nào đã thực thi.

Manifest liệt kê đường dẫn tương đối, kích thước và SHA-256 của các tệp. Hash giúp phát hiện nội dung thay đổi sau khi xuất, nhưng không chứng minh nhãn là đúng hoặc dữ liệu do cảm biến thật tạo ra. Vì vậy manifest phải đi cùng provenance, đơn vị đo và mô tả cách thu. Một gói có checksum đầy đủ nhưng không nêu synthetic hay independent vẫn chưa đủ cho đánh giá khoa học.

Mật khẩu, token và URL tải tạm có quyền truy cập không thuộc evidence công khai. Nếu cần tái lập một lỗi xác thực, chỉ giữ loại lỗi, thời gian và đường dẫn API đã che thông tin. Không đưa toàn bộ secrets vào ZIP chỉ để làm gói có vẻ đầy đủ.

Cảnh báo và can thiệp vận hành. Thông báo dashboard hiển thị khi có hành động mới; trang AI giữ các audit gần nhất. Khi toàn dashboard tạm dừng refresh, cảnh báo cũng tạm dừng. Chưa cấu hình email, Telegram, SMS hay thông báo hệ điều hành khi đóng trình duyệt. Không ghi rằng hệ thống đã gửi cảnh báo ra kênh bên ngoài nếu chưa có cấu hình và biên nhận gửi.

Nút dừng tự động ở trang AI là thao tác toàn profile, gỡ chặn tự động của các request mới. Disable trong chi tiết rule chỉ dừng rule được chọn; nếu policy vẫn bật, evidence mới có thể tạo một rule khác. Khi cần dừng hẳn lab, dừng container và Docker/WSL nhưng giữ volume và artifact.



Hình 4.10. Chi tiết điều kiện tự động với model, native, Shadow và thời hạn chặn.

Khi một Candidate ngoài profile bị giữ lại, thông báo cần cho biết lý do cụ thể để người vận hành biết đó là giới hạn bảo vệ chủ động. Tự động hóa không nên giấu lỗi trong một trạng thái chung như pending vô thời hạn. Các sự kiện đáng theo dõi gồm generation failed, native failed, auto-held, chuyển Shadow, chuyển Active, hết lease và dừng chính sách.

Thông báo trên dashboard chỉ hữu ích khi trang còn mở và cơ chế làm mới hoạt động. Hệ thống hiện chưa có bằng chứng nghiệm thu email, Telegram, SMS hay thông báo hệ điều hành khi trình duyệt đóng. Các kênh đó là công việc mở rộng riêng, không được ghi là chức năng đã có chỉ vì giao diện có danh sách cảnh báo.

CHƯƠNG 5. CƠ SỞ TOÁN HỌC VÀ THUẬT TOÁN GENTI OOF

Chương này trình bày cơ sở toán học của phần sinh quy tắc và phần tổng hợp phát hiện trước khi mô tả dataset. Mục 5.3 diễn giải Algorithm 1 của GenTI gốc, công thức đánh giá trong paper và ranh giới với triển khai AHR-IDPS. Các mục về SFT, QLoRA, chọn mẫu, Shadow và Active giải thích cấu hình của đề tài; OOF tại mục 5.4 tạo dự đoán ngoài fold để huấn luyện Fusion, không sinh rule và không phải bước fine-tune LLM. Trình bày phương pháp gốc không đồng nghĩa đã tái lập kết quả thực nghiệm của paper [9].

5.1. Mô hình hóa bài toán và hợp đồng đầu ra

Mô hình toán bắt đầu từ việc phân tách phân loại event với sinh quy tắc. Ký hiệu, cấu trúc input và hợp đồng output xác định đối tượng tối ưu và giúp tránh đánh đồng khả năng sinh một chuỗi hợp lệ với hiệu quả phòng vệ.

Bảng 5.1. Ký hiệu và phạm vi bài toán

Ký hiệu	Ý nghĩa	Phạm vi
e và x	Event chuẩn và vector được phép học	Phát hiện từ telemetry
y	Nhãn oracle dùng huấn luyện hoặc đánh giá	Không đưa vào input suy luận
g	Group hoặc chiến dịch của event	Tách dữ liệu và OOF
q E B C	Yêu cầu, evidence, benign, CTI nếu có	Input của GenTI
r và a	Rule và chuỗi JSON assistant	Đầu ra tác vụ sinh
θ và φ	Tham số model nền và adapter	Huấn luyện LLM
f m và H	Các base model và meta-model Fusion	Phân loại event
K	Số fold ngoài nhóm	K bằng 3 trong run đã đối chiếu

Với phát hiện, mục tiêu là ánh xạ một event sang điểm rủi ro hoặc nhãn. Với GenTI, mục tiêu là ánh xạ yêu cầu và quan sát sang một chương trình đối sánh có cấu trúc. Mô hình ngôn ngữ không trực tiếp tối ưu số request bị chặn trong quá trình SFT;

nó tối ưu xác suất token target. Khoảng cách giữa token target và hiệu quả phòng vệ được xử lý bằng các tầng kiểm định bên ngoài.

Mô hình hóa đầu vào và đầu ra GenTI. Một mẫu huấn luyện sinh rule được viết dưới dạng:

$$d_i = (q_i, E_i, B_i, C_i, a_i).$$

Trong đó a là câu trả lời JSON chứa rule, summary và thông tin ATT&CK theo hợp đồng. E và B phải là những quan sát thích hợp với yêu cầu. CTI có thể rỗng; hệ thống không tự thêm một tham chiếu không có thật để hoàn thành trường dữ liệu. Target a chỉ có trong phần assistant của mẫu SFT, không được sao chép vào prompt x khi đánh giá.

Prompt được dựng qua cùng tokenizer và chat template của model:

$$x_i = \text{Template}(S, q_i, E_i, B_i, C_i).$$

S là chỉ dẫn hệ thống. Sau suy luận, chương trình ứng dụng phân tích JSON và kiểm schema. Một chuỗi đúng ngữ pháp JSON nhưng có field không được phép, rule nhiều dòng hoặc phụ thuộc tài nguyên ngoài vẫn có thể bị từ chối theo contract. Vì vậy ta phân biệt tập chuỗi hợp lệ về JSON và tập đầu ra hợp lệ của ứng dụng.

$$\mathcal{A}_{contract} \subseteq \mathcal{A}_{JSON}.$$

LLM thực hiện phân bố xác suất trên chuỗi token, còn backend thực hiện ràng buộc cấu trúc sau giải mã. Việc dùng temperature bằng 0 và seed cố định hỗ trợ tái lập trong runtime hiện có, nhưng không chứng minh tính đúng ngữ nghĩa hoặc sự đồng nhất tuyệt đối qua mọi engine lượng tử.

5.2. Tối ưu SFT cập nhật QLoRA và chương trình học

Loss có mask, cập nhật hạng thấp và chương trình học cùng xác định cách adapter được tối ưu. Các công thức làm rõ phần tham số được học, vai trò validation và giới hạn của việc sử dụng token loss làm chỉ số chất lượng.

Với chuỗi đáp án có T token, xác suất tự hồi quy được phân tích thành:

$$p_{\theta,\phi}(a \mid x) = \prod_{t=1}^{T} p_{\theta,\phi}\,(a_t \mid x, a_{<t}).$$

Gọi M là tập vị trí token assistant không bị mask. Hàm mất mát trên một mẫu là:

$$\ell_i = -\frac{1}{|M_i|}\sum_{t\in M_i} \log p_{\theta,\phi}(a_{i,t} \mid x_i, a_{i,<t}).$$

Prompt và padding được gán label bỏ qua, trong triển khai dùng giá trị -100. Điều này làm model học câu trả lời thay vì được thưởng vì lặp lại input. Khi batching, cần giữ đúng mask và attention mask; nếu prompt bị tính vào loss, một phần cải thiện có thể chỉ đến từ dự đoán các đoạn lặp dễ của yêu cầu.

Hàm mục tiêu tổng quát trên N mẫu là trung bình mất mát theo quy ước weighting đã chọn. Không tự coi trung bình loss từng batch là trung bình per-token của toàn bộ corpus nếu số token mỗi batch khác nhau. Khi so sánh run, cần giữ tokenizer, masking và cách tổng hợp metric.

Loss thấp đo mức khớp với target của corpus. Nó không kiểm engine, oracle PCAP hoặc phạm vi chặn. Một target sai vẫn có thể được học với loss thấp. Do đó loss được dùng chọn checkpoint theo validation, còn chất lượng vận hành cần những phép đo khác.

QLoRA và cập nhật hạng thấp. Đối với một phép biến đổi tuyến tính, LoRA biểu diễn phần cập nhật bằng hai ma trận hạng thấp: [5]

$$W' = \widetilde{W} + \frac{\alpha}{r}BA.$$

Ở đây W̃ là trọng số nền lượng tử được dùng trong forward, A có kích thước r nhân d_in và B có kích thước d_out nhân r. Các tham số adapter được tối ưu trong khi trọng số nền được giữ theo cơ chế fine-tune. Trong artifact của đề tài, r bằng 16, alpha bằng 32 và dropout bằng 0,05; các projection đích là q, k, v và o của attention.

$$N_{LoRA} = r(d_{in} + d_{out}).$$

$$N_{full} = d_{in}d_{out}.$$

Khi r nhỏ so với hai chiều ma trận, phần tham số cần cập nhật giảm đáng kể. Đây là công thức cho một ma trận, không phải số tham số trainable chính xác của cả model. Muốn báo con số toàn model phải đếm đúng các module đã gắn adapter và đối chiếu dtype, optimizer state, activation và bộ nhớ thực.

NF4-double trong run làm giảm yêu cầu bộ nhớ trọng số nền, nhưng không loại bộ nhớ activation hoặc optimizer. Vì vậy tăng context có thể làm chi phí tăng dù rank

giữ nguyên. Corpus thực tế được phân tích ở chương dataset cho phép chọn context dựa trên token thay vì suy từ số ký tự.

Chương trình học và lựa chọn checkpoint. Gọi d_i là mức khó của mẫu train và s là stage. Tập dùng ở stage s là:

$$D_s = \{i \in D_{train} : d_i \leq s\}, \qquad s \in \{1,2,3,4\}.$$

Adapter được cập nhật liên tiếp trên các tập tích lũy. Validation giữ cố định để việc so sánh stage có cùng căn cứ. Checkpoint được chọn theo:

$$s^* = \arg\min_{s \in \{1,2,3,4\}} L_{val}(\phi_s).$$

Nếu hai stage có cùng giá trị theo độ chính xác lưu, cần một quy tắc tie-break được công bố thay vì chọn theo test. Run của đề tài ghi stage 4 được chọn; các con số loss cụ thể được đối chiếu ở chương dữ liệu và kết quả. Công thức này không khẳng định curriculum tốt hơn mọi cách train trộn; kết luận so sánh cần một đối chứng riêng.

Thuật toán 5.1 Fine tune GenTI theo chương trình học

1. Kiểm corpus, target, provenance và family.
2. Khóa train, validation, test và unknown_test trước tối ưu.
3. Tokenize bằng template của model; mask prompt và padding.
4. Với s từ 1 đến 4, lấy D_s và tiếp tục cập nhật cùng adapter.
5. Lưu checkpoint và tính loss trên validation cố định.
6. Chọn s sao cho validation loss nhỏ nhất; khóa lựa chọn.
7. Sinh raw output của test và unknown_test một lần theo cấu hình đã chốt.
8. Lưu model identity, tokenizer, raw output và cấu hình kiểm định.

Không dùng test để chọn stage rồi vẫn gọi test là đánh giá kín. Nếu đầu ra test được phân tích để sửa prompt hoặc target, cần chuyển tập đó sang vai trò development và giữ một tập kín khác cho kết quả cuối.

5.3. Thuật toán GenTI nguyên bản và áp dụng trong hệ thống

GenTI nguyên bản và phạm vi áp dụng trong AHR IDPS

Nguồn phương pháp là bài "GenTI: Benchmarking LLMs for Autonomous IDPS Rule Generation for Unseen Attacks", Hassan Jalil Hadi, Rehana Yasmin và Ali Shoker, arXiv:2606.05844, công bố ngày 04/06/2026. Phần này được đối chiếu với

Algorithm 1, các mục IV–VI của bản gốc và bản dịch tiếng Việt. GenTI là một quy trình tạo, sửa và kiểm chứng quy tắc có sử dụng LLM; GTI là corpus phục vụ quy trình đó. Một lần gọi mô hình sinh được chuỗi rule chưa đồng nghĩa với hoàn thành toàn bộ GenTI.

Dữ liệu phục vụ phương pháp gốc

GTI tổ chức khoảng 150 nghìn quy tắc IDPS và 50 nghìn quy tắc YARA. Nhóm IDPS tổng hợp chữ ký Snort, Suricata và nguồn cộng đồng; nhóm YARA bổ sung các mẫu nhận diện ở cấp payload. Đơn vị học là quy tắc gắn ngữ cảnh, yêu cầu và chú giải, không phải một dòng sự kiện mạng. Vì vậy, số sự kiện thu thập trong AHR-IDPS không được dùng thay cho số mẫu rule của GTI, và quy mô GTI không được ghi thành quy mô dữ liệu đã huấn luyện của đề tài.

Quy trình xây dựng corpus bắt đầu bằng thu thập, chuẩn hóa và loại trùng. Bộ phân tích tách header, chiều luồng, giao thức, cổng, điều kiện payload của rule IDPS; với YARA, cấu trúc điều kiện được biểu diễn bằng cây cú pháp. Từ đó hệ thống rút IoC, CVE, CWE và họ mối đe dọa, tra OTX, lưu thông tin theo STIX/TAXII, ánh xạ ATT&CK và D3FEND, rồi gắn thuộc tính, sự kiện, tag, galaxy và nguồn gốc qua MISP. Các trường chất lượng, mức nghiêm trọng và độ tin cậy đi cùng thông tin xuất xứ để hỗ trợ chọn lọc mẫu.

Paper còn sử dụng LLM trong làm giàu corpus: Qwen 7B tạo CTI tổng hợp khi thiếu IoC, LLaMA 3.2 7B hỗ trợ xây dựng prompt, DeepSeek 7B hỗ trợ diễn đạt và phần giải thích. CTI tổng hợp phải được nhận diện đúng là nội dung sinh, không tự trở thành tình báo đã xác minh. Lược đồ GTI có 48 trường được phân nhóm theo thông tin quy tắc, bối cảnh và chú giải. Sản phẩm cuối cùng ghép yêu cầu đầu vào, ngữ cảnh, cấu trúc rule và các thông tin phản hồi, cho phép mô hình học cả sinh rule và kiểm tra rule.

Huấn luyện trong paper

Corpus được chia phân tầng 80% train, 10% validation và 10% test, xét nguồn rule, ATT&CK, mức nghiêm trọng và độ khó; một rule chỉ thuộc một phân vùng. Cách chia này không tự bảo đảm toàn bộ họ tấn công của test vắng mặt trong train. Khi đánh giá khả năng tổng quát hóa theo họ, cần một thiết kế tách nhóm riêng và công bố rõ tiêu chí đó.

CoT trong phương pháp gốc bổ sung phần diễn giải logic phát hiện và liên hệ ngữ cảnh vào ví dụ học. CoVe bổ sung ví dụ xác minh cú pháp, logic và dương tính giả. Khi triển khai, những giải thích có thể kiểm tra phải gắn với evidence, lỗi parser và kết quả replay; một lời giải thích trôi chảy không thay được kết quả chạy engine.

Chương trình học gồm bốn giai đoạn nối tiếp: đầu tiên học rule đơn giản và header cơ bản; sau đó thêm ngữ cảnh CTI và giải thích đơn giản; tiếp theo tăng các ví dụ khó định hướng kiểm chứng CoVe; cuối cùng học trên toàn dải độ khó. Trọng số ở cuối giai đoạn trước khởi tạo cho giai đoạn sau. Độ khó xét số ràng buộc header và payload, mức phong phú CTI và ví dụ suy luận hay xác minh đi kèm.

Thí nghiệm của tác giả dùng DeepSeek 7B, QLoRA, tám GPU NVIDIA A100, tốc độ học 10^{-4} và dừng sớm theo điểm validation. Đây là cấu hình của paper. Cấu hình Qwen chạy với GPU L4 trong đề tài là lựa chọn triển khai riêng; không được mô tả như đã tái lập nguyên vẹn thí nghiệm tám A100.

Đầu vào và các biến của thuật toán

Gọi P là yêu cầu của nhà phân tích; S là tập payload đại diện; $\mathcal{C}$ là kho CTI. Quy tắc ở lần lặp thứ k ký hiệu r_k; ngữ cảnh đe dọa là c_k; phần diễn giải đi kèm được ký hiệu C_k. Hai ký hiệu $\mathcal{C}$ và C_k chỉ những đối tượng khác nhau. Đầu ra là cặp quy tắc đã qua xác minh $\hat{r}$ và ngữ cảnh $\hat{c}$.

Trình tự đầy đủ của Algorithm 1

1. BuildPrompt xử lý P, S để tạo prompt có cấu trúc, prompt theo yêu cầu người dùng và tập đặc trưng nổi bật F_0. Đặc trưng có thể mô tả giao thức, hướng luồng và dấu hiệu payload cần phát hiện. Đây là bước chuyển yêu cầu nghiệp vụ thành đầu vào mà mô hình có thể dùng.

2. LLMGenerate nhận hai prompt cùng F_0, tạo bản rule đầu tiên r_0 và diễn giải C_0. Bản rule này mới là ứng viên. Hành động alert, drop hoặc reject trong chuỗi ký tự chưa chứng minh sensor đã nạp hay thực thi được rule.

3. MapCTI đối chiếu r_0, S với kho $\mathcal{C}$ để tạo c_0. Ngữ cảnh giúp kiểm tra rule đang phát hiện hành vi nào, liên quan kỹ thuật hay lỗ hổng nào và vì sao nội dung đó phù hợp. Liên kết CTI phải có căn cứ nguồn, không suy ra một mã ATT&CK chỉ từ tên gọi tấn công.

4. Khởi tạo $k = 0$. Khi cặp rule và ngữ cảnh chưa hội tụ, EvaluateRule đánh giá r_k trên mẫu thử và tạo vector M_k. Algorithm 1 liệt kê DR, FPR, CPU và độ bao phủ. Phần mô tả của paper yêu cầu lưu lượng độc hại lẫn lành tính; nếu chỉ có tấn công, không thể xác định FPR.

5. Nếu MeetsThresholds chấp nhận M_k, thoát vòng lặp. Nếu chưa đạt, BuildFeedback biến lỗi và các chỉ số thành phản hồi tập trung: bỏ sót mẫu nào, nhầm mẫu lành nào hoặc ràng buộc nào chưa phù hợp. LLMRefine dùng prompt ban đầu, diễn giải hiện tại và phản hồi mới để tạo r_{k+1}, C_{k+1}. UpdateCTI cập nhật ngữ cảnh theo rule mới, tăng k rồi quay lại đánh giá.

6. Sau vòng tinh luyện, CoVe kiểm tra theo thứ tự SyntacticOK, LogicOK và FPAnalysisOK. Cổng thứ nhất yêu cầu cú pháp được chấp nhận. Cổng thứ hai xét ý định phát hiện và tính nhất quán với ngữ cảnh. Cổng thứ ba kiểm tra nguy cơ dương tính giả trên traffic lành. Trong mã giả Algorithm 1, bất kỳ cổng nào thất bại đều trả RejectRule và kết thúc.

7. Nếu vượt cả ba cổng, gán $\hat{r} = r_k$, dùng BuildContext tạo $\hat{c}$ từ diễn giải và CTI, rồi trả về cặp kết quả. Ngữ cảnh đầu ra gồm giải thích, báo cáo liên hệ MITRE và hướng dẫn triển khai ngắn. Việc đưa rule vào vị trí chặn của một hạ tầng cụ thể cần cơ chế triển khai riêng.

Mã giả diễn giải tương ứng với paper

Đầu vào: yêu cầu P, payload S, kho CTI.

BuildPrompt(P,S) → P_struct, P_user, F(0).

LLMGenerate(P_struct,P_user,F(0)) → r(0), C(0).

MapCTI(r(0),S,kho CTI) → c(0); đặt k = 0.

Lặp khi cặp (r(k),c(k)) chưa hội tụ:

– EvaluateRule(r(k),S) → M(k).

– Nếu MeetsThresholds(M(k)), thoát vòng lặp.

– BuildFeedback(M(k)) → F(k+1).

– LLMRefine(P_struct,P_user,C(k),F(k+1)) → r(k+1), C(k+1).

– UpdateCTI(r(k+1),kho CTI) → c(k+1); tăng k.

Sau vòng lặp: nếu SyntacticOK sai → RejectRule.

Nếu LogicOK sai → RejectRule; nếu FPAnalysisOK sai → RejectRule.

Ngược lại trả r(k) và BuildContext(C(k),c(k)).

Mã giả trên giữ nguyên các nhánh quyết định của Algorithm 1. Phần diễn giải trong paper có đề cập việc quay lại tinh luyện khi kiểm chứng thất bại, nhưng Algorithm 1 không vẽ nhánh quay lại đó mà trả RejectRule. Nếu xây dựng cơ chế thử lại sau CoVe, cần công bố đó là lựa chọn triển khai. Paper không chỉ rõ ngưỡng số, số vòng tối đa, thời gian chờ hay định nghĩa hội tụ có thể thực thi; các giới hạn này phải được bổ sung và ghi rõ trong một hệ thống vận hành.

Một ví dụ để đọc thuật toán

Ví dụ minh họa giả định, không phải kết quả đo của AHR-IDPS: yêu cầu là phát hiện một mẫu payload script trong truy vấn của endpoint thử nghiệm. Bộ mẫu có 10 request tấn công và 10 request lành. Rule đầu tiên dùng dấu hiệu quá rộng nên bắt 10 request tấn công nhưng nhầm 4 request lành: TP = 10, FN = 0, FP = 4, TN = 6; DR = 100% và FPR = 40%. Khả năng bắt đủ tấn công trong ví dụ này chưa đủ để chấp nhận rule.

Phản hồi cho mô hình nêu rõ bốn request bị nhầm và yêu cầu ràng buộc endpoint, hướng luồng và nội dung phù hợp hơn. Rule sửa lại bắt 9 request tấn công và không bắt request lành: TP = 9, FN = 1, FP = 0, TN = 10; DR = 90%, FPR = 0%. Mẫu tấn công còn bỏ sót phải được giải thích hoặc dùng để tinh luyện tiếp theo chính sách đã công bố. Không thể kết luận rule tốt chỉ vì FPR giảm; ngưỡng chấp nhận phải xét cả DR.

Tiếp đến engine kiểm tra cú pháp, người hoặc bộ xác minh kiểm tra logic với evidence và CTI, rồi replay benign để kiểm tra FP. Chỉ khi ba cổng đều đạt mới có rule đã xác minh. Đối với AHR-IDPS, rule còn đi qua vòng đời Candidate → kiểm chứng native → Shadow → Active và enforcement tại gateway. Phần vòng đời này được trình bày như sự bổ sung của hệ thống.

Các công thức đánh giá trong paper

Với N rule, $\mathbf{1}\{\cdot\}$ là hàm chỉ báo. Syntax Accuracy là tỷ lệ rule được parser của engine chấp nhận:

$$SA = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\{ParseOK(\hat{r}_i)\}.$$

Semantic Similarity đối chiếu tập trường F như giao thức, chiều, cổng, flow, content và action. Đây là tỷ lệ trường phù hợp với rule tham chiếu, không phải độ tương đồng cosine giữa hai câu:

$$SS_i = \frac{1}{|F|}\sum_{f \in F} \mathbf{1}\{f(\hat{r}_i) = f(r_i)\}, \qquad SS = \frac{1}{N}\sum_{i=1}^{N} S\,S_i.$$

CTI Coverage dùng Jaccard giữa tập kỹ thuật ATT&CK chuẩn T_i và tập dự đoán $\hat{T}_i$:

$$CC_i = \frac{|T_i \cap \hat{T}_i|}{|T_i \cup \hat{T}_i|}, \qquad CC = \frac{1}{N}\sum_{i=1}^{N} C\,C_i.$$

Paper không quy định rõ trường hợp cả hai tập rỗng. Khi triển khai phải công bố cách xử lý mẫu thiếu chú giải; không mặc nhiên ghi độ bao phủ hoàn hảo cho dữ liệu không có CTI chuẩn.

Hiệu quả phát hiện và dương tính giả được đo bằng replay có nhãn:

$$DR = \frac{TP}{TP + FN}, \qquad FPR = \frac{FP}{FP + TN}.$$

$$SE = \alpha DR + (1 - \alpha)(1 - FPR), \qquad \alpha = 0.5.$$

Điểm tổng hợp trong công thức (8) của paper là:

$$CS = 0.2SA + 0.3SS + 0.2MC + 0.3SE.$$

Ký hiệu MC ở công thức tổng hợp không thống nhất với CC được định nghĩa trước đó. Có thể hiểu đây là thành phần độ bao phủ CTI/MITRE, nhưng đó là diễn giải; báo cáo giữ nguyên ký hiệu gốc và nêu rõ điểm chưa thống nhất. CPU được theo dõi riêng, không xuất hiện như một số hạng trong SE hay CS. Khi thiếu chuẩn CTI, rule tham chiếu hoặc replay có nhãn, chưa đủ căn cứ tính đầy đủ CS.

Ranh giới giữa paper và kết quả của đề tài

Trong AHR-IDPS, LLM fine-tune sinh candidate; bộ kiểm chứng native đánh giá trước khi cho phép đi tiếp. Shadow quan sát đối sánh trước khi Active; Active chỉ có tác dụng chặn khi tầng enforcement nhận đúng rule và áp dụng trên đường request. Suricata phát alert ở vai trò IDS khác với gateway từ chối request ở vai trò IPS. Cần dùng evidence về request, rule, trạng thái áp dụng và phản hồi gateway để chứng minh hành động chặn, không chỉ dùng nhãn Active trên dashboard.

Việc AI chọn evidence và benign là một cơ chế chọn mẫu của ứng dụng. Nhãn do AI suy ra phải được ghi nguồn và độ bất định; một mẫu được mô hình gọi là benign chưa phải ground truth độc lập. Các cổng xác minh, nguồn CTI, vòng lặp sửa rule và số liệu native phải được thể hiện theo đúng mức thực hiện; không ghi rằng đã tái lập toàn bộ GenTI nếu mới triển khai một phần quy trình.

OOF ở mục 5.4 là phương pháp tạo dự đoán ngoài tập huấn luyện cho tầng Fusion của đề tài, không phải bước trong Algorithm 1 của GenTI. OOF hỗ trợ hạn chế rò rỉ khi huấn luyện bộ kết hợp phân loại. GenTI học và kiểm chứng quy tắc. Hai bài toán có thể phối hợp nhưng cần dataset, nhãn và cách đánh giá riêng. Những biểu thức chọn mẫu, Shadow và điều kiện Active dưới đây mô hình hóa AHR-IDPS, không phải công thức do tác giả GenTI công bố.



Hình 5.1a. Trình tự GenTI gốc và ranh giới với vòng đời rule của AHR-IDPS

Chu trình GenTI được biểu diễn qua lựa chọn mẫu, generation và các cổng xác minh. Ràng buộc slot, grammar, native, Shadow và lease giúp nối đầu ra LLM với điều kiện thực thi có thể kiểm tra.

Pool quan sát P được tạo từ dữ liệu HTTP mới đã làm sạch và đánh số slot. Model trả hai tập chỉ số I_E và I_B cùng task và reason. Các slot còn lại được coi là chưa được chọn hoặc uncertain:

$$I_E, I_B \subseteq \{0, \dots, |P| - 1\}.$$

$$I_E \cap I_B = \varnothing.$$

$$I_U = \{0, \dots, |P| - 1\} \backslash (I_E \cup I_B).$$

Trong triển khai, mỗi nhóm được chọn tối đa ba slot từ pool tối đa sáu quan sát; slot phải là số nguyên, không lặp và tồn tại. Các quan sát được chọn cần cùng endpoint theo kiểm tra backend. Phải có ít nhất một evidence và một benign để tiếp tục generation. Những ràng buộc này kiểm tính nhất quán cấu trúc, không tự xác nhận model gán vai trò đúng về ngữ nghĩa.

Thuật toán 5.2 Review rồi sinh Candidate

1. Kiểm policy, model ready và boot smoke của phiên hiện tại.

2. Lấy HTTP GET DVWA trong cửa sổ mới; lọc nguồn và khử URI lặp.

3. Tạo pool cùng endpoint theo shortlist, làm sạch field nhạy cảm.

4. Gọi model chính với schema review; kiểm slot và hai nhóm.

5. Nếu thiếu vai trò hoặc sai hợp đồng, lưu insufficient hoặc failed, chưa sinh rule.

6. Dựng yêu cầu generation từ dữ liệu đã chọn và profile cho phép.

7. Gọi model sinh một output; kiểm contract và identity không đổi.

8. Cấp SID theo quy trình, lưu Candidate nguyên văn và provenance.

Pool fingerprint và khóa generation giảm xử lý lặp hoặc chồng lấn. Cooldown hiện có giới hạn tần suất chu kỳ; nó không phải thời gian model bắt buộc hoàn thành. Trong kịch bản quay, cần phân biệt đang chờ cooldown, đang review và đang generation thay vì xem mọi khoảng đợi là lỗi.

Các cổng kiểm định và thực thi. Cổng hợp đồng và native. Đặt C(r) là điều kiện hợp đồng, G(r) là grammar của profile, S(r) là syntax native đạt và P(r) là parity trên bộ case. Cổng native tự động của triển khai chính yêu cầu:

$$V(r) = C(r) \wedge G(r) \wedge S(r) \wedge P(r) \wedge Z(r).$$

Z là điều kiện control có ít nhất 50 attack, 50 benign với FP và FN bằng không theo receipt. Receipt phải gắn nội dung rule cuối, version, model artifact và engine. Đây là điều kiện cho bộ control được sử dụng, không phải một định lý rằng rule không bao giờ báo nhầm trên mọi traffic.

Cổng Shadow. Gọi t_0 là thời điểm bắt đầu Shadow; n_+ và n_- là số method–URI khác nhau được matcher phân vào hai nhóm. Điều kiện chuyển tự động là:

$$H(r,t) = [t - t_0 \geq 60] \wedge [n_+ \geq 3].$$

$$H(r,t) \leftarrow H(r,t) \wedge [n_- \geq 10] \wedge [n_+ + n_- \geq 20].$$

Ký hiệu gán ở dòng thứ hai chỉ nhằm tách công thức cho dễ đọc; trong mã đây là một biểu thức boolean gồm bốn điều kiện. Bộ đếm dùng tập định danh nên gửi lại cùng một method và URI không tạo thêm quan sát độc lập. n_+ là match count, không phải TP; n_- là nonmatch count, không phải TN khi chưa có oracle live.

Match và lease. Với profile chính, matcher yêu cầu method bằng GET, URI sau percent-decode một lần bắt đầu bằng đường dẫn đã định và chứa literal script mục tiêu không phân biệt hoa thường. Dấu cộng không được tự đổi thành dấu cách trong phép chuẩn hóa URI này. Những semantics khác grammar không được dịch gần đúng để chặn.

Đặt A(r,t) là Active, L(r,t) là lease còn hiệu lực, Q(t) là policy và checker sẵn sàng, M_r(h) là matcher trên request h. Quyết định chặn do rule có thể viết:

$$B_r(h,t) = A(r,t) \wedge V(r) \wedge L(r,t) \wedge Q(t) \wedge M_r(h).$$

Nếu có ít nhất một rule hợp lệ thỏa điều kiện, gateway trả 403 và ghi audit trước khi chuyển tới DVWA. Khi checker lỗi, từ chối do hạ tầng được xử lý riêng, không được tính vào B_r như một true positive. Lease của profile là 3.600 giây; disable hoặc hết hạn làm ngừng thực thi và giữ evidence.

5.4. OOF Fusion và huấn luyện dự đoán ngoài nhóm

OOF tạo điểm cho meta-model từ những base model không học nhóm của mẫu được chấm. Cơ chế chia nhóm, vector meta, hàm mục tiêu và quy trình refit được trình bày cùng nhau để làm rõ khác biệt giữa train và inference.

Với event i, ba base model tạo điểm Network, Log và Graph. Nếu meta-model được train trên dự đoán in-sample của các base model, nó có thể học một bức tranh lạc

quan vì các model này đã nhìn thấy chính những mẫu đang được chấm. OOF thay các điểm đó bằng dự đoán từ model không fit trên nhóm của mẫu i. [10, 11]

OOF không sửa nhãn sai, không tự loại near-duplicate và không bảo đảm các group độc lập ngữ nghĩa. Nó xử lý một loại rò rỉ cụ thể giữa base training và meta-training. Các group cần được định nghĩa theo đơn vị có tương quan như chiến dịch hoặc phiên phù hợp, thay vì tùy tiện gán mỗi record một group riêng.

Chia nhóm và điều kiện độc lập. Cho D_train được chia thành K fold F_1 đến F_K. Các fold rời nhau và phủ toàn bộ train:

$$F_k \cap F_l = \varnothing \quad (k \neq l), \qquad \bigcup_{k=1}^{K} F_k = D_{train}.$$

Điều kiện theo group yêu cầu một group không nằm đồng thời ở hai fold:

$$g_i = g_j \Rightarrow \text{fold}(i) = \text{fold}(j).$$

Mã hiện tại dùng GroupKFold với K bằng 3 và kiểm mỗi tập fit có đủ hai lớp. Không mô tả cấu hình này như StratifiedGroupKFold hoặc random row split. Khi một nhóm quá lớn hoặc nhãn lệch, cần kiểm độ phủ fold thay vì mặc định mỗi fold có tỷ lệ lớp giống nhau.

Điểm ngoài fold. Với mỗi nhánh m và fold k, fit base model trên phần train còn lại:

$$f_m^{(-k)} = \text{Fit}_m(D_{train} \backslash F_k).$$

Với i thuộc fold k, điểm OOF là:

$$z_{im}^{OOF} = f_m^{(-k)}(x_i).$$

Mỗi mẫu nhận một bộ điểm từ các model không học nhóm của nó. Các phép fit phụ thuộc dữ liệu như scaler, classifier và Isolation Forest phải được fit trong phần train của fold. Các biến đổi cố định như hashing cần giữ cùng hợp đồng giữa fold và inference.

Bảng 5.2. Chấm OOF theo fold và nhóm

Fold được chấm	Nhóm dùng fit base model	Nhóm được nhận điểm OOF
F1	F2 và F3	Chỉ F1
F2	F1 và F3	Chỉ F2

Fold được chấm	Nhóm dùng fit base model	Nhóm được nhận điểm OOF
F3	F1 và F2	Chỉ F3

Bảng trên minh họa cơ chế, không phải bảng kết quả của một lượt train mới. Sau ba vòng, ma trận OOF có N_train hàng và ba cột base score. Event ID và fold ID được giữ trong manifest để kiểm không mẫu nào bị chấm thiếu hoặc bị điền bằng điểm in-sample.

Đặc trưng meta và hàm mục tiêu. Trong mã được đối chiếu, vector Fusion có tám trường theo thứ tự cố định:

$$u_i = [s_{rule}, z_{net}^{OOF}, z_{log}^{OOF}, z_{graph}^{OOF}, s_{ctx}, v_i, \delta_i, m_i].$$

v là severity chia 10; δ là density được chặn ở 25 rồi chia 25; m là số nguồn điểm thiếu chia 5 theo hàm feature_vector. Trong nhánh Colab đã kiểm, fusion_matrix truyền density bằng 0. Công thức chuẩn hóa không có nghĩa mọi run đều đo cùng một mật độ live; cần đọc cách tạo trường ở đúng đường train và inference.

Meta-model XGBoost học từ u và nhãn train. Hàm mục tiêu nhị phân có thể biểu diễn:

$$\mathcal{L}_{meta} = \sum_i \ell_{BCE}\,(y_i, H(u_i)) + \sum_j \Omega\,(T_j).$$

$$\ell_{BCE}(y, p) = -y\log p - (1-y)\log(1-p).$$

Các hạng regularization của cây kiểm soát độ phức tạp theo cấu hình XGBoost. Xác suất được so với threshold để tạo nhãn; run hiện có dùng 0,55 cho Fusion OOF. Báo cáo không coi threshold này là đã được tối ưu bằng test.

Weighted hybrid là baseline khác, sử dụng trung bình có trọng số của các nguồn đang có:

$$s_{weighted} = \frac{\sum_{j \in A} w_j\, s_j}{\sum_{j \in A} w_j}.$$

Các trọng số rule, network, log, graph và context trong nhánh đã đối chiếu lần lượt là 0,25; 0,25; 0,20; 0,25; 0,05. OOF là cách tạo dữ liệu cho meta-model, còn weighted hybrid là công thức tổng hợp cố định; không gọi hai cơ chế này là cùng một thuật toán.

Thuật toán OOF Fusion hoàn chỉnh. Thuật toán 5.3 Huấn luyện và suy luận OOF Fusion

1. Khóa train, validation và test theo group; loại field tiết lộ đáp án.

2. Khởi tạo ma trận điểm OOF có N_train hàng, ba cột và fold ID chưa gán.

3. Chia train thành ba GroupKFold.

4. Ở mỗi fold, fit Network, Log và Graph trên phần nhóm còn lại.

5. Chấm đúng phần giữ ngoài fold, ghi điểm vào hàng OOF tương ứng.

6. Kiểm mỗi event đã có đúng một bộ điểm ngoài fold và đủ provenance.

7. Ghép các điểm OOF với rule/context và các trường meta hợp lệ.

8. Fit meta-model XGBoost trên ma trận này và nhãn train.

9. Fit lại ba base model trên toàn train để tạo model inference cuối.

10. Chấm validation và test bằng base model cuối, dựng vector meta rồi qua meta-model đã học.

11. Báo metric trên split cố định, lưu model, feature schema và fold manifest.

Tại inference không cần tạo ba fold mới. Một event đi qua ba base model cuối, sau đó qua meta-model. Với K bằng 3 và ba nhánh, quy trình có chín lượt fit nhánh theo fold và ba lượt refit nhánh trước meta-model; mỗi nhánh có thể chứa nhiều estimator nội bộ. Con số này là số lượt huấn luyện thành phần theo thiết kế, không phải số model phải chạy đồng thời cho mỗi request.

Không đánh giá chất lượng tổng quát bằng cách đưa train qua các base model cuối rồi chấm meta như một test độc lập. Những base model đã fit toàn train và phân bố điểm có thể khác OOF. Validation, test và external holdout mới cung cấp căn cứ đánh giá với dữ liệu không dùng fit tương ứng.

5.5. Chỉ số đánh giá quan hệ thuật toán và giới hạn

Các chỉ số được gắn với đơn vị và mẫu số, sau đó đối chiếu với mục tiêu của từng thuật toán. Sự phân biệt này chỉ ra vì sao OOF, syntax, replay và cổng triển khai bổ sung nhau nhưng không thay thế nhau.

Với một đơn vị đánh giá có oracle, các chỉ số nhị phân là:

$$Precision = \frac{TP}{TP + FP}, \qquad DR = Recall = \frac{TP}{TP + FN}.$$

$$F_1 = \frac{2TP}{2TP + FP + FN}, \qquad FPR = \frac{FP}{FP + TN}.$$

Ở GenTI, cần thêm tỷ lệ qua hợp đồng và syntax theo toàn bộ task:

$$R_{contract} = \frac{N_{contract}}{N_{task}}, \qquad R_{syntax} = \frac{N_{syntax}}{N_{task}}.$$

R_syntax không bằng DR. Một rule đúng syntax vẫn có thể không tạo alert trên attack. Nếu chỉ báo syntax trên các mẫu đã qua contract, cần ghi rõ đó là tỷ lệ có điều kiện; không bỏ những task thất bại trước native khỏi mẫu số chính mà không nói.

Với nhiều family, micro cộng confusion count trước khi tính tỷ lệ; macro DR lấy trung bình DR của các family có mẫu số attack hợp lệ:

$$DR_{macro} = \frac{1}{|\mathcal{F}|} \sum_{f \in \mathcal{F}} D\,R_f.$$

Case tổng hợp dùng chung giữa paraphrase không tạo thêm capture độc lập. Đối với bootstrap hoặc khoảng tin cậy, cần xét đơn vị group/capture phù hợp; công thức nhị thức dựa trên các thử độc lập không tự áp được cho mọi bộ control. Một metric undefined phải giữ null, và lỗi hạ tầng phải được báo riêng thay vì biến thành một quyết định model tùy ý.

Quan hệ giữa các thuật toán và giới hạn. GenTI tối ưu việc sinh câu trả lời theo corpus rồi được kiểm ngoài; OOF tổ chức điểm dự đoán để học Fusion; native replay kiểm hành vi rule; Shadow và lease kiểm điều kiện vận hành. Các cơ chế này bổ sung nhau nhưng không thay thế nhau. Fusion tốt không tự chứng minh LLM sinh rule tốt, và một rule native đạt không tự chứng minh meta-model có khả năng khái quát.

Chương dataset tiếp theo gắn các ký hiệu với tập dữ liệu và cấu hình thực tế của đề tài. Phần kết quả giữ riêng các mẫu số để người đọc đối chiếu công thức. Khi mở rộng corpus, profile hoặc vị trí thực thi, cần kiểm lại những giả định liên quan thay vì tiếp tục dùng một chứng nhận của cấu hình cũ.

CHƯƠNG 6. DATASET VÀ HUẤN LUYỆN MÔ HÌNH AI

Chương này trình bày hai bài toán học trong AHR-IDPS: phân loại hoạt động từ telemetry và sinh quy tắc phát hiện từ yêu cầu có bằng chứng. Bài toán thứ nhất dùng các event đã gán nhãn và đặc trưng tương ứng với Network, Log, Graph, Fusion. Bài toán thứ hai dùng các cặp yêu cầu, evidence, benign và rule tham chiếu để fine-tune LLM. Các nguồn dữ liệu, cách chia tập, hàm mục tiêu và mẫu số đánh giá phải được giữ riêng cho từng bài toán.

Một số lượng lớn event không tự tạo thành tập huấn luyện sinh rule. Event mô tả điều đã quan sát; target SFT phải mô tả quy tắc mong muốn và có căn cứ kiểm định. Ngược lại, một corpus gồm nhiều cách diễn đạt cho cùng rule không thể được báo như từng ấy tình huống mạng độc lập. Việc phân tích dữ liệu trong chương dựa trên manifest, các tập chia đã dùng và tokenizer của artifact thực tế.

6.1. Thiết kế dataset và phân biệt các mục tiêu học

Dữ liệu phân loại và corpus sinh rule được thiết kế theo hai mục tiêu đầu ra khác nhau. Cần đọc số event, số cặp, số rule và family cùng provenance để xác định quy mô thực và mức độc lập của dữ liệu.

Dữ liệu cho phát hiện và tương quan. Dataset telemetry cần phản ánh hoạt động web hợp lệ, hoạt động có ý định tấn công và các dấu vết cảm biến cùng thời điểm. Các trường scenario và session hỗ trợ nối các quan sát của cùng hoạt động và chia tập có kiểm soát. Nhãn phải nêu rõ là nhãn ý định của kịch bản, nhãn cảnh báo của engine hay xác nhận hành vi thực tế; ba khái niệm này không thay thế nhau.

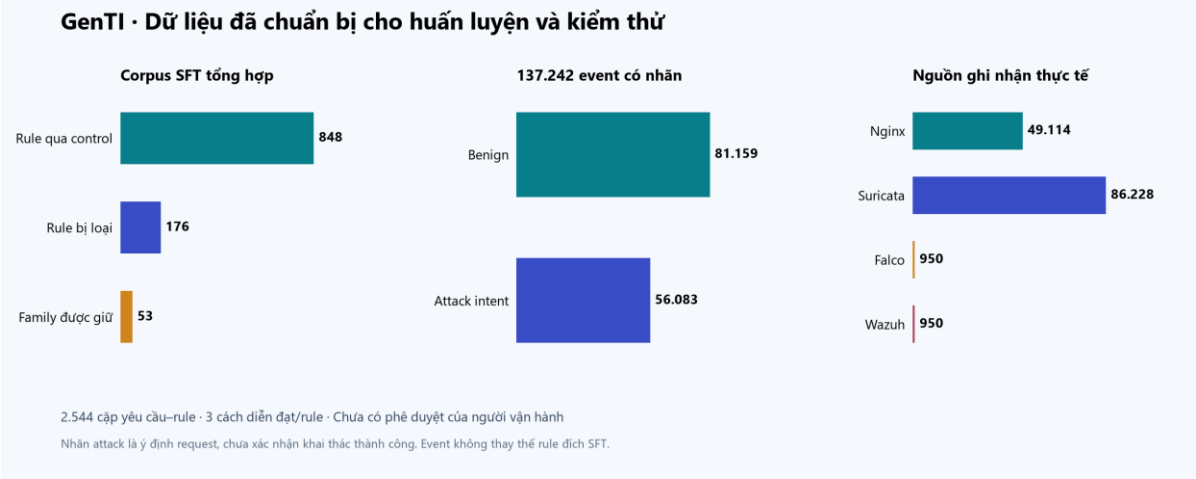
Lượt huấn luyện DVWA trong Drive dùng 112.816 event có nhãn ý định. Hồ sơ telemetry tổng hợp của hệ thống chính gồm 137.242 event, được tạo từ các chiến dịch có nguồn sensor và marker để nối với request đã gửi. Hai số này có phạm vi khác nhau: số đầu là snapshot đầu vào cho run phân loại trong Drive; số sau là tập telemetry tổng hợp phục vụ phần dữ liệu hệ thống. Không cộng trực tiếp hai số vì snapshot 112.816 nằm trong tổng hợp 137.242.

Bảng 6.1. Số sự kiện theo nguồn sensor của telemetry tổng hợp

Nguồn sensor	Số event
nginx	49114
suricata	86228
falco	950
wazuh	950
Tổng	137242

Nguồn: Manifest telemetry chính và telemetry-audit trong source-kit Drive; bản sao nằm trong evidence/.

Theo manifest, tập có 104 nhóm chiến dịch, gồm 56.083 event mang nhãn attack và 81.159 benign. Một request có thể tạo nhiều record ở Nginx, Suricata, Falco hoặc Wazuh, vì vậy số event không phải số request độc lập.



Hình 6.1. Phân bố telemetry của hệ thống chính. Số event khác với số cặp SFT dùng huấn luyện LLM.

Dữ liệu cho AI sinh rule. Fine-tune cần một loại dữ liệu khác: mỗi dòng gắn một yêu cầu phát hiện với evidence, benign, CTI nếu có và target rule. Corpus trong run Colab có 2.544 dòng, 53 family và bốn mức khó. Target của corpus đã qua kiểm tra máy; manifest ghi 0 dòng được người phê duyệt và cho phép machine-validated. Vì thế báo cáo gọi đây là rule tham chiếu tổng hợp có biên nhận kiểm tra, không gọi là bộ gold do chuyên gia duyệt.

```
{
  "id": "sample-id", "family_id": "family-id", "difficulty": 1,
  "task": "Yêu cầu phát hiện có căn cứ",
  "evidence": {"http": {"method": "GET", "uri": "..."}},
  "benign_examples": [{"http": {"method": "GET", "uri": "..."}}],
  "cti": {"references": []},
  "target": {"rule": "alert ...", "summary": "...", "mitre_attack": []},
  "provenance": {"source": "...", "license": "...",
    "review_status": "machine_validated"}
}
```

Đây là minh họa schema. Trong dữ liệu thật, ID, source, license và receipt phải có giá trị kiểm chứng được. Target rule không được xuất hiện nguyên văn trong prompt đầu vào.

Target rule cần trả lời được yêu cầu phát hiện và đồng thời tránh các benign đã xác nhận. Chỉ dùng một payload tấn công rồi yêu cầu model tạo rule có thể khiến model chọn dấu hiệu quá ngắn hoặc quá rộng. Benign cùng endpoint giúp kiểm tra liệu model đang học điều kiện phân biệt hay chỉ học tên đường dẫn có lỗ hổng. Các mẫu không phân biệt được bằng trường quan sát phải được giữ uncertain hoặc loại khỏi target được phê duyệt.

6.2. Thu thập chuẩn hóa chia tập và mở rộng dữ liệu

Quy trình dữ liệu nối việc thu nhận với làm sạch, kiểm target và chia nhóm. Nguồn bổ sung chỉ có giá trị khi schema, oracle, quyền sử dụng và cách ánh xạ được kiểm tra trước khi đưa vào train hoặc benchmark.

Các công cụ thu trong dự án, gồm authorized_monitoring_scenarios.py và collect_dvwa_training_dataset.ps1, cần được chạy trong cùng phạm vi lab và lưu tham số phiên. Sau mỗi lượt, bộ dữ liệu phải đi kèm số request đã gửi, số event theo nguồn, khoảng thời gian, trạng thái thu và lỗi. Chỉ lưu CSV cuối cùng sẽ làm mất khả năng giải thích tại sao một sensor có nhiều record hơn sensor khác.

Khi thu phục vụ sinh rule, cần giữ các trường HTTP đủ liên quan nhưng loại thông tin nhạy cảm như cookie, token và dữ liệu tài khoản khỏi prompt công khai. Một mẫu được làm sạch quá mức có thể không còn dấu hiệu phân biệt; trường hợp đó nên

được đánh dấu không phù hợp để sinh, thay vì để LLM tự đoán phần payload bị mất. Provenance cần mô tả bước làm sạch và giữ liên kết tới raw đã được kiểm soát quyền.

Đối với gói model từ Colab, các artifact cần thu gồm run-config, tokenizer, adapter-config, lịch sử loss, checkpoint được chọn, dataset manifest và raw candidate. Tệp ZIP sau train chỉ là phương tiện đóng gói. Để chứng minh run dùng model nào và stage nào, phải đối chiếu metadata thực chạy thay vì dựa vào tên ZIP hoặc một ô notebook chưa được cập nhật.

Làm sạch và chuẩn hóa

Dữ liệu nhiều nguồn có thể dùng định dạng thời gian, mức severity và tên trường khác nhau. Bước chuẩn hóa cần chuyển về cùng quy ước nhưng giữ giá trị gốc để đối chiếu. Khi một log thiếu timestamp, phải phân biệt thời gian thu nhận với thời gian hoạt động; dùng thời gian thu nhận như thời gian nguồn mà không ghi chú sẽ làm sai phép đo độ trễ.

Các đặc trưng số cần xử lý missing theo hợp đồng đã học, tránh để trường bị thiếu được diễn giải thành không có rủi ro. Categorical hoặc text cần có xử lý đối với giá trị chưa gặp. Chuỗi URI cần giữ sự khác nhau giữa raw và normalized để kiểm tra encoding; việc decode nhiều lần không kiểm soát có thể làm dữ liệu train không còn tương đương với engine runtime.

Exact duplicate cần được xác định bằng nội dung có ý nghĩa, không chỉ bằng event_id. Hai event có ID khác nhau nhưng cùng payload được nhân bản vẫn có thể làm tập test quá dễ. Với rule, fingerprint loại metadata như SID, rev và msg giúp nhận ra các chữ ký tương đương bề mặt. Tuy nhiên near-duplicate về điều kiện phát hiện còn cần kiểm tra theo cấu trúc và family.

Các trường nhãn, marker, tên chiến dịch và điểm model cũ được giữ ngoài biểu diễn học. Khi dựng văn bản Log AI, cần xác nhận những trường này không bị nối lại vào text. Khi dựng graph, các cạnh và node feature cũng không được encode trực tiếp kết quả tương lai. Giảm leakage là yêu cầu xuyên suốt pipeline, không phải một thao tác xóa cột duy nhất.

Đối với corpus SFT, mỗi dòng cần đúng schema, có yêu cầu, evidence và benign phù hợp, target một rule được phép và provenance. Kiểm syntax chỉ loại được lỗi engine; replay mới kiểm được hành vi trên case. Nếu target bị sai, loss thấp có thể chỉ

chứng minh model học tốt một đáp án sai. Vì vậy chất lượng target quan trọng hơn việc nhân số dòng bằng paraphrase.

Tập hiện tại được phép sử dụng các dòng machine-validated theo hợp đồng nghiên cứu, nhưng chưa có human review. Báo cáo giữ nguyên trạng thái đó. Không chuyển cờ reviewed sang true bằng script chỉ để thỏa điều kiện nhập dữ liệu, vì thao tác này làm mất ý nghĩa của provenance và không tạo thêm một lượt đánh giá chuyên môn.

Chia train validation test

Train dùng để fit model; validation dùng chọn cấu hình hoặc ngưỡng; test giữ cho đánh giá cuối cùng. Các event cùng scenario hoặc chiến dịch có tương quan cần được nhóm trước khi chia. Tỷ lệ 60/15/15/10 trong đề cương là một phương án thiết kế có holdout, không phải tỷ lệ được tự áp lên hồ sơ đã chạy. Run DVWA hiện có giữ các kích thước thực tế 66.547 train, 24.320 validation và 21.949 test.

Nếu muốn thêm holdout mới, cần tạo partition mới trước khi xem kết quả và ghi manifest riêng. Không lấy lại những mẫu đã dùng chọn model rồi gọi chúng là test kín. Chia theo ngày, host, app hoặc chiến dịch giúp kiểm tra độ khái quát mạnh hơn random row split, nhưng cần đủ số nhóm và độ phủ loại hành vi để tránh một split chỉ chứa một số category.

Hợp đồng corpus giữ các biến thể thuộc cùng family ở cùng partition, bổ sung fingerprint rule sau khi loại metadata như SID, rev hoặc msg. Các family dành cho unknown_test không được xuất hiện trong dữ liệu phát triển. Theo tập chia đã xuất: train có 1.824 dòng; validation, test và unknown_test mỗi tập 240 dòng. Mỗi tập test có 5 family, mỗi family 48 mẫu. Vì vậy 240 mẫu không tương đương 240 họ tấn công độc lập.

Bảng 6.2. Partition và số family của corpus fine-tune

Partition	Số cặp	Family
Train	1824	38
Validation	240	5
Test	240	5

Partition	Số cặp	Family
Unknown test	240	5
Tổng	2544	53

Nguồn: genti-dataset-manifest.json; SHA-256 corpus: f0995033eb866eeb4a61cd73f9b0fafcd38c7ff2d859d42274362de511075f11.

Dataset bổ sung. Bảng 6.3. Nguồn bổ sung cho PCAP và corpus GenTI

Nguồn	Vai trò nên dùng	Giới hạn cần giữ
Suricata Verify	Case PCAP và rule nhỏ để kiểm engine/buffer/flow	Đọc config, test.yaml, oracle và license. Không coi mọi file là attack có nhãn.
CIC-IDS2017 PCAP	Test ngoài miền và đánh giá IDS/flow	Không chỉ dùng CSV. Capture có traffic nền; phải join flow/time và kiểm scope HTTP.
ET Open	Target rule có nguồn để mở rộng SFT	Chọn subset Suricata tương thích; giữ license theo rule và xây cặp bằng chứng/benign có kiểm định.
Lab đa phiên/app	Nguồn ưu tiên cho task HTTP hiện tại	Capture độc lập, oracle kịch bản, benign cùng endpoint; tách khỏi dữ liệu phát triển.

Suricata Verify cung cấp kiểm thử với input/config để xác minh output engine.[3] CIC công bố PCAP cùng nhãn flow và lịch tấn công, nhưng cần xử lý nhãn theo đơn vị đánh giá.[7] Rule ET Open cung cấp điều kiện phát hiện; nó không tự có đủ cặp prompt–evidence–benign cho fine-tune.[8] Đề tài nên ưu tiên corpus HTTP đa nguồn được review, dùng Verify cho semantics và CIC làm test ngoài miền; không lấy toàn bộ CIC CSV để thay target rule.

CIC có thể hỗ trợ đánh giá phát hiện theo flow và cung cấp PCAP ngoài môi trường DVWA, nhưng CSV đặc trưng flow không tự chứa đầy đủ payload để viết rule HTTP. Khi ánh xạ sang hệ thống, cần xác định đơn vị flow, timestamp, 5-tuple và điều

kiện cảm biến. Một model học trên trường CIC không thể nạp trực tiếp vào vector event lab khi thứ tự và ý nghĩa đặc trưng chưa tương đương.

Để dùng traffic web từ CIC cho GenTI, cần trích các giao dịch HTTP có oracle, xác định phạm vi và xây target rule có benign. Nếu một rule của lab cố định đường dẫn DVWA, việc nó không khớp ứng dụng khác có thể phản ánh scope chứ không chỉ phản ánh chất lượng model. Bộ đánh giá phải kiểm tra scope trước khi quy kết false negative.

Rule công khai có thể cung cấp target ban đầu, nhưng thường chưa có đủ yêu cầu và cặp bằng chứng cho SFT. Người xây corpus phải xác nhận engine tương thích, nguồn và license theo từng rule; sau đó viết yêu cầu dựa trên điều kiện thực, bổ sung attack và benign, chạy native rồi review. Không suy ngược một mô tả tấn công quá rộng từ một content match ngắn.

Trong giai đoạn đầu, nên ưu tiên các rule HTTP có semantics nằm trong khả năng quan sát của lab. Những rule phụ thuộc flowbit, dữ liệu ngoài, nhiều rule hoặc PCRE phức tạp cần một evaluator hỗ trợ tương ứng. Đưa chúng vào contract chỉ cho phép rule độc lập sẽ làm tăng lỗi hợp đồng mà không phản ánh đúng khả năng ngôn ngữ của LLM.

6.3. Huấn luyện đăng ký và tái lập mô hình phát hiện

Các nhánh Network, Log, Graph và Fusion được xem xét từ biểu diễn đến artifact triển khai. Việc đăng ký cần giữ schema, threshold và identity để kết quả huấn luyện có thể được sử dụng và rollback một cách nhất quán.

Huấn luyện Network AI

Network AI sử dụng biểu diễn số từ dữ liệu mạng và HTTP đã làm sạch. Các trường có thể liên quan tới phương thức, trạng thái, lưu lượng và ngữ cảnh flow theo mapper hiện có. Việc feature nào được dùng phải dựa vào danh sách feature_names của run; không thêm tên đặc trưng trong báo cáo chỉ vì chúng thường có trong một IDS khác.

Mô hình XGBoost kết hợp các cây quyết định theo gradient boosting. Ở bước lặp, một cây mới giảm phần lỗi còn lại theo hàm mục tiêu; đầu ra tổng được chuyển qua sigmoid để tạo điểm nhị phân. Hàm phạt giúp kiểm soát độ phức tạp. Các cấu hình

max_depth, eta, subsample và colsample ảnh hưởng độ khớp và tính khái quát, nên phải lưu cùng artifact và seed.

Với lớp mất cân bằng, cần xem precision, recall và PR-AUC thay vì chỉ accuracy. Nếu dùng cân bằng lớp hoặc trọng số, chỉ fit chúng từ train. Thay đổi ngưỡng để tăng recall thường ảnh hưởng false positive; ngưỡng nên được chọn bằng validation và giữ cố định khi mở test. Những ngưỡng đã có trong hồ sơ phải được báo đúng, không tối ưu lại bằng test rồi ghi như kết quả gốc.

Huấn luyện Log AI

Log AI biểu diễn nội dung được phép của event rồi phân loại. Trong mã được đối chiếu, tên lớp MiniLMEmbedder là tên giao diện, còn backend thực tế là HashingVectorizer 384 chiều chuẩn L2. Báo cáo không gọi đây là suy luận Transformer MiniLM. Sự phân biệt này ảnh hưởng cả cách giải thích khả năng ngữ nghĩa lẫn chi phí tài nguyên.

Sau biểu diễn, StandardScaler và MLP được dùng để học nhãn, đồng thời Isolation Forest học benign để bổ sung điểm bất thường. Hashing tránh cần một vocabulary lưu riêng nhưng có thể gây va chạm; nó không tự hiểu nghĩa của những cách diễn đạt mới. Một kết quả test rất cao trên log template cần được kiểm tra trên template và nguồn khác trước khi gán khả năng hiểu ngôn ngữ tổng quát.

Để theo dõi drift, có thể so phân bố trường, tỷ lệ missing, mật độ hash và điểm model theo cửa sổ. Đây là thiết kế đo đề xuất, không phải kết quả drift đã có. Khi nhận thấy phân bố đổi, cần điều tra nguồn và nhãn trước khi retrain; một thay đổi do định dạng log cũng có thể làm model thay đổi mà không có tấn công mới.

Huấn luyện Graph AI

Graph AI sử dụng quan hệ của các thực thể và sự kiện để tạo biểu diễn có ngữ cảnh. Graph chỉ nên chứa thông tin sẵn có tại thời điểm chấm điểm; đưa toàn bộ tương lai của chiến dịch vào graph khi đánh giá một event đầu phiên sẽ làm kết quả không tương đương điều kiện online. Cần lưu quy tắc tạo node, edge và cửa sổ để tái lập.

Trong nhánh đã train, thuật toán dùng mean aggregation theo phong cách GraphSAGE, ghép đặc trưng node với trung bình láng giềng, pool graph rồi đưa vào MLP. Không có bằng chứng đây là temporal GNN đa tầng train end-to-end. Đánh giá

graph cần xem mô hình có thực sự bổ sung tín hiệu quan hệ hay chủ yếu lặp lại đặc trưng event; baseline graph heuristic và ablation cạnh hỗ trợ trả lời câu hỏi này.

Network chuyển event đã làm sạch thành vector đặc trưng mạng/HTTP, rồi dùng XGBoost histogram:

$$\hat{p}(x) = \sigma\left(\sum_{k=1}^{K} f_k(x)\right), \qquad \mathcal{L} = \sum_i \ell_{BCE}(y_i, \hat{p}_i) + \sum_k \Omega(f_k).$$

Mã train cấu hình max_depth=5, eta=0,08, subsample=0,9, colsample_bytree=0,9, max_bin=256 và seed 42; số round là tham số, mặc định 120. Khi yêu cầu CUDA mà XGBoost fallback CPU, train dừng với lỗi.

Log tạo văn bản từ field được phép, rồi dùng HashingVectorizer 384 chiều, chuẩn L2 và không alternate sign. **Tên lớp MiniLMEmbedder trong mã là hợp đồng giao diện; backend thực tế là hashing, không phải chạy mô hình Transformer all-MiniLM.** Vector đi qua StandardScaler và MLP một hidden layer 32 node. IsolationForest 80 cây được fit trên benign; điểm cuối lấy

$$s_{log} = \max(p_{MLP},\ 0{,}5\, clip(0{,}5 - decision_function_{IF}, 0{,}1)).$$

Đây là thuật toán đã train trong nhánh Colab, không gán nhãn mới cho rule GenTI.

Graph xây graph nhân quả từ event hiện tại, tạo node feature và vector láng giềng trung bình:

$$m_v = \frac{1}{|N(v)|}\sum_{u \in N(v)} x_u\,, \quad h_v = [x_v \,\|\, m_v], \quad g = [mean_v(h_v) \,\|\, max_v(h_v) \,\|\, graph_features].$$

Vector graph được chuẩn hóa rồi phân loại bằng MLP 32 node; baseline dùng graph feature và MLP 24 node. Đây là **GraphSAGE-style mean aggregation cộng MLP**; không có căn cứ mô tả như một GNN temporal đa tầng được train end-to-end trên lịch sử toàn hệ thống. Những tên artifact không thay thế việc đọc thuật toán thực thi.

Fusion

Weighted hybrid chuẩn hóa lại trọng số của các nguồn đang có:

$$s = \frac{\sum_{j \in A} w_j s_j}{\sum_{j \in A} w_j}, \quad (w_{rule}, w_{net}, w_{log}, w_{graph}, w_{ctx}) = (0{,}25, 0{,}25, 0{,}20, 0{,}25, 0{,}05).$$

Rule score dùng severity, confidence và hệ số sensor: $s_r = clip(0{,}55 severity + 0{,}25 confidence + 0{,}20 engine_weight, 0{,}1)$ khi có bằng chứng rule. Context kết hợp mật độ và điểm AI có sẵn; không cộng nhãn chiến dịch để tăng rủi ro.

Fusion OOF chia train thành ba group-fold. Mỗi base model chỉ sinh điểm cho fold chưa được nó fit; các điểm OOF ghép với rule/context, severity, density và missing count để train XGBoost meta-model. Sau đó fit base model trên toàn train, chấm validation/test và dùng meta-model đã học.

Thuật toán B Fusion dự đoán ngoài fold theo nhóm

Khóa các nhóm train, validation và test.
Với từng fold trong ba group-fold của train:
 Fit Network, Log và Graph trên các nhóm còn lại.
 Chỉ lưu dự đoán cho nhóm được giữ ngoài fold.
Fit Fusion XGBoost trên điểm OOF và context hợp lệ.
Fit lại base model trên toàn bộ train.
Chấm validation và test cố định bằng meta-model.

Threshold trong run là 0,55 cho weighted/Fusion và 0,5 cho các baseline. Không mô tả những ngưỡng cố định này là đã được tối ưu bằng test.

Điểm fusion cần đi kèm nguồn đang có và nguồn bị thiếu. Khi chỉ có một số model, việc chuẩn hóa lại trọng số phải được công bố để người vận hành không tưởng điểm được tính từ đủ mọi nhánh. Các điểm chưa hiệu chuẩn không mặc nhiên có cùng ý nghĩa xác suất; cộng chúng có thể hữu ích như baseline nhưng cần đánh giá theo mục tiêu vận hành.

OOF giảm việc meta-model học từ các dự đoán in-sample quá lạc quan. Tuy nhiên nó không tự xử lý mọi leakage do cùng chiến dịch hoặc template nằm ở nhiều fold. Vì vậy group-fold phải dùng đúng đơn vị chia nhóm. Sau OOF, base model được fit lại trên toàn train, còn meta-model không được fit lại bằng test.

Model registration and rollback. Model registry cần lưu loại bài toán, schema đặc trưng, phiên bản hoặc identity artifact, checksum, thời gian train, dữ liệu và ngưỡng. Một artifact benchmark-only cần được đánh dấu khác model deployable. Việc

đăng ký metadata vào DB không tự chứng minh service đã nạp artifact đó; runtime cần báo identity đang dùng và đối chiếu với manifest.

Rollback phải khôi phục một bộ nhất quán gồm model, tokenizer hoặc feature schema, prompt và cấu hình. Chỉ đổi trọng số trong khi giữ template không tương thích có thể làm đầu ra sai hợp đồng. Với rule đã gắn model hash, thay model không có nghĩa rule cũ tự được phê duyệt lại; cần giữ lịch sử và cổng kiểm định phù hợp.

Logistic Regression dùng StandardScaler rồi xác suất $p(y = 1 \mid x) = \sigma(w^T x + b)$, class_weight=balanced và tối đa 500 iteration. Random Forest dùng 100 cây, max_depth=12, random_state=42; xác suất là trung bình xác suất leaf từ các cây. Cả hai dùng cùng vector Network, train/test như run; chúng không sinh rule. IsolationForest trong Log học độ bất thường từ benign bằng chiều dài đường cô lập; điểm decision_function được chuyển/clamp theo công thức Log ở trên, không được báo như một xác suất attack đã hiệu chuẩn độc lập.

Phát hiện Suricata dùng decoder ứng dụng và sticky buffer HTTP trước khi đối sánh các điều kiện rule. Trong profile tự động, method phải có đúng ba byte GET; URI normalized phải bắt đầu bằng /vulnerabilities/xss_r/? và chứa <script không phân biệt hoa/thường. Matcher gateway percent-decode một lần theo latin1; dấu cộng vẫn là ký tự URI. Chỉ grammar giới hạn này được chấp nhận tự động; PCAP parity kiểm việc diễn giải giữa engine và gateway. Không coi một matcher Python tổng quát là thay thế Suricata cho mọi signature/PCRE/flowbit.

6.4. Corpus GenTI hợp đồng tác vụ và phân tích chất lượng

Phân tích corpus được nối với mục tiêu generation và review. Phân bố family, category, token và trạng thái kiểm duyệt giúp xác định giới hạn học được, khoảng trống validation và ưu tiên mở rộng có căn cứ.

AI sinh rule là tác vụ được LLM thực hiện. GenTI là phương pháp tổ chức dữ liệu, prompt, huấn luyện và xác minh để tác vụ đó tạo ra một Candidate có thể kiểm tra. Không có hai model độc lập mang tên “AI sinh rule” và “GenTI” trong triển khai chính. Cùng model Qwen được gọi cho review chọn mẫu và generation, nhưng hai lời gọi có schema và mục tiêu khác nhau. Corpus hiện tại học đầu ra rule, không chứa bộ nhãn review riêng để bảo đảm model chọn benign chính xác.

Phần này mô tả thuật toán theo mã nguồn đang có, bao gồm các nhánh phân loại đã được thực nghiệm trong Drive. Thuật toán mô tả không đồng nghĩa mọi nhánh cùng được bật trong luồng GenTI hiện tại.

Input gồm yêu cầu phát hiện, evidence, benign và CTI khi thực sự có. Output gồm rule một dòng, mô tả và danh sách ATT&CK theo contract. Những thông tin này phải nhất quán: summary không được khẳng định một lớp tấn công rộng hơn điều kiện rule; ATT&CK không được điền tùy tiện chỉ để làm đầu ra có vẻ đầy đủ. Corpus hiện có CTI rỗng, nên không có căn cứ coi model đã được kiểm chứng trên lập luận CTI phức tạp.

Prompt không chứa nguyên target để model sao chép. Target chỉ xuất hiện ở phần assistant dùng tính loss. Những dữ liệu có ý nghĩa kiểm định như label, ID, marker và điểm AI cũ được giữ ngoài prompt theo hợp đồng làm sạch. Khi cần giải thích một kết quả, dùng phần mô tả, các field đầu vào và biên nhận kiểm định; không coi một câu giải thích trôi chảy là chứng minh rule đúng.

Review yêu cầu model chọn evidence, benign và uncertain từ pool giới hạn. Generation yêu cầu model tạo rule phù hợp với task đã hình thành. Hai tác vụ dùng cùng trọng số trong hệ thống chính nhưng có schema khác nhau. Việc fine-tune generation không bảo đảm chất lượng phân vai benign; corpus review có nhãn cần được xây riêng để đánh giá độ tin cậy của khâu tự chọn mẫu.

Khi đánh giá review, cần tính tỷ lệ chọn sai benign, tỷ lệ bỏ sót evidence phù hợp, tỷ lệ uncertain và tỷ lệ schema hợp lệ. Một model luôn chọn mọi mẫu là attack có thể tăng cảm giác quyết đoán nhưng làm giảm giá trị học quy tắc phân biệt. Backend cần kiểm tra slot thực sự tồn tại và không bị dùng đồng thời ở hai vai trò; kiểm tra cấu trúc này không thay thế xác nhận ngữ nghĩa.

Phân tích corpus huấn luyện thực tế

Phân tích được chạy trên file rule-corpus hiện có, đối chiếu checksum với dataset-manifest đã tải từ Drive. Hợp đồng kiểm rule, provenance và receipt machine-validation được dùng lại từ notebook. Chia tập lấy đúng ID đã dùng trong run Colab, không chia lại rồi gọi kết quả là phân tích cùng run. Độ dài token dùng tokenizer và chat template lấy từ gói model gốc, không suy từ số ký tự.

Bảng 6.4. Tóm tắt kiểm toán corpus SFT thực tế

Đại lượng	Giá trị
Dòng SFT / rule khác nhau / family	2.544 / 848 / 53
Train: dòng / rule khác nhau / family	1.824 / 608 / 38
Mỗi validation/test/unknown: dòng / rule / family	240 / 80 / 5
Human reviewed / CTI tham chiếu	0 / 0
Nguồn dữ liệu	Một bộ tổng hợp AHR
Giao family/signature/prompt giữa mọi split	0 / 0 / 0 (exact checks)

Nguồn: evidence/corpus-analysis.json; 0 human review và 0 CTI là kết quả đếm thực tế, không phải thiếu cột thống kê.

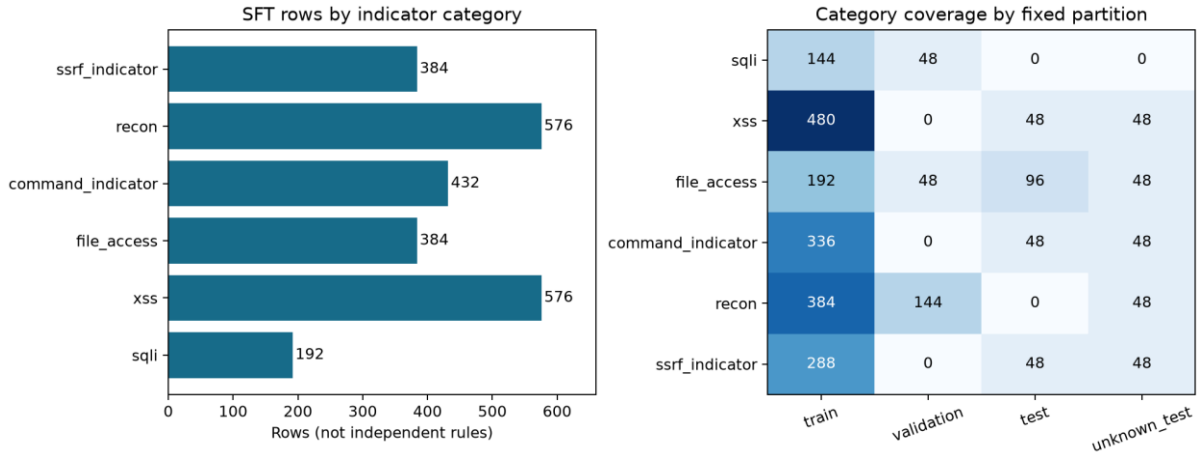
Ba paraphrase cho cùng target tăng độ đa dạng diễn đạt nhưng không làm số chữ ký độc lập tăng gấp ba. Train 1.824 cặp có 608 target rule; mỗi split còn lại có 80 target rule và 5 family. Kiểm exact family, fingerprint signature sau bỏ metadata, và prompt cho tất cả cặp partition đều cho giao nhau bằng 0. Kết quả này chưa loại trừ mọi tương đồng ngữ nghĩa hoặc template gần giống.

Bảng 6.5. Số dòng theo category trong corpus và các split

Nhóm	Toàn bộ	Train	Val	Test	Unknown
sqli	192	144	48	0	0
xss	576	480	0	48	48
file_access	384	192	48	96	48
command_indicator	432	336	0	48	48
recon	576	384	144	0	48
ssrf_indicator	384	288	0	48	48

Tổng corpus có GET 1.272 và POST 1.272; tám endpoint, mỗi endpoint 318 dòng. Mỗi difficulty có 636 dòng, riêng train có 456 dòng ở mỗi mức khó. Phân bố đều theo endpoint/difficulty là dấu vết của bộ sinh template, không chứng minh độ đa

dạng traffic thực địa. POST ở đây vẫn dùng marker trong URI; không có cơ sở công bố coverage với POST body chỉ từ thống kê method.



Hình 6.2. Phân bố dòng SFT theo nhóm chỉ báo và độ phủ của từng partition. Giá trị 0 cho biết nhóm không có trong split, không phải model phát hiện 0 phần trăm.

Validation hiện chỉ có SQLi, file access và recon; không có XSS, command hoặc SSRF. Vì checkpoint được chọn bằng validation loss, nó chưa được chọn trên tập phủ đủ các nhóm mục tiêu. Test cũng không có SQLi; unknown_test không có SQLi. Không nên gán các chỉ số tổng cho riêng SQLi. Đối với run sau, chia family phải đồng thời kiểm category coverage; các family khác nhau của cùng category vẫn có thể được giữ độc lập.

Ý nghĩa phân bố và mức độc lập

Ba paraphrase cho cùng một target làm tăng tính đa dạng của câu yêu cầu nhưng không tạo ba chữ ký độc lập. Vì vậy cần công bố đồng thời số dòng, số rule khác nhau và số family. Nếu chỉ ghi 2.544 mẫu mà không nêu 848 rule và 53 family, người đọc có thể đánh giá quá cao độ rộng của corpus.

Tám endpoint có số dòng bằng nhau và bốn mức khó có số dòng bằng nhau phản ánh thiết kế bộ sinh tổng hợp. Cân bằng này giúp phân tích có kiểm soát, nhưng không phản ánh tần suất thực trong lab hoặc production. Độ khó được gán bởi chương trình học cũng chưa phải độ khó được đo bằng chuyên gia hoặc lỗi trên capture độc lập.

Validation

Validation chỉ có ba nhóm SQLi, file access và recon; thiếu XSS, command và SSRF. Do checkpoint được chọn bằng loss trên tập này, kết quả chọn stage chưa đánh giá đều các nhóm mục tiêu. Với run tiếp theo, cần chia theo family đồng thời bảo đảm

category coverage, sử dụng family khác nhau của cùng nhóm để vừa giữ độc lập vừa có độ phủ.

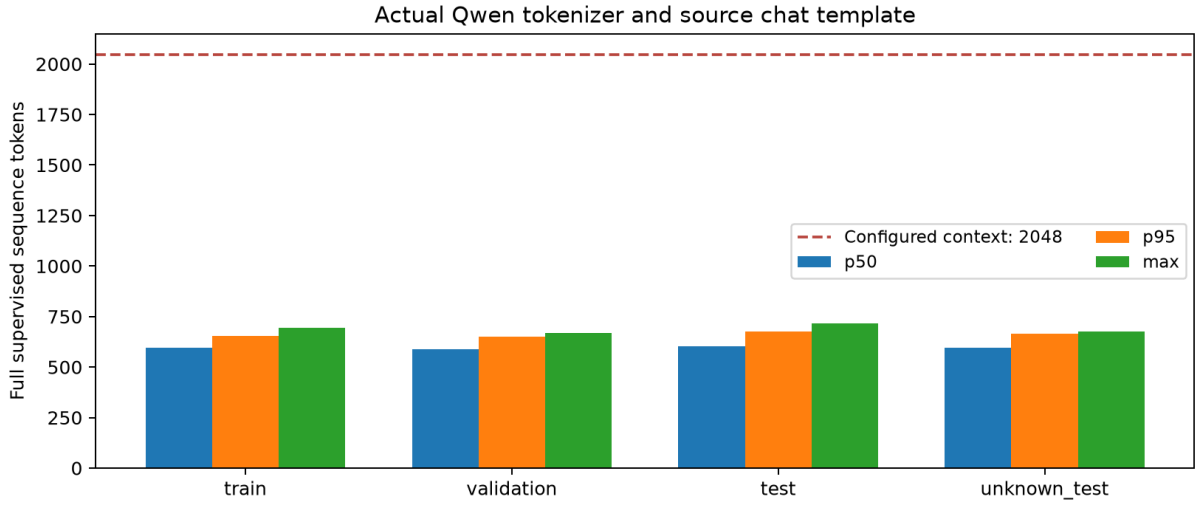
Test và unknown_test hiện cũng không chứa SQLi. Do vậy không thể dùng tỷ lệ syntax tổng của hai tập để khẳng định tỷ lệ syntax riêng cho SQLi. Bảng coverage là một phần cần thiết của báo cáo, giúp xác định những kết luận nào được dữ liệu hỗ trợ và những kết luận nào vẫn cần thêm mẫu.

Bảng 6.6. Phân bố token chuỗi SFT đầy đủ theo tokenizer thực

Tập	Min	P50	P95	Max	Trên 2.048
train	547	595.0	654.0	696	0
validation	551	588.0	650.1	670	0
test	556	603.5	676.1	716	0
unknown_test	552	594.0	665.05	676	0

Nguồn: Token được encode từ system + user + assistant target với tokenizer/chat template của gói Qwen đã xuất. Không nạp trọng số model.

Không có chuỗi nào trong corpus hiện vượt 2.048 token. Ở train, prompt lớn nhất 543 token, target lớn nhất 154 token; chuỗi đầy đủ lớn nhất 696 token. Do đó nâng context hoặc chuyển model lớn hơn chưa giải quyết nút thắt chính: nguồn dữ liệu, hard negatives và chất lượng target. Khi thêm body/header/CTI vào corpus mới phải đo lại; không dùng số liệu độ dài này làm bảo đảm cho dữ liệu mới.



Hình 6.3. P50, P95 và max chuỗi SFT đầy đủ; đường đứt là context cấu hình 2.048.

Các thống kê độ dài được tính bằng tokenizer và chat template từ gói model gốc. Số ký tự hoặc số từ không thay thế số token, đặc biệt khi URI có percent encoding. Không có chuỗi nào vượt 2.048 trong corpus đã kiểm, nên tăng context ngay lúc này chưa giải quyết thiếu nguồn và thiếu benign khó. Với dữ liệu thêm body, header hoặc CTI dài, cần đo lại trước khi tái sử dụng cấu hình.

L4 phù hợp làm điểm xuất phát cho QLoRA 7B đã dùng, nhưng khả năng chạy batch cụ thể còn phụ thuộc độ dài, optimizer và phần mềm. Không có log VRAM đủ để công bố peak bộ nhớ của run GenTI. Khi train lại, cần log allocated và reserved theo bước cùng throughput token để xác định giới hạn thực tế, thay vì coi tên GPU là bảo đảm mọi cấu hình đều chạy được.

Phân tích corpus 50.000 mẫu và thực nghiệm cục bộ

Corpus mở rộng có 50.000 prompt khác nhau, 1.696 rule tham chiếu và 53 họ chữ ký. Các biến thể thay cách diễn đạt tác vụ và thứ tự bằng chứng; không được xem là 50.000 cuộc tấn công hoặc rule độc lập. Tham chiếu đã qua Suricata 8.0.6 trên 16.384 HTTP control: nhận 5.088 cặp gốc, loại 1.056 bản nháp, sau đó mới augmentation theo nhóm đã khóa.

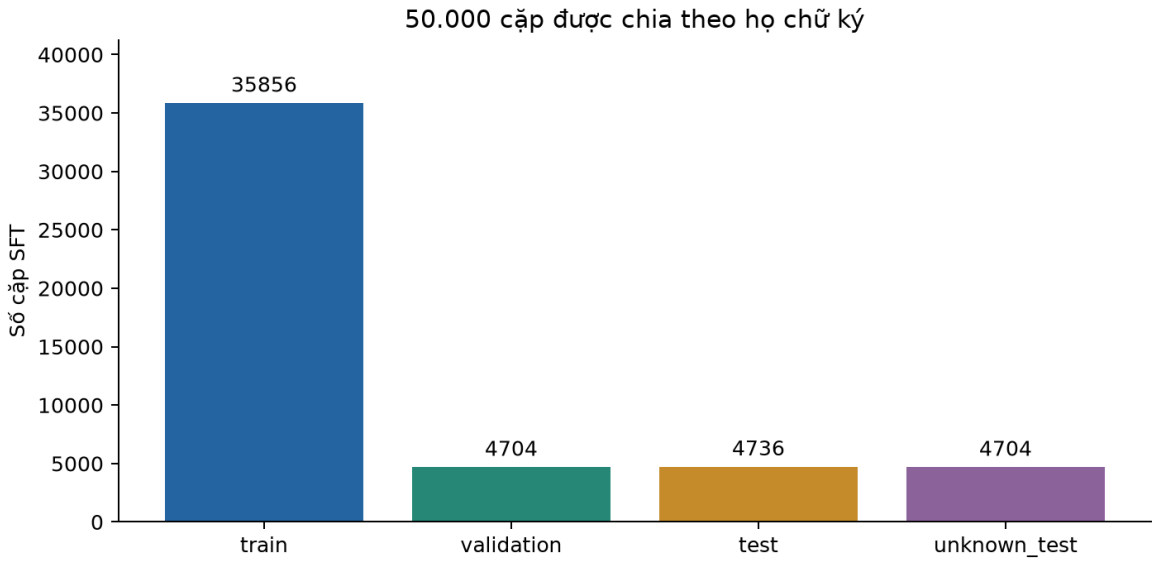
Nguồn Kaggle gồm HTTP CSIC, SQL Injection, SQLi/XSS/command injection và URL có nhãn. Không cộng các CSV chồng lấn thành số quan sát độc lập. Bộ SQLi/XSS được tác giả ghi chưa hoàn thiện; chỉ phần qua lọc nhãn, loại input mâu thuẫn và kiểm replay được dùng làm bằng chứng yếu trong train. Các bộ giấy phép chưa rõ được lưu nghiên cứu riêng. URL có nhãn không tự trở thành cặp yêu cầu–rule chuẩn. Nguồn và giấy phép khai báo được ghi tại [14, 15].

Trong corpus trước augmentation có 416 HTTP control dương dùng payload Kaggle; nhãn này không chứng minh khai thác thành công. Mọi biến thể của cùng họ và fingerprint chữ ký nằm trong cùng partition. Kiểm giao nhau family, chữ ký chuẩn hóa và prompt giữa các tập đều bằng 0; các kiểm tra này chưa loại bỏ mọi tương đồng ngữ nghĩa giữa template.

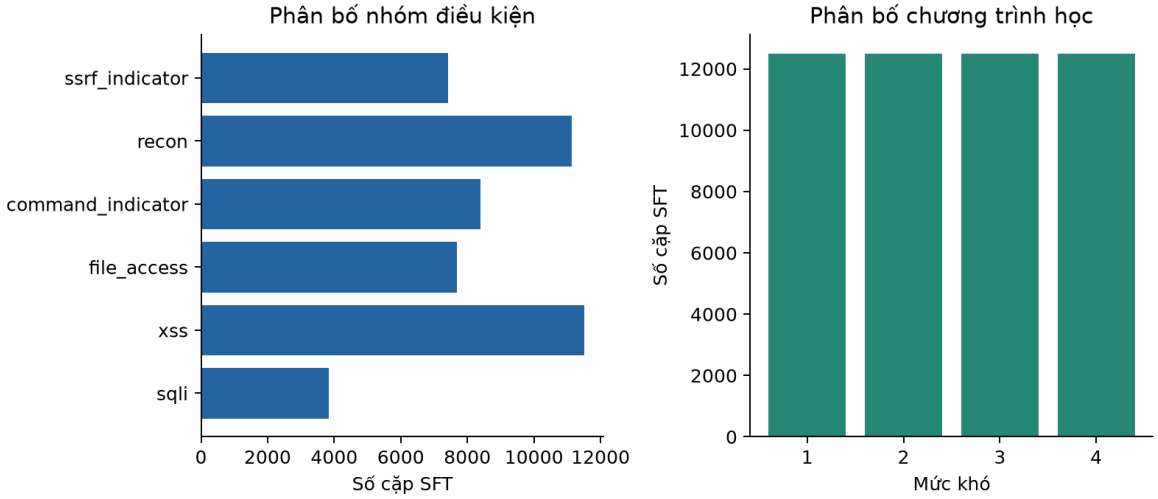
Bảng 6.7. Corpus mở rộng và quy mô chia tập

Đại lượng	Giá trị
Số prompt khác nhau	50.000

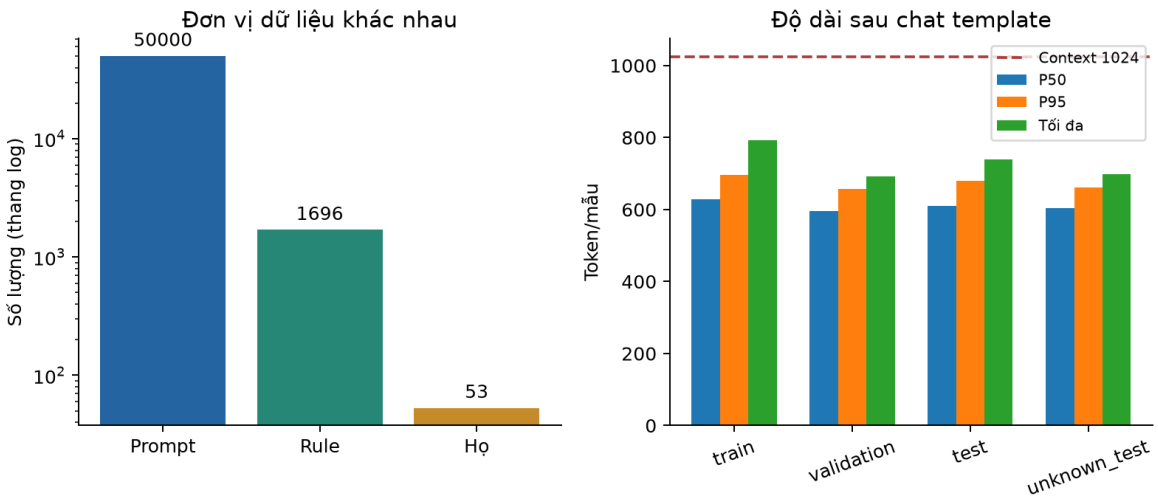
Đại lượng	Giá trị
Rule tham chiếu khác nhau	1.696
Họ chữ ký	53
train	35.856
validation	4.704
test	4.736
unknown_test	4.704
Human reviewed	0; không gán nhãn đã duyệt chuyên gia



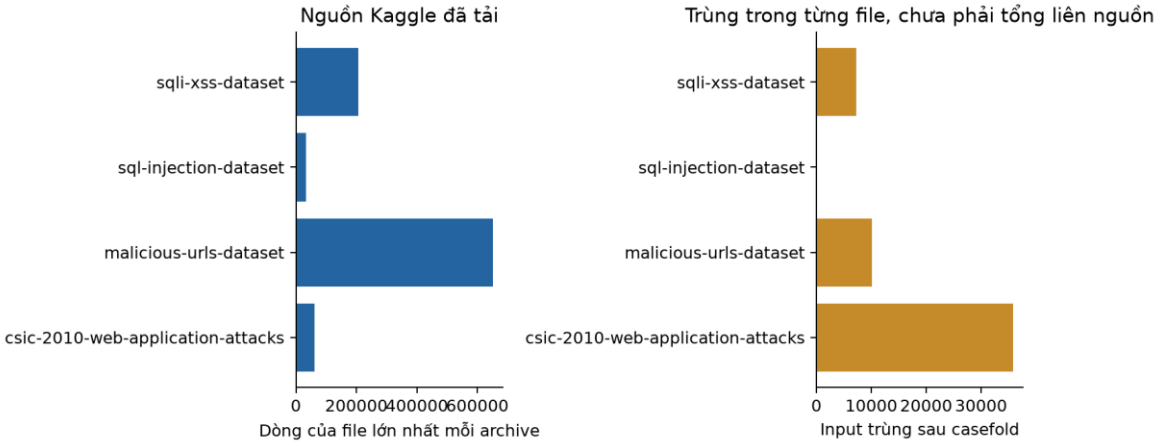
Hình 6.4. Chia 50.000 cặp SFT theo họ chữ ký.



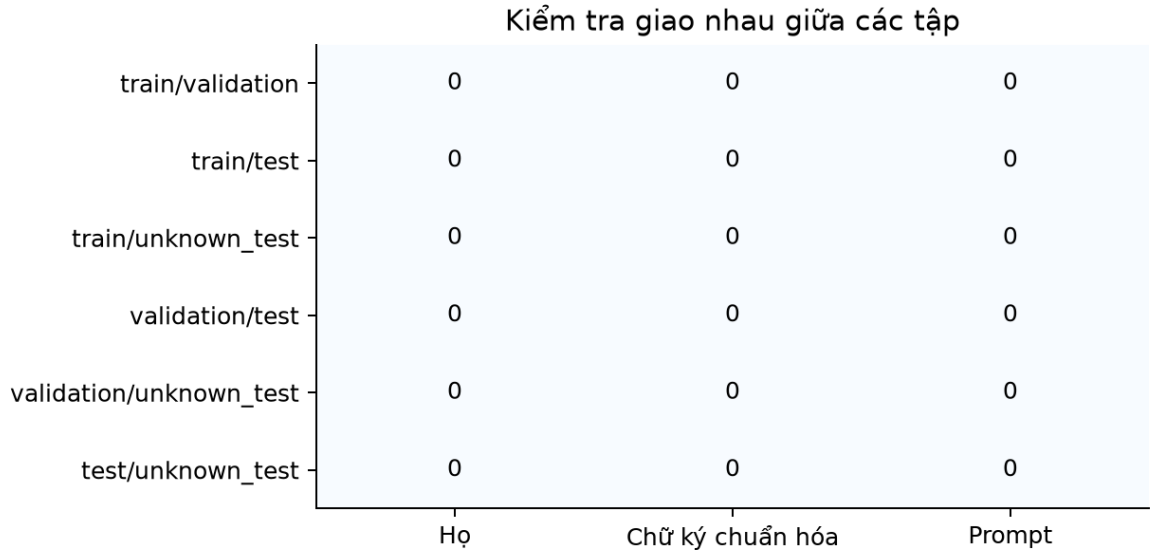
Hình 6.5. Phân bố nhóm điều kiện và mức khó.



Hình 6.6. Đơn vị dữ liệu và độ dài token đo theo chat template.



Hình 6.7. Quy mô và số input trùng trong file chính của mỗi bộ Kaggle.



Hình 6.8. Kiểm tra giao nhau giữa các partition.

6.5. Fine tune Qwen và lựa chọn checkpoint

Cấu hình QLoRA và chương trình học được đối chiếu với log thực chạy. Việc chọn checkpoint, giữ raw output và nêu giới hạn của loss giúp phân biệt kết quả huấn luyện với chất lượng rule còn cần kiểm ngoài model.

Run-config, adapter-config và deployment manifest cùng xác nhận model Qwen/Qwen2.5-Coder-7B-Instruct. Model nền cố định revision để có thể truy vết nguồn trọng số; fine-tune dùng LoRA trên model lượng tử NF4-double. LoRA giữ phần lớn trọng số nền và học phần cập nhật nhỏ trong các projection attention, phù hợp với giới hạn bộ nhớ GPU một máy.[5, 6]

Bảng 6.8. Cấu hình model và adapter được xác nhận từ artifact Colab

Tham số	**Giá trị đã xác nhận**
Model	Qwen2.5-Coder-7B-Instruct
GPU	NVIDIA L4
Seed	42
Chiều dài tối đa	2048
Quantization train	nf4-double
LoRA rank / alpha	16 / 32
LoRA dropout	0,05
Target modules	q_proj, k_proj, v_proj, o_proj
Bias	none
Torch ghi trong run-config	2.11.0+cu130
Stage đã chọn	4

QLoRA giữ trọng số nền NF4, tối ưu cập nhật LoRA $\Delta W = (\alpha/r)BA$ trên các projection q/k/v/o; $r = 16$, $\alpha = 32$, dropout 0,05. Trong forward, phần trainable là adapter; không full fine-tune mọi trọng số 7B. Prompt gồm system, task, evidence, benign và CTI. Token prompt/padding có label -100, chỉ token assistant tham gia cross entropy. Tokenizer/template phải khớp và không được cắt mất target cho vừa context.

Thuật toán C Fine tune theo chương trình học

Kiểm corpus, provenance và family.
Khóa train, validation, test và unknown_test.
Tokenize prompt và target; mask prompt và padding.
Với stage từ 1 đến 4:
  Lấy các dòng train có difficulty không lớn hơn stage.
  Tiếp tục cập nhật cùng adapter và lưu checkpoint.
  Tính loss trên validation cố định.
Chọn checkpoint có validation loss nhỏ nhất.
Khóa lựa chọn; sinh candidate test một lần.
Lưu raw output, config, tokenizer và checksum.

Không tối ưu model trực tiếp theo số rule Active hay kết quả Shadow hiện tại; đó là các kiểm định vận hành khác mục tiêu token loss.

Chương trình học và lựa chọn checkpoint. Ở mỗi stage, notebook lấy các dòng train có difficulty không lớn hơn stage hiện tại. Mỗi mức khó có 456 dòng, tạo các tập tích lũy lần lượt 456, 912, 1.368 và 1.824 dòng. Adapter được cập nhật liên tiếp qua bốn stage; validation giữ nguyên 240 mẫu để chọn checkpoint. Không dùng test hoặc unknown_test để quyết định stage.

Loss token là trung bình âm log xác suất của token đích sau phần prompt:

$$\mathcal{L}_{SFT} = -\frac{1}{|M|}\sum_{t\in M}\log p_\theta(y_t \mid x, y_{<t}),$$

với M là vị trí token câu trả lời không bị mask. Loss thấp hơn cho thấy model dự đoán target tốt hơn trên tập đó, nhưng không trực tiếp cho biết rule có phát hiện đúng trên traffic mới hay không.

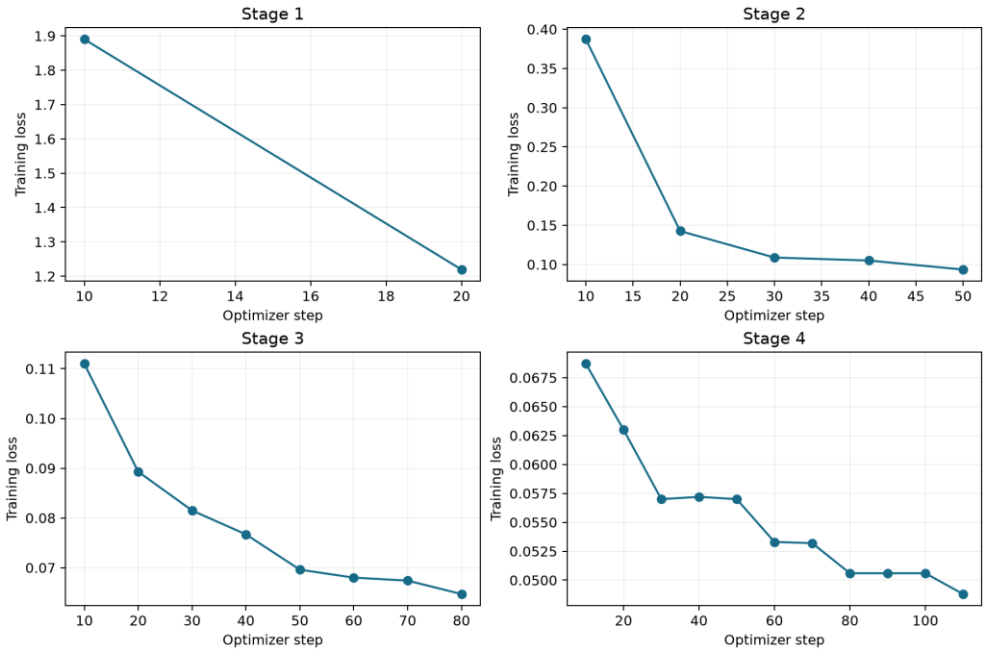
Bảng 6.9. Loss, số bước và thời gian train theo stage

Stage	Cặp tích lũy	Step	Train loss	Val loss	Giây
1	456	29	1.346249	0.832084	548.8
2	912	57	0.157815	0.218949	1010.5
3	1368	86	0.077448	0.117605	1480.4

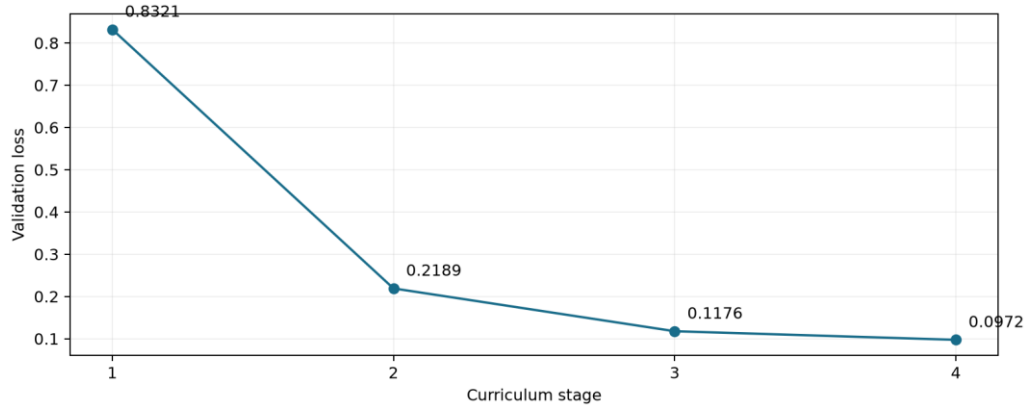
Stage	Cặp tích lũy	Step	Train loss	Val loss	Giây
4	1824	114	0.055240	0.097243	1970.6

Nguồn: training-history.json và stage-selection.json; thời gian là train_runtime từng stage, tính bằng giây.

Stage 4 có validation loss nhỏ nhất 0,097243 và được ghi selected_stage=4 trong manifest. Tổng train_runtime là 5.010,2981 giây, khoảng 83,5 phút; tổng số optimizer step của các stage là 286. Không dùng tổng này để mô tả thời gian toàn phiên Colab, vì chưa bao gồm tải trọng số, mọi lượt evaluate phụ, sinh 960 đầu ra và merge/export. Run-config xác nhận GPU NVIDIA L4 và Torch 2.11.0+cu130; chưa có log VRAM/time-series riêng của chính lượt GenTI để báo peak bộ nhớ.



Hình 6.9. Loss train của bốn stage, vẽ lại trực tiếp từ log gốc để nhãn dễ đọc. Step được ghi theo từng Trainer, không phải một trục step liên tục.



Hình 6.10. Validation loss dùng chọn stage.

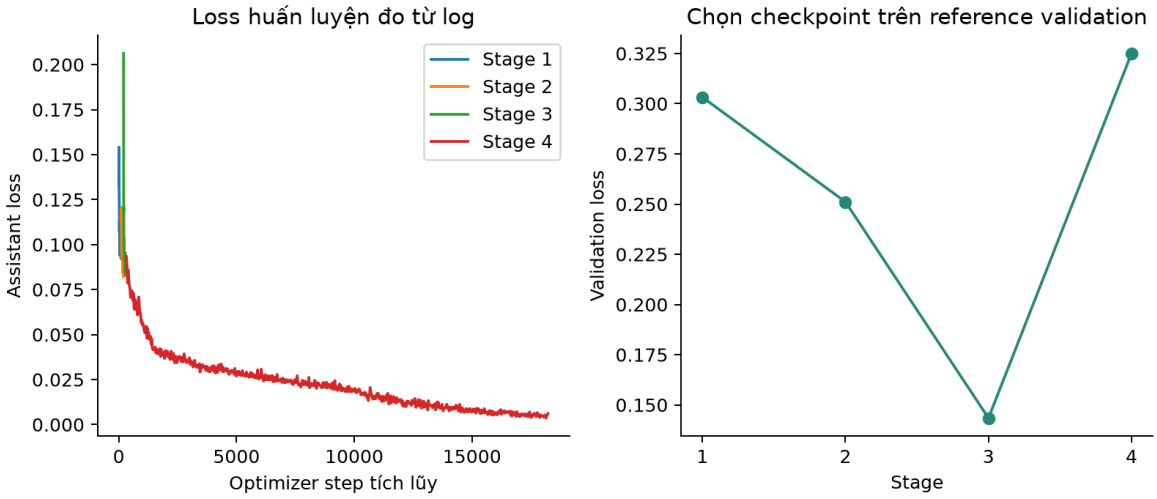
Các stage tiếp tục cập nhật cùng adapter trên tập tích lũy, vì vậy tổng số lượt một mẫu dễ được nhìn thấy có thể lớn hơn một mẫu khó. Khi mở rộng corpus, cần xem xét ngân sách cập nhật và cân bằng curriculum để tránh lặp quá nhiều template dễ. Hiệu quả curriculum cần một đối chứng được thiết kế trước nếu muốn kết luận nó tốt hơn việc train trộn toàn bộ dữ liệu.

Lượt cục bộ hoàn tất trên RTX 3050 Laptop 4 GB, tiếp tục adapter Qwen2.5-Coder-1.5B với optimizer mới. Stage 4 đi hết 35.856 dòng train trong một epoch; tổng 18,228 optimizer step gồm các stage warmup. Checkpoint được chọn là stage 3 theo validation trên từng tham chiếu, tách khỏi test. Checkpoint 1.5B được chọn đã được áp dụng vào runtime chính sau khi chuyển GGUF và kiểm định lại; bản 7B được giữ để rollback.

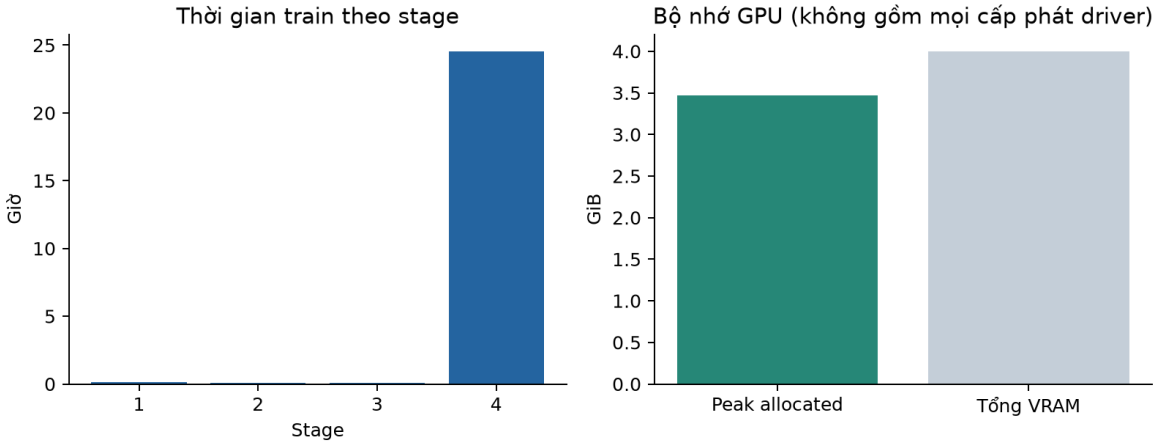
Bảng 6.10. Kết quả lượt train cục bộ và lựa chọn checkpoint

Stage	Dòng train đủ điều kiện	Optimizer step	Validation loss
1	8,964	100	0.303209
2	17,928	100	0.251091
3	26,892	100	0.143439
4	35,856	17,928	0.325025

Validation loss giảm đến stage 3 rồi tăng ở stage 4 dù training loss tiếp tục thấp. Đây là dấu hiệu cần kiểm tra khả năng khái quát hóa, phù hợp với nguy cơ học quá sát các template lặp; loss đơn lẻ chưa xác định được toàn bộ nguyên nhân. Vì tiêu chí chọn stage đã khóa theo validation, adapter dùng trong benchmark là checkpoint stage 3, tương ứng 300 bước tích lũy. Toàn bộ lượt chạy đã hoàn tất một epoch ở stage 4, nhưng không được mô tả adapter được chọn là đã học hết tập train trong epoch đó. Checkpoint stage 4 vẫn được giữ để nghiên cứu tiếp.



Hình 6.11. Loss train và validation của các stage đã hoàn tất.



Hình 6.12. Thời gian train và bộ nhớ GPU đo được.

CHƯƠNG 7. KẾT QUẢ VÀ ĐÁNH GIÁ

Chương này tổng hợp các kết quả đã có bằng chứng của hệ thống, gồm dữ liệu telemetry, phân loại event, fine-tune LLM, kiểm tra cú pháp, vòng đời rule và enforcement HTTP. Nguồn chính là artifact Colab trong Drive, manifest cục bộ và hồ sơ lab ngày 02/11/2026; phân tích corpus và bộ preflight replay được bổ sung ngày 06/11/2026. Những ảnh dashboard đã lưu không được xem là trạng thái trực tiếp của máy tại ngày hoàn thiện tài liệu.

Kết quả được trình bày theo từng đơn vị đo. Số event không được cộng với số dòng SFT; syntax pass không được thay cho recall; số case kiểm thử parity không được thay cho số capture độc lập; trạng thái Active không được dùng một mình để chứng minh chặn. Cách giữ mẫu số giúp kết luận vừa phản ánh đúng công việc đã làm vừa chỉ ra thí nghiệm cần bổ sung.

7.1. Kết quả triển khai và mức bao phủ giám sát

Kết quả tích hợp được đọc cùng coverage sensor và khả năng quan sát trên giao diện. Số record, trạng thái dịch vụ và incident là các bằng chứng bổ sung, nhưng không tự tạo thành tỷ lệ phát hiện của toàn hệ thống.

Các phép thử chạy với policy tự động bật và model boot check đạt. Request thật đi qua gateway, sau đó API và log sensor được đối chiếu. Mục đích là kiểm chứng hệ thống thực hiện đúng hợp đồng đã định nghĩa, không đưa lưu lượng vào một mạng hoặc mục tiêu bên ngoài lab.

Bảng 7.1. Tổng hợp các phép thử vận hành có hồ sơ

Nhóm thử	Kết quả	Phạm vi
Hồi quy	82/82 đạt	Backend, GenTI, corpus, native và điều khiển
Parity mở rộng	24/24 PCAP	Suricata và gateway cùng quyết định
HTTP thật	24/24 case	Method, encoding, URI và ranh giới

Nhóm thử	Kết quả	Phạm vi
Vòng đời IPS	12/12 bước	Native, Shadow, Active, rollback và lỗi worker
Chu kỳ model mới	Đạt	Tự chọn mẫu, sinh rule, native, Shadow, Active
IDS → Sự cố	Đạt	3 probe → alert và Incident ID

Nguồn: Các summary.json ngày 02/11 và regression.log; bản sao gọn nằm trong evidence/ của gói báo cáo.

Hồ sơ cho thấy luồng chính hoạt động qua nhiều tầng: model fine-tune tạo Candidate, engine native kiểm nội dung cuối, worker theo dõi Shadow rồi tự Active, gateway phản ứng trước DVWA và dashboard giữ dữ liệu. Đây là kết quả tích hợp có giá trị vì nó nối đầu ra mô hình với tác động có thể kiểm tra, thay vì chỉ trình bày một rule được sinh trong notebook.

Tuy nhiên số lượt hồi quy đạt không phải tỷ lệ khả dụng theo thời gian. Bộ 82 test kiểm một tập điều kiện cụ thể và không thay thế thử tải dài hạn. Báo cáo chưa có số uptime nhiều ngày hoặc dữ liệu failure rate để công bố độ sẵn sàng production. Những kết quả ổn định được giới hạn ở các phép restart, stale version và pause worker đã lưu.

Mức độ bao phủ của các sensor. Phân bố record và ý nghĩa coverage. Telemetry tổng hợp có 49.114 record Nginx, 86.228 record Suricata, 950 record Falco và 950 record Wazuh. Chênh lệch này phản ánh loại record, cách thu và kịch bản, không tự chứng minh Suricata phát hiện nhiều tấn công hơn hoặc Falco kém hiệu quả hơn. Một flow có thể sinh nhiều event, còn một runtime alert chỉ xuất hiện khi thỏa điều kiện cụ thể.

Để đo coverage sensor đúng, cần đặt mẫu số là số case thuộc khả năng quan sát và có oracle của sensor đó. Dữ liệu hiện đủ để công bố tổng record theo nguồn nhưng chưa đủ một confusion matrix độc lập cho từng sensor trên toàn bộ kịch bản. Vì vậy báo cáo không tự tạo tỷ lệ phát hiện riêng cho Falco hoặc Wazuh từ số lượng record.

Kiểm chứng IDS và incident. Ba request probe vô hại tới prefix acceptance tạo ba alert SID 9000001 trên Suricata. Theo log chuẩn hóa, có 6 event Suricata gồm

HTTP và alert, cùng 3 event Nginx; các alert được liên kết sang một Incident ID. Request trả 404 từ ứng dụng nhưng vẫn được sensor quan sát: 404 không phủ nhận khả năng IDS phân tích request, và cũng không chứng minh khai thác thành công. Hồ sơ marker và ID được giữ trong evidence/ids-incidents.json.

Probe IDS được thiết kế vô hại để kiểm đường alert và liên kết incident, nên kết quả này là bằng chứng tích hợp chứ không phải đánh giá khả năng phát hiện một khai thác thành công. Nó cũng chứng minh status ứng dụng và tín hiệu sensor phải được đọc riêng: request nhận 404 vẫn có thể tạo một alert đúng theo rule kiểm thử.

Kết quả kiểm tra cổng và giao diện. Gateway của lab sử dụng loopback 8080 để truy cập ứng dụng, dashboard và API theo đường dẫn. Wazuh Manager API 55000 là dịch vụ quản trị riêng và cần fragment tương ứng. Hồ sơ giao diện đã có các ảnh AI, Candidate, Active và Phản ứng; các ảnh minh họa chức năng và thông tin được hiển thị, không thay việc kiểm định endpoint hoặc xác thực API.

Điểm hữu ích của giao diện chính là hiển thị kết quả của đúng model người thực hiện đã fine-tune, trạng thái smoke test và lý do Candidate chờ. Người vận hành có thể đọc nội dung rule, native receipt, bộ đếm Shadow và lease mà không phải chọn evidence hoặc xác nhận thủ công cho từng vòng GenTI. Việc giữ nội dung trong PostgreSQL giúp tải lại trang vẫn truy xuất được kết quả.

Tính đúng của giao diện cần được đọc cùng hồ sơ backend: một rule có trạng thái Active nhưng không được gateway dùng vẫn là lỗi tích hợp. Trong các case chặn đã lưu, audit và upstream là căn cứ chính; ảnh Phản ứng là phần minh họa giúp theo dõi quyết định đó. Tài liệu không công bố giao diện Wazuh Dashboard đã được cài chỉ từ việc manager API sẵn sàng.

7.2. Đánh giá phân loại hiệu năng và phương pháp hybrid

Confusion count, ngưỡng và các chỉ số giúp so sánh model trên cùng dữ liệu. Những kết quả này được đặt cạnh điều kiện hiệu năng và phạm vi đo để tránh suy tốc độ hoặc khả năng production từ một snapshot offline.

Công thức và cách đọc. Tỷ lệ parse JSON đánh giá đầu ra theo hợp đồng. Syntax native đánh giá rule có được engine chấp nhận hay không. Replay trên PCAP có nhãn đánh giá quyết định phát hiện trong bộ case cụ thể. Shadow đo số lượt quan

sát thật trước triển khai. Enforcement xác nhận tác động chặn. Mỗi mức có mẫu số, thời điểm và điều kiện riêng.

Chỉ số phân loại được tính bằng:

$$Precision = \frac{TP}{TP + FP}, \qquad Recall = \frac{TP}{TP + FN},$$
$$F_1 = \frac{2TP}{2TP + FP + FN}, \quad FPR = \frac{FP}{FP + TN}.$$

TP/TN cần oracle hoặc nhãn phù hợp. Số URI khớp trong Shadow không tự trở thành TP, vì người vận hành chưa cung cấp nhãn độc lập cho mọi request live.

PR-AUC xem chất lượng xếp hạng trên nhiều ngưỡng và hữu ích khi attack hiếm, trong khi confusion matrix phụ thuộc ngưỡng được chọn. Giá trị average precision trong artifact cần được đọc theo cách tính của thư viện đã dùng, không tự thay bằng diện tích hình học của một đường PR khác. ROC-AUC cao cũng không đủ đảm bảo số báo nhầm chấp nhận được trong traffic có rất nhiều benign.

Nếu một chỉ số có mẫu số bằng không, phải giữ undefined hoặc null và nêu lý do. Ví dụ không có alert thì precision không thể được gọi là 100%. Khi so sánh model, cần cùng test, cùng nhãn và công bố ngưỡng. Các mô hình dùng biểu diễn khác nhau vẫn có thể được so trên cùng tập, nhưng kết quả không chứng minh chúng nhận cùng lượng thông tin đầu vào.

Chỉ số phân loại event DVWA. Preprocessing ghi 66.547 train, 24.320 validation và 21.949 test. Test gồm 8.926 attack và 13.023 benign; tổng confusion count của từng model phải bằng 21.949. Các ngưỡng được lấy nguyên từ hồ sơ; Fusion OOF và weighted hybrid dùng 0,55, các dòng còn lại 0,5.

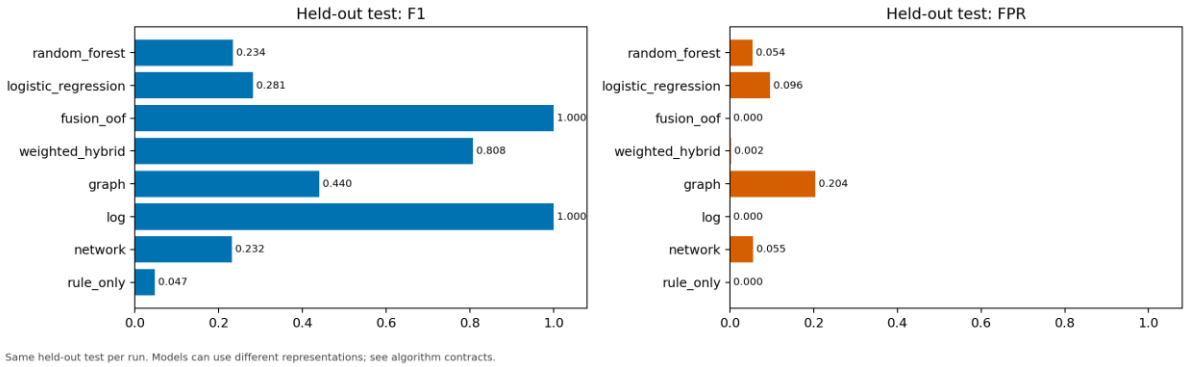
Bảng 7.2. Kết quả phân loại test DVWA, cùng mẫu số 21.949 event

Model	Precision %	Recall %	F1 %	FPR %
rule_only	100.000	2.409	4.704	0.000
network	63.820	14.150	23.164	5.498
log	100.000	100.000	100.000	0.000
graph	55.145	36.623	44.015	20.418
weighted_hybrid	99.557	67.948	80.770	0.207

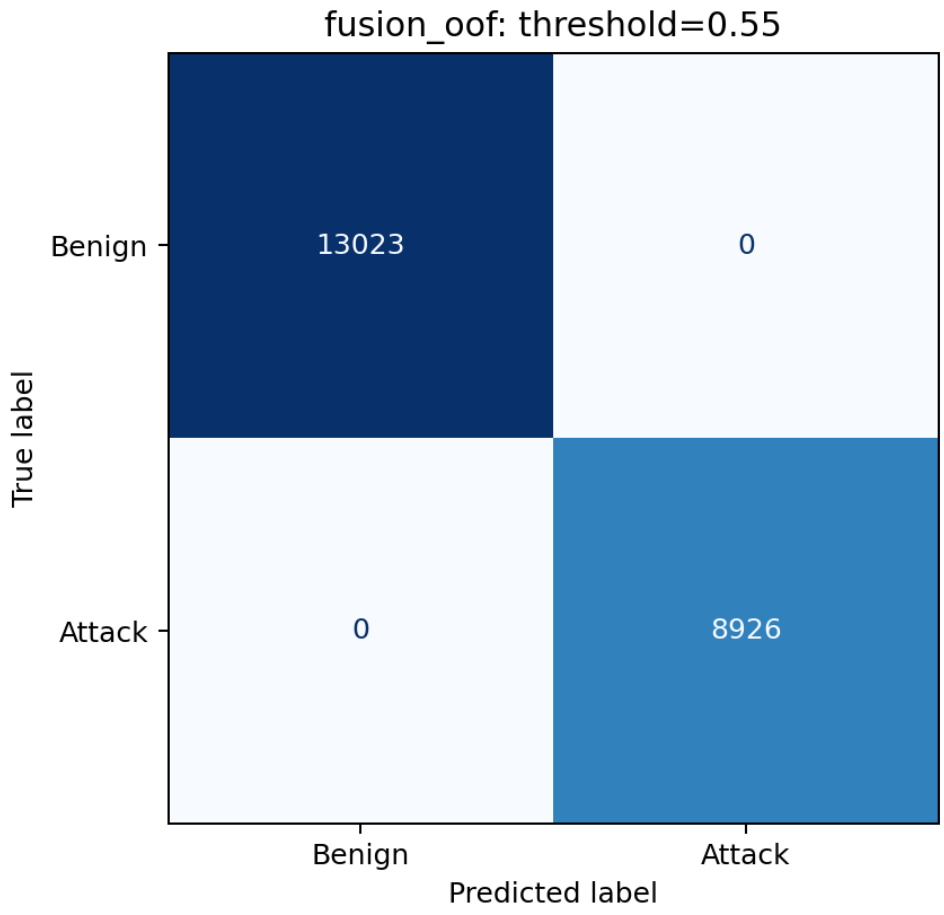
Model	Precision %	Recall %	F1 %	FPR %
fusion_oof	100.000	100.000	100.000	0.000
logistic_regression	57.236	18.653	28.137	9.552
random_forest	64.593	14.307	23.425	5.375

Nguồn: comparison.json của run DVWA trong Drive. Các giá trị biểu diễn phần trăm; số liệu gốc trong evidence/dvwa-comparison.json.

Log và Fusion OOF đạt F1 bằng 1 trong snapshot này, trong khi Network và Graph thấp hơn nhiều. Điều cần làm từ kết quả này là kiểm tra dữ liệu, độ tương đồng payload và nhãn, không suy rằng hệ thống sẽ không có báo động nhầm ngoài thực địa. Các model có thể sử dụng biểu diễn khác nhau; chỉ số trên cùng test không chứng minh các đặc trưng có mức thông tin tương đương. Event cùng chiến dịch hoặc biến thể gần nhau vẫn có thể khiến đánh giá lạc quan, kể cả khi đã tách một số định danh.



Hình 7.1. Hình gốc từ Drive: F1 và FPR các nhánh phân loại trên held-out test DVWA. Đây là kết quả phân loại event, không phải đánh giá GenTI.



Hình 7.2. Confusion matrix Fusion OOF: 8.926 TP, 13.023 TN, FP=FN=0 trong snapshot DVWA.

Confusion count và PR AUC của cùng tập test DVWA 21.949 event

Bảng 7.3. Confusion count và PR AUC trên test DVWA

Model	TP	FP	TN	FN	PR AUC
rule_only	215	0	13023	8711	0.425681
network	1263	716	12307	7663	0.518226
log	8926	0	13023	0	1.000000
graph	3269	2659	10364	5657	0.599334
weighted_hybrid	6065	27	12996	2861	0.995949
fusion_oof	8926	0	13023	0	1.000000
logistic_regression	1665	1244	11779	7261	0.513127

Model	TP	FP	TN	FN	PR AUC
random_forest	1277	700	12323	7649	0.519160

Bảng confusion cho phép kiểm tra tổng mẫu số và giải thích các chỉ số làm tròn. Chẳng hạn rule_only chỉ có 215 TP và 8.711 FN; precision 100% đi cùng recall rất thấp. Weighted hybrid có 6.065 TP, 27 FP và 2.861 FN; nó giảm báo nhầm nhiều so với một số nhánh nhưng vẫn bỏ sót trong snapshot. Vì vậy không nên chọn một chỉ số riêng để mô tả toàn bộ chất lượng.

Latency throughput và tài nguyên Docker. Những thời gian đã có căn cứ. Tổng train_runtime của bốn stage GenTI là 5.010,2981 giây, xấp xỉ 83,5 phút trên L4 theo log. Đây là thời gian train được Trainer ghi, chưa gồm toàn bộ chuẩn bị, tải model, sinh đầu ra, merge và export. Một chu kỳ Shadow đã lưu đủ điều kiện sau 63 giây; con số này phụ thuộc các URI đã được gửi và ngưỡng quan sát, không phải tốc độ inference của model.

Hồ sơ triển khai có giới hạn CPU và memory trong Compose, nhưng giới hạn đó không phải mức sử dụng đo được. Báo cáo chưa có đầy đủ time-series CPU/RAM, peak VRAM của run GenTI hoặc benchmark p95 gateway dưới tải. Vì thế không đưa một bảng số hiệu năng giả định để lấp các mục của đề cương.

Thiết kế phép đo còn thiếu. Bảng 7.4. Trạng thái bằng chứng hiệu năng

Đại lượng	Trạng thái bằng chứng	Việc cần làm tiếp
Train runtime theo stage	Có log Trainer	Giữ cấu hình cùng số liệu
Thời gian đủ Shadow ở case đã lưu	Có audit 63 giây	Không suy thành latency model
Thời gian review generation theo phân vị	Chưa có bộ đo đầy đủ	Chạy nhiều task cố định, lưu thời gian

Đại lượng	Trạng thái bằng chứng	Việc cần làm tiếp
p95 p99 kiểm tra gateway	Chưa có phép đo tải đủ	So baseline và policy bật
Throughput toàn pipeline	Chưa công bố từ benchmark tải	Đếm request và event cùng cửa sổ
Peak CPU RAM VRAM	Chưa có time-series GenTI đầy đủ	Thu telemetry tài nguyên trong run mới

Để đo so sánh, cần warm-up và giữ cùng dataset, tốc độ gửi, số client, cấu hình Docker. Nên báo cả lỗi và timeout thay vì chỉ thống kê những request thành công. Một hệ thống từ chối nhiều request do checker lỗi có thể có latency thấp nhưng khả dụng kém; hai khía cạnh phải được báo cùng nhau.

So sánh rule only AI only và hybrid. Rule-only trong snapshot có precision cao nhưng recall thấp, cho thấy bộ rule được dùng chỉ bao phủ một phần nhãn attack. Network và các baseline tabular có recall thấp trên biểu diễn hiện tại; điều này có thể liên quan chất lượng đặc trưng, phân bố và ngưỡng, không đủ để kết luận mọi cách tiếp cận network đều không phù hợp. Graph có FPR lớn hơn, cần phân tích graph construction và các node có nhiều benign bị gộp.

Log và Fusion OOF đạt confusion hoàn hảo trong snapshot. Đây là kết quả đã lưu và được trình bày nguyên trạng, nhưng cần kiểm tra độ tương đồng template, nhãn, nguồn text và tương quan chiến dịch trước khi suy rộng. Fusion hoàn hảo khi một base branch đã hoàn hảo cũng chưa chứng minh lợi ích bổ sung độc lập của các nhánh còn lại. Ablation bỏ Log, bỏ marker ngầm và giữ nguồn ngoài miền sẽ cung cấp bằng chứng mạnh hơn.

Weighted hybrid đạt F1 khoảng 80,77% ở ngưỡng 0,55, thấp hơn OOF trong run này. Kết quả phù hợp với việc học một meta-model có thể tận dụng phi tuyến, nhưng khác biệt cũng có thể chịu ảnh hưởng chất lượng base score và dữ liệu. Muốn khẳng định phương pháp tốt hơn cần lặp theo nhóm hoặc nhiều seed, báo mức biến thiên và giữ test độc lập; hồ sơ hiện chưa đủ cho kiểm định ý nghĩa thống kê rộng.

7.3. Kiểm chứng tổng quát hóa phản ứng và sai sót

Holdout, benchmark ngoài miền và phép thử chặn cung cấp những loại bằng chứng khác nhau. Đối chiếu chúng cùng phân tích false positive và false negative giúp xác định hiệu quả đã chứng minh và phạm vi cần kiểm tiếp.

Unknown trong corpus GenTI. Unknown_test giữ 5 family không có trong train và có 240 dòng. Đầu ra fine-tune đạt 235/240 syntax theo hợp đồng. Kết quả này chứng minh mức tuân thủ cú pháp trên family holdout của corpus đã xây, không chứng minh phát hiện attack chưa biết bằng PCAP và không chứng minh chặn zero-day trong môi trường độc lập.

Nếu task unknown vẫn dùng template và nguồn tổng hợp tương tự train, khoảng cách phân bố có thể nhỏ hơn so với một ứng dụng thật mới. Do đó nên giữ thêm external test theo nguồn hoặc app và xây oracle cho capture độc lập. Số family cần được báo cùng số dòng; 240 paraphrase không phải 240 nhóm tấn công độc lập.

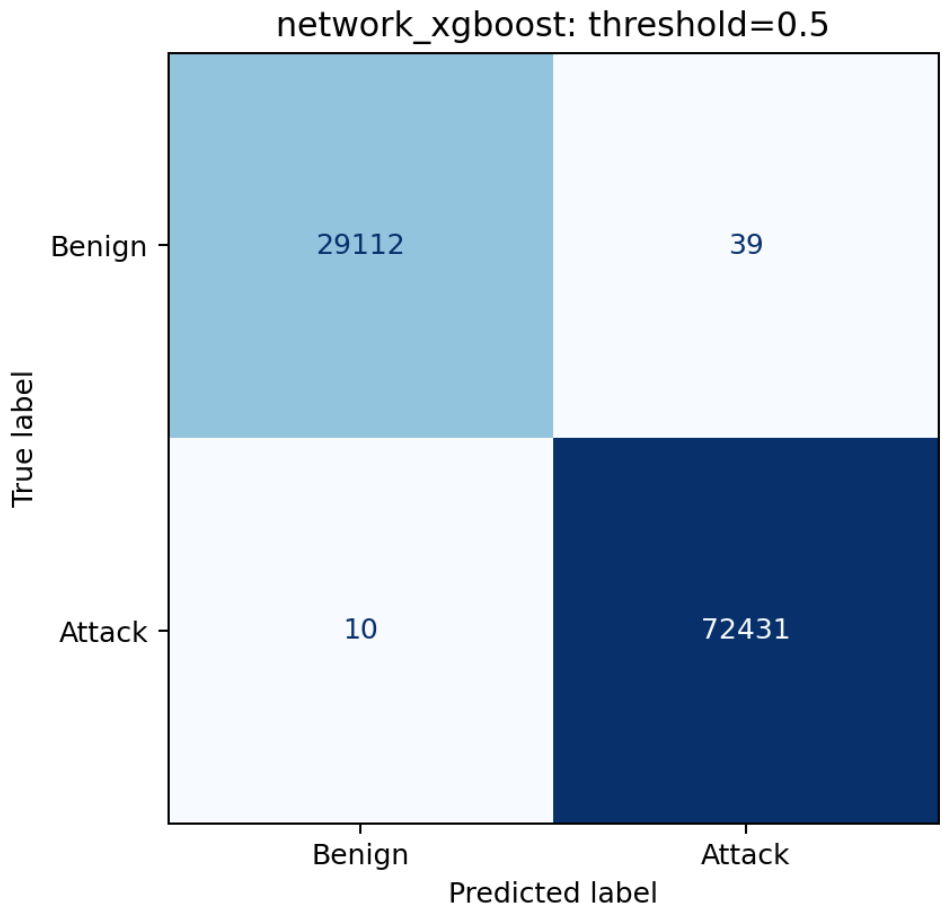
Benchmark CIC riêng. Hồ sơ CIC ghi rõ contract benchmark-only và deployable=false. Từ 2.830.743 dòng CSV, sau sampling và loại trùng/xung đột, giữ 507.960 dòng; train 304.776, validation 101.592 và test 101.592. Benchmark này dùng đặc trưng CIC; không thể nạp thẳng model vào luồng event lab nếu chưa có ánh xạ đặc trưng và kiểm định tương đương.

Bảng 7.5. Chỉ số benchmark CIC, không dùng làm model runtime

Tập	Precision %	Recall %	F1 %	FPR %
validation	99.9489	99.9848	99.9669	0.1269
test	99.9462	99.9862	99.9662	0.1338

Nguồn: report.json của run CIC: test TP=72.431, FP=39, TN=29.112, FN=10; contract CIC-native-benchmark-only.

Chính báo cáo nguồn lưu ý exact-deduplicated stratified rows chưa loại bỏ toàn bộ near-duplicate hoặc flow có tương quan. Các chỉ số rất cao cần được kiểm tra tiếp theo ngày, host hoặc miền dữ liệu khác trước khi dùng làm bảo đảm vận hành. Không đưa benchmark CIC vào số event telemetry của AHR-IDPS hoặc số cặp SFT.



Hình 7.3. Confusion matrix gốc của benchmark CIC. Mẫu số test là 101.592 dòng, độc lập với corpus rule và tập PCAP IPS.

Benchmark CIC giúp chứng minh đã có một đường xử lý dữ liệu flow riêng với confusion count rõ. Tuy nhiên contract benchmark-only không cho phép dùng kết quả đó làm chứng nhận model runtime AHR-IDPS. Cũng không được cộng 507.960 dòng benchmark vào số telemetry lab hoặc số cặp fine-tune rule.

Kết quả incident evidence và response. Rule tự động khi đã có rule Active. Rule được sinh trong chu kỳ tự động mới, qua native rồi vào Shadow trong khi hai rule Active khác vẫn hoạt động. Năm lần gửi lặp cùng một URI chỉ làm bộ đếm tăng một. Sau các request bổ sung, Shadow đạt 6 URI khớp, 20 không khớp, tổng 26 URI khác nhau; 63 giây sau bắt đầu đã đủ điều kiện và tự Active.

Nội dung thực đã lưu của rule là:

alert tcp any any -> any any (msg:"AHR lab xss-07"; flow:established,to_server; http.method; content:"GET"; bsize:3; http.uri; content:"/vulnerabilities/xss_r/?"; startswith; content:"<script"; nocase; sid:10484622; rev:1;)

Action của Suricata vẫn là alert. Khâu ngăn chặn được thực hiện ở gateway theo profile đã đối chiếu, không phải tự đổi alert thành drop hoặc reload sensor. Trường sensor_deployed=false vì thế phù hợp với kiến trúc.

Case này kiểm một rủi ro thực tế của chu trình tự động: một rule đang chặn không được làm mọi Shadow mới thiếu quan sát. Hồ sơ cho thấy bộ đếm unique và thứ tự ghi nhận đã hoạt động trong tình huống có nhiều rule. Dù vậy 26 URI trong case vẫn là dữ liệu có kiểm soát; không dùng nó để ước lượng tần suất tấn công ngoài lab.

Các case IPS đã lưu có đối chiếu status, header rule, upstream và bản ghi applied. Điều này cho phép kết luận request cụ thể bị chặn trước DVWA ở đường gateway. Các record Response Engine dạng lab-audit-wrapper chỉ chứng minh đã ghi audit theo chính sách; chúng không được tính như đã cô lập container hoặc chặn mạng host.

Điều kiện hoàn thành của một incident không nhất thiết là có block. Một incident có thể phục vụ điều tra và kết luận benign hoặc uncertain. Tương tự, một rule ngoài profile giữ Candidate có thể là đầu ra đúng của mô hình nhưng chưa được hỗ trợ thực thi tự động. Đánh giá cần tách chất lượng đề xuất và độ rộng điểm enforcement.

Phân tích false positive và điều chỉnh ngưỡng. False positive cần được nhóm theo endpoint, method, encoding, nguồn, family và loại benign. Những benign gần chữ ký như văn bản trích mã có thể khó hơn trang hợp lệ không chứa dấu hiệu. Nếu chỉ bổ sung benign quá dễ, FPR thấp không cung cấp nhiều thông tin. Mỗi nhóm lỗi cần giữ case và rule hash để tái hiện sau sửa.

False negative cũng cần phân biệt rule không sinh được, rule syntax lỗi, rule quá hẹp, case ngoài scope và sensor không có visibility. Trong hiệu quả đầu cuối, các lỗi sinh rule vẫn là task chưa được giải quyết; trong đánh giá riêng semantics, có thể báo điều kiện chỉ trên rule hợp lệ nhưng phải nêu rõ mẫu số. Lỗi hạ tầng không được che dưới một nhãn FN hoặc TP tùy tiện.

Ngưỡng phân loại được điều chỉnh trên validation theo mục tiêu vận hành, ví dụ giới hạn FPR rồi tối đa recall. Sau chọn ngưỡng, giữ cố định khi mở test. Với chữ ký literal của gateway, thay điều kiện không chỉ là đổi ngưỡng số: nó có thể thay

semantics và bắt buộc chạy lại syntax, replay, parity và vòng đời. Không chỉnh trực tiếp một rule Active rồi giữ receipt cũ.

Shadow giúp thấy mức match và nonmatch nhưng chưa có nhãn live độc lập, nên không thể tự chọn ngưỡng để đạt FPR mục tiêu từ bộ đếm này. Cần thêm quy trình review mẫu hoặc benchmark có oracle. Các feedback do hệ thống tự sinh phải được đánh dấu để tránh trở thành xác nhận vòng tròn khi train lại.

Hạn chế và nguy cơ ảnh hưởng kết quả

Thứ nhất, corpus SFT tổng hợp và chưa có human review khiến target còn mang giả định của bộ tạo dữ liệu.

Thứ hai, family holdout giúp giảm trùng nhưng số family test nhỏ và có các biến thể gần nhau.

Thứ ba, Colab chỉ kiểm syntax trong hồ sơ hiện có; unknown_test không chứng minh hiệu quả phát hiện chưa biết.

Thứ tư, lượng tử GGUF, engine Suricata và prompt review khác ngữ cảnh train đòi hỏi nghiệm thu runtime riêng.

Thứ năm, bộ native 50/50 và ma trận 24 là probe có kiểm soát cho một chữ ký đã giới hạn, không phải tập blind độc lập để báo cáo FPR thực địa.

Thứ sáu, AI review có thể giải thích sai hoặc phụ thuộc status được cung cấp; việc tự chọn benign chưa tạo ra oracle đáng tin tuyệt đối. Các gate kiểm định giảm rủi ro triển khai trong phạm vi lab nhưng không sửa được mọi hạn chế của dữ liệu.

Ngoài các giới hạn trên, dữ liệu event có tương quan và sample size theo family nhỏ làm các khoảng tin cậy dựa trên giả định độc lập trở nên lạc quan. Các phép thử thành công có thể chịu selection bias nếu chỉ lưu lượt cuối; việc giữ failed output và manifest giúp giảm rủi ro này. Hệ thống local cũng chưa phản ánh phân tán, tải lớn và độ trễ mạng giữa nhiều host.

Độ khớp của log giữa nhiều sensor có thể phụ thuộc marker thử và thứ tự gửi. Nếu triển khai thật không có marker, cần một thuật toán liên kết dựa trên trường tự nhiên và đo chất lượng liên kết. Những ảnh dashboard có giá trị minh họa nhưng không thay thế dữ liệu gốc. Báo cáo giữ các giới hạn này để người đọc đánh giá kết quả theo đúng phạm vi.

7.4. Đánh giá GenTI từ cú pháp đến phát hiện trên PCAP

Kết quả contract và syntax được phân biệt với DR và FPR có oracle. Việc giữ mẫu số và provenance của control cho phép diễn giải đúng thành công của model, đồng thời chỉ rõ bộ replay nào chưa có kết quả thực thi.

Sau khi chọn stage bằng validation, notebook sinh một lần trên test và unknown_test. Mỗi tập có 240 yêu cầu cho đối chứng tắt adapter và 240 yêu cầu cho model fine-tune. Các file candidate giữ nguyên đầu ra raw trong trường generated; bộ native evaluator đọc đầu ra theo hợp đồng JSON trước khi gọi Suricata. Do đó một mẫu có thể hỏng ở parser mà chưa từng đến bước syntax native.

Bảng 7.6. Kiểm tra hợp đồng đầu ra và syntax native trên Colab

Tập / chế độ	Mẫu	JSON + rule	Đến native	Syntax đạt
Test / tắt adapter	240	0	0	0
Test / fine-tune	240	240	239	211
Unknown / tắt adapter	240	0	0	0
Unknown / fine-tune	240	240	240	235

Nguồn: Bốn file candidates.jsonl và bốn file candidates.native.jsonl từ run GenTI trong Drive; script tổng hợp nằm trong gói báo cáo.

Ở model fine-tune, cả 240/240 đầu ra mỗi tập đều parse được JSON chứa trường rule. Test có 211/240 mẫu được Suricata chấp nhận, đạt 87,92% theo mẫu số toàn bộ yêu cầu. Unknown_test có 235/240 mẫu đạt, tương ứng 97,92%. Đây là tỷ lệ thành công từ yêu cầu đến cú pháp trong hợp đồng hiện dùng.

Trong test, evaluator ghi kết quả syntax cho 239 mẫu: 211 đạt và 28 lỗi native; một mẫu bị từ chối do phụ thuộc tài nguyên ngoài hoặc trạng thái cross-rule. Unknown_test có 235 đạt và 5 lỗi native trên 240 mẫu đã kiểm tra. Nếu chỉ lấy mẫu đã đến native, test đạt 211/239; báo cáo vẫn giữ 211/240 làm chỉ số chính để không bỏ mất mẫu bị từ chối.

Cả 240 đầu ra ở mỗi tập đối chứng đều bị lỗi parse JSON ngay từ đầu vào evaluator. Vì vậy cột native không có mẫu nào thực sự được gọi, thay vì “Suricata bác

240 rule". Điều này cho thấy fine-tune đã cải thiện sự tuân thủ hợp đồng trong thiết kế prompt được dùng. Nó chưa đủ để kết luận model nền không biết cú pháp Suricata, vì có thể model nền trả rule trong Markdown hoặc văn bản thay vì JSON bắt buộc. Một đối chứng chuẩn hóa định dạng riêng sẽ cần một thí nghiệm mới, không sửa ngược các file đã lưu.

Tất cả 960 dòng native kết hợp bốn file đều có replay rỗng hoặc chưa chạy. Không thể tính DR, Recall, FPR, độ chính xác ngăn chặn hoặc composite quality cho toàn corpus từ các file này. Các tỷ lệ 87,92% và 97,92% chỉ phản ánh syntax theo hợp đồng. Unknown_test cũng chỉ kiểm tra các family được giữ lại trong corpus tổng hợp, không đồng nghĩa thử thành công trước zero-day hay tấn công ngoài lab.

Một mẫu test bị từ chối trước native do phụ thuộc ngoài hoặc trạng thái cross-rule. Khi báo cáo tỷ lệ chính 211/240, mẫu đó vẫn nằm trong mẫu số hiệu quả đầu cuối. Tỷ lệ 211/239 có thể được dùng để mô tả có điều kiện những mẫu đã đến engine, nhưng không được thay thế âm thầm tỷ lệ chính. Cách báo hai mẫu số giúp chỉ ra điểm loại ở contract và điểm loại ở syntax.

Đối chứng tắt adapter thất bại JSON theo prompt hiện tại cho thấy tuân thủ hợp đồng được cải thiện sau fine-tune. Nó chưa chứng minh model nền không biết Suricata, vì định dạng trả lời có thể khác yêu cầu evaluator. Muốn tách năng lực định dạng và năng lực rule cần một đối chứng mới được chuẩn hóa, giữ nguyên hồ sơ gốc.

Tỷ lệ phát hiện trên PCAP và diễn giải. Trong biên nhận rule tự động đã chạy ngày 02/11, TP=50, TN=50, FP=0, FN=0 trên control tổng hợp của hồ sơ GET XSS:

$$DR = \frac{50}{50+0} = 100\%, \qquad FPR = \frac{0}{50+0} = 0\%.$$

Cách viết phù hợp là: "Trên bộ 50 attack và 50 benign tổng hợp thuộc hồ sơ GET XSS, rule đạt DR 100% và FPR 0%; kết quả chưa đại diện cho traffic độc lập hoặc toàn bộ các họ tấn công." Đây là diễn giải lại confusion count đã có, không phải phép đo mới trên 480 candidate.

Ngay cả nếu giả định các case là Bernoulli độc lập, Wilson 95% cho 50/50 phát hiện có cận dưới khoảng 92,9%; 0/50 báo nhầm có cận trên khoảng 7,1%. Tập control tổng hợp có tương quan không thỏa độc lập hoàn toàn, nên những cận này chỉ minh họa giới hạn mẫu nhỏ, không là khoảng tin cậy thực địa của hệ thống. Không có lỗi quan sát được không đồng nghĩa xác suất lỗi bằng không.

Kết luận được dữ liệu hỗ trợ. Kết quả 50/50 xác nhận rule đã kiểm phân biệt đúng các control tổng hợp được thiết kế cho profile GET XSS. Nó hỗ trợ quyết định gate trong lab, cùng với parity và Shadow. Tuy nhiên đây không phải kiểm thử blind độc lập; các biến thể có thể cùng dựa trên chữ ký và cùng cơ chế sinh. Vì vậy số quan sát không lỗi không đồng nghĩa xác suất lỗi bằng không trên traffic thật.

Không thể sử dụng native pass 97,92% ở unknown_test như DR 97,92%. Hai chỉ số có oracle và mẫu số khác nhau: syntax hỏi engine nhận rule, DR hỏi attack có nhãn trong scope có tạo alert đúng không. Báo cáo giữ cách diễn giải này trong bảng và kết luận để tránh biến một kết quả kỹ thuật thành tuyên bố phát hiện không có dữ liệu hỗ trợ.

Trạng thái bộ replay mở rộng. Bộ mới có 1.280 PCAP khác hash cho 480 task candidate, mỗi task 3 attack và 5 benign. Tổng 3.840 quyết định là số lượt dự kiến; các paraphrase dùng chung case nên không phải 3.840 capture độc lập. Hai tập đã qua preflight về ID, family, manifest, checksum và nhãn. Trạng thái vẫn là preflight_only, metrics null vì chưa chạy native replay bộ mới.

Các công cụ evaluator và hợp đồng liên quan đã có 17 test đạt trong lượt chuẩn bị. Kết quả test công cụ giúp kiểm tra cách xử lý lỗi và mẫu số, nhưng không tạo DR cho các candidate. Khi chạy replay, cần lưu output mới và bổ sung bảng theo family, category, source cùng số rule và PCAP độc lập.

Kiểm chứng bộ luật nền và benchmark nhánh fine-tune cục bộ

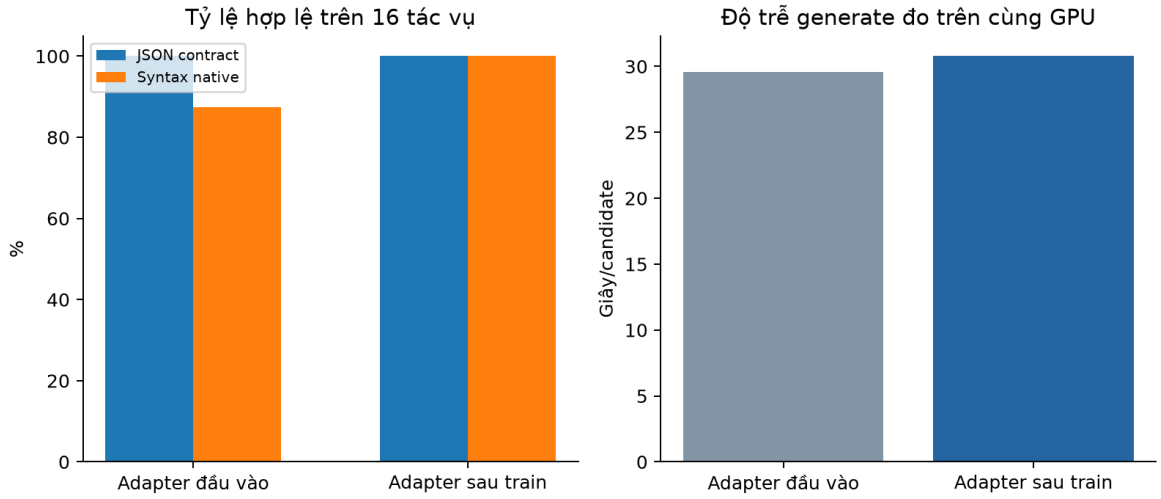
Bộ luật nền đã được kiểm bằng 130 trường hợp Suricata (33 dương, 33 lành, 64 ranh giới method/endpoint), 13 tình huống syscall Falco và 118 trường hợp Wazuh. Cả 10 luật Falco mới có cảnh báo runtime và ánh xạ tương ứng trong Wazuh. Đây là kiểm chứng cấu hình có kiểm soát; không dùng số case đạt để suy ra DR/FPR trên traffic độc lập.

Benchmark khóa trước train gồm 16 tác vụ từ test và unknown-test, sinh 32 candidate trên cùng model nền 1.5B với adapter đầu vào và adapter sau train. JSON contract và syntax được tính trên mọi candidate; DR/FPR/F1 replay được tính có điều kiện trên candidate qua syntax. Fixture lấy từ corpus nên đây là diagnostic theo tác vụ, chưa phải đánh giá capture độc lập hoặc chất lượng production.

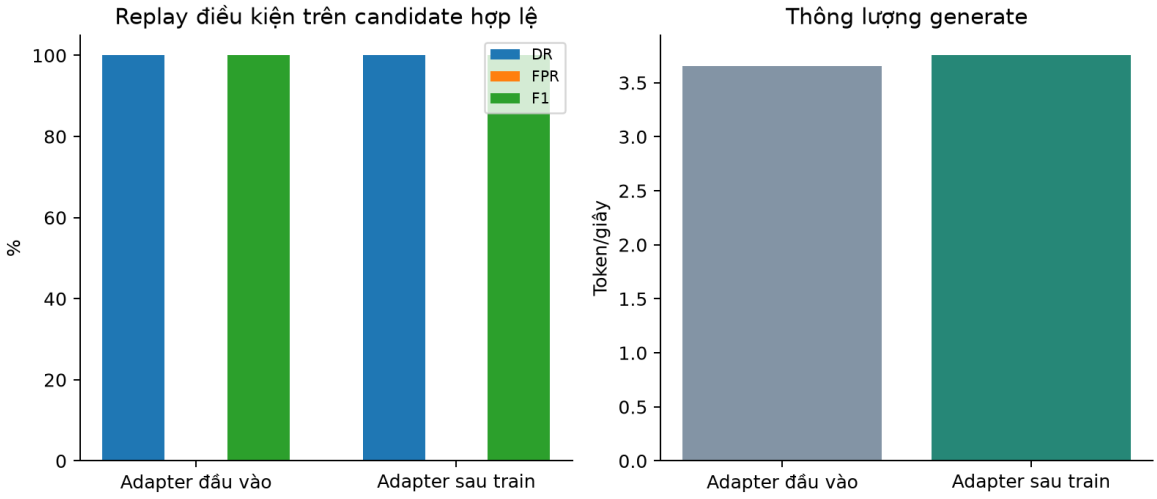
Bộ diagnostic đã khóa bao phủ 10 họ, với 10 tác vụ mức khó 1 và sáu tác vụ mức khó 2; chưa kiểm mức 3–4 trong lượt benchmark này. Không lựa chọn lại task sau khi xem kết quả. Mỗi tác vụ có ba control dương và năm phản ví dụ; các quyết định rule–case cùng template có tương quan, vì vậy không xem chúng là các cuộc tấn công độc lập.

Bảng 7.7. Benchmark diagnostic của adapter cục bộ

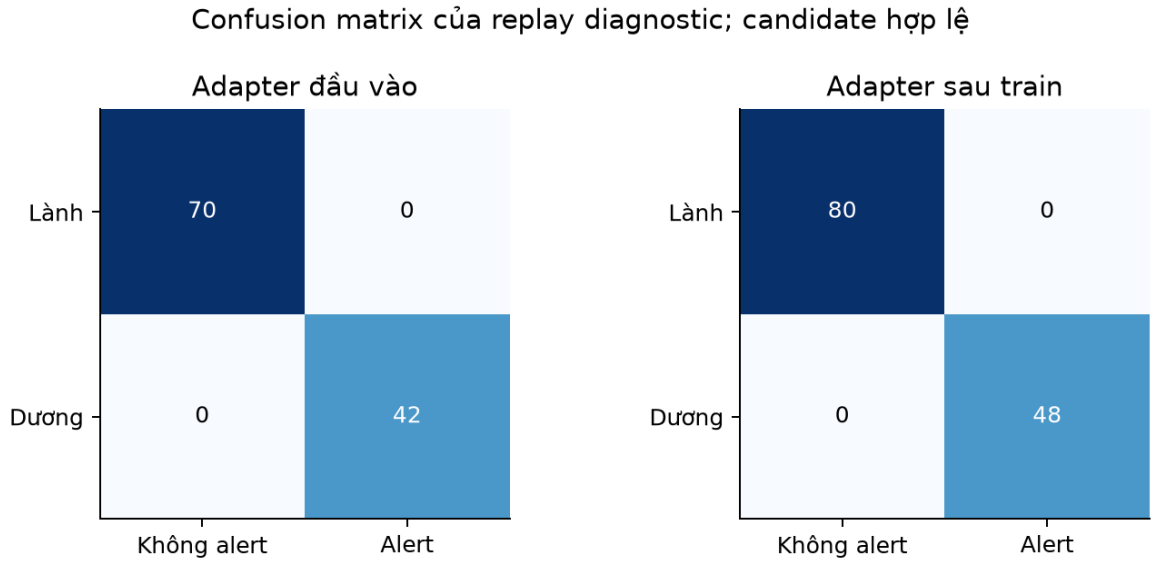
Chỉ số	Adapter đầu vào	Adapter chọn stage 3
Contract JSON	16/16	16/16
Syntax native	14/16	16/16
DR trên candidate hợp lệ	100.00%	100.00%
FPR trên candidate hợp lệ	0.00%	0.00%
F1 trên candidate hợp lệ	100.00%	100.00%
Generate trung bình (giây)	29.61	30.78
Generate token/giây	3.65	3.75



Hình 7.4. Contract, syntax native và độ trễ generate trước–sau fine-tune.



Hình 7.5. DR, FPR, F1 có điều kiện và thông lượng generate trên diagnostic.



Hình 7.6. Confusion matrix trên các quyết định rule–case có syntax hợp lệ.

Nghiệm thu mô hình chính và chu trình chặn HTTP ngày 12/11/2026

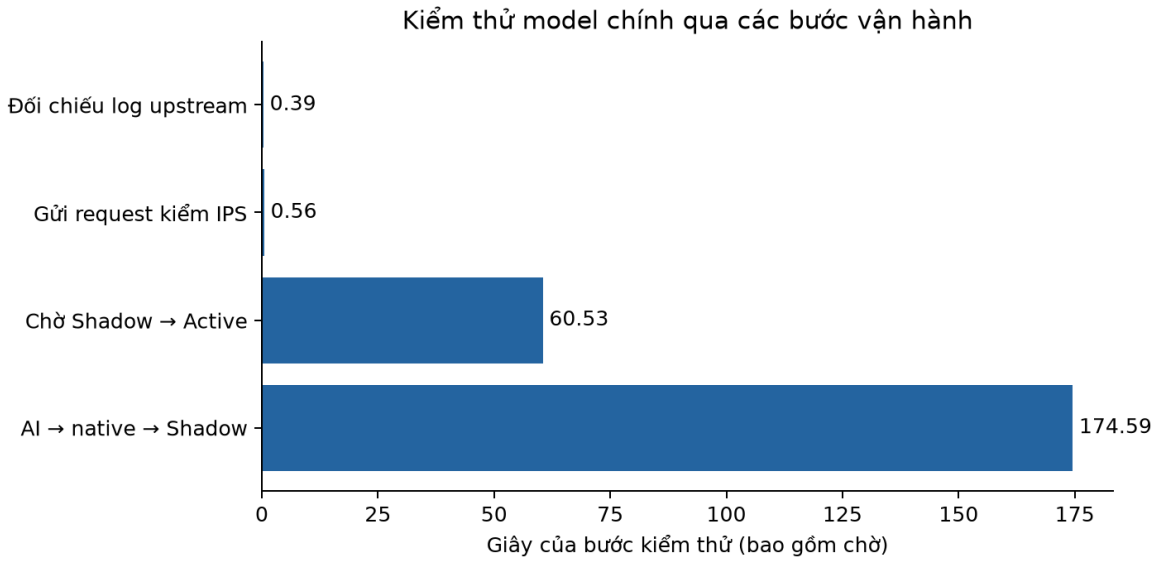
Model đang chạy có artifact ecc4941e369a49a2833907a208029245e183708234ee2e21e4d18b3ddca96697. Startup smoke test dùng đúng startup ID mới. Từ request thật qua Nginx, model tự chọn bằng chứng và phản ví dụ, sinh rule genti-b69220d1ac9e403a49a95da3; rule này mang cùng artifact. Worker kiểm Suricata với 50 control dương và 50 control lành cùng đối chiếu gateway, sau đó tự chuyển Shadow và Active khi đủ các ngưỡng đã khóa. Không đổi ngưỡng và không dùng thao tác approve tay để làm phép thử đạt.

Bảng 7.8. Kiểm chứng luồng vận hành với model chính mới

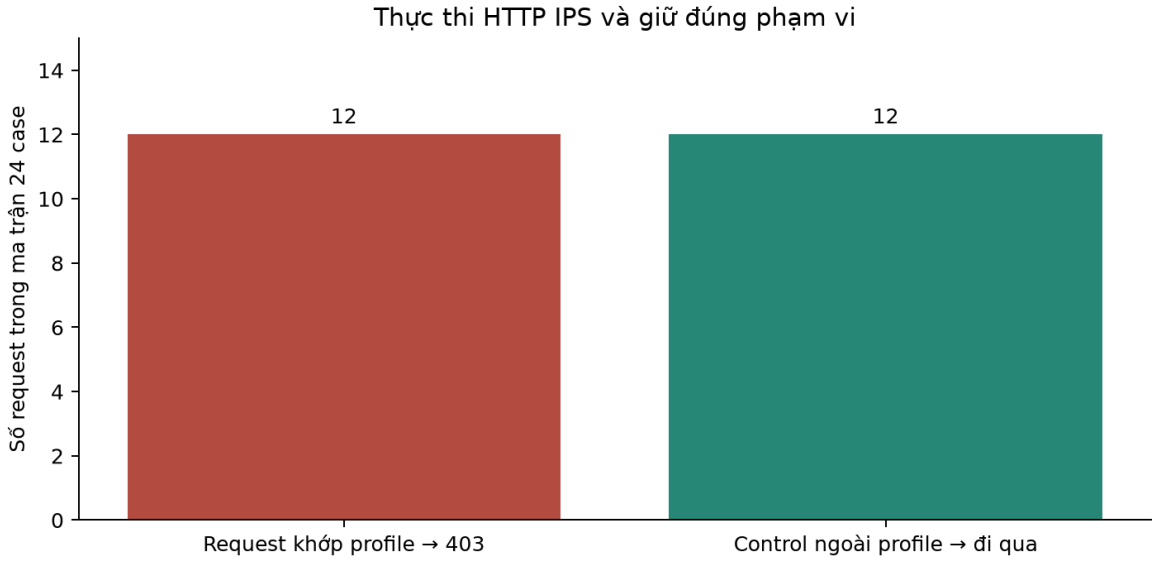
Kiểm tra	Kết quả	Thời gian (giây)
Model, hash và heartbeat	Đạt	0.16
Endpoint nội bộ không lộ ra gateway	Đạt	0.02
Policy tự động và rule cũ đã nghỉ	Đạt	0.12
HTTP thật tạo bằng chứng mới	Đạt	0.06
Model sinh rule → native → Shadow	Đạt	174.59
Shadow quan sát, chưa chặn	Đạt	1.11
Worker tự chuyển Active	Đạt	60.53
Chặn request khớp, cho mẫu lành qua	Đạt	0.56
Tắt/bật policy và chống revision cũ	Đạt	0.23
Lịch sử rule và phản ứng có thể tra cứu	Đạt	5.28
Chặn trước upstream DVWA	Đạt	0.39
Worker gián đoạn và tự phục hồi	Đạt	18.22

Biên nhận native của rule đạt TP=50, TN=50, FP=0, FN=0 trên control tổng hợp. Trong mẫu HTTP Active, 5 request bị trả 403 với upstream trống, còn 5 request lành có upstream DVWA. Ma trận mở rộng có 24 case so sánh method, encoding và đường dẫn; 12 request khớp profile bị chặn và 12 control không bị chặn. Các control ngoài profile không được diễn giải là mọi cuộc tấn công ở phạm vi khác đã được bảo vệ.

Khi chuyển model, các rule của artifact trước không được tái sử dụng để chứng nhận model mới. Một lượt thử đầu ghi nhận HTTP 500 trong khoảng fail-closed trước khi worker tự vô hiệu hóa các lease cũ; hồ sơ lỗi được giữ và lượt nghiệm thu sau chờ trạng thái nghỉ hoàn tất. Khi chủ động pause worker, đường được bảo vệ cũng fail-closed; health endpoint vẫn hoạt động và chặn được phục hồi sau unpause. IPS ở đây là Nginx auth_request trên HTTP lab; Suricata vẫn là IDS và engine kiểm rule, không phải NFQUEUE inline IPS.



Hình 7.7. Thời gian các bước nghiệm thu, gồm thời gian chờ của phép thử.



Hình 7.8. Kết quả 24 request kiểm phạm vi HTTP IPS trên runtime mới.

7.5. Mức đáp ứng mục tiêu và hướng phát triển

Bảng 7.9. Đối chiếu mục tiêu, kết quả có căn cứ

Mục tiêu	Kết quả có căn cứ	Giới hạn còn lại
Thu thập đa nguồn	Manifest 137.242 telemetry event	Chưa có recall toàn bộ sensor theo oracle độc lập
Huấn luyện phân loại	Artifact và metrics DVWA, CIC riêng	Cần external holdout và kiểm leakage sâu hơn

Mục tiêu	Kết quả có căn cứ	Giới hạn còn lại
Fine tune LLM	Qwen 1.5B cục bộ, chọn stage 3 theo validation; đã áp dụng làm model chính	Corpus một nguồn, chưa human review
Sinh rule đúng hợp đồng	JSON và syntax theo tập cố định	Syntax chưa là phát hiện
Tự động vòng đời	Native Shadow Active có audit	Chỉ grammar được kiểm chứng
Ngăn chặn thật	Case gateway block trước DVWA	Chưa là IPS toàn mạng hoặc POST body
Truy vết và hiển thị	Rule identity, receipt, incident, response	Chưa có cảnh báo ngoài trình duyệt
Đánh giá PCAP rộng	Kit và preflight đã chuẩn bị	Chưa chạy bộ mới hoặc external capture

Kế hoạch nghiệm thu tiếp theo. Giữ checkpoint đã chọn, model/tokenizer/prompt hash, config, raw candidate và split. Chạy syntax để xác định đầu ra qua hợp đồng, sau đó PCAP theo task, báo cả rule invalid và engine errors. Tập diagnostic có thể chạy trước để phát hiện lỗi điều kiện; khi dùng kết quả đó sửa prompt/model, nó trở thành development, không phải test kín cuối.

Tiếp theo, thu hoặc tuyển chọn bộ capture độc lập khác phiên/nguồn, gắn oracle và scope, chạy một lần candidate cuối cùng rồi phân tích false positive/false negative theo category, method, encoding và ứng dụng. Cuối cùng mới nghiệm thu trên runtime đích và Shadow/Active theo gate đang có. Rule đạt test offline không được tự mở rộng phạm vi IPS của ứng dụng.

Bước tiếp theo là mở rộng từng profile có điều kiện, không mở đồng thời mọi loại tấn công. Mỗi profile mới cần contract trường quan sát, rule grammar, native case, benign khó, parity gateway và rollback. Cuối cùng mới thử tải, cảnh báo ngoài trình duyệt và đường mạng Kali hoặc IPS inline nếu mục tiêu đồ án được mở rộng tới các thành phần này.

Kết luận đánh giá. AHR-IDPS đã chứng minh được một chu trình từ bằng chứng cảm biến tới rule do model fine-tune tạo ra, kiểm định độc lập và thực thi HTTP có giới hạn. Run L4 có metadata và loss; đầu ra syntax có mẫu số; hồ sơ lab có kiểm tra parity, vòng đời và dấu vết chặn trước ứng dụng. Đây là những kết quả có thể bảo vệ bằng artifact của đề tài.

Giới hạn còn lại là chất lượng tham chiếu chủ yếu được kiểm trên dữ liệu tổng hợp, mức độc lập của capture và review chuyên gia chưa đủ. Corpus 50.000 mẫu tăng số prompt và phạm vi thực nghiệm, nhưng vẫn chỉ có 1.696 rule tham chiếu và 53 họ; cần đánh giá theo đơn vị này khi diễn giải khả năng tổng quát hóa.

TÀI LIỆU THAM KHẢO VÀ NGUỒN THỰC NGHIỆM

[1] OISF. Suricata command line options, tài liệu cho kiểm tra cấu hình và đọc PCAP. https://docs.suricata.io/en/suricata-8.0.6/command-line-options.html

[2] OISF. EVE JSON output, tài liệu định dạng cảnh báo và metadata. https://docs.suricata.io/en/latest/output/eve/eve-json-output.html

[3] OISF. Suricata Verify, kho kiểm thử engine gồm cấu hình, PCAP và expected output. https://github.com/OISF/suricata-verify

[4] Wazuh. Wazuh server API và securing API, tài liệu quản trị manager qua HTTPS. https://documentation.wazuh.com/current/user-manual/api/index.html và https://documentation.wazuh.com/current/user-manual/api/securing-api.html

[5] Hugging Face. PEFT quantization, hướng dẫn huấn luyện adapter trên model lượng tử. https://huggingface.co/docs/peft/developer_guides/quantization

[6] Qwen. Qwen2.5-Coder-7B-Instruct, model card của model nền. https://huggingface.co/Qwen/Qwen2.5-Coder-7B-Instruct

[7] Canadian Institute for Cybersecurity. CIC-IDS2017, mô tả dataset, nhãn flow và PCAP. https://www.unb.ca/cic/datasets/ids-2017.html

[8] Emerging Threats. ET Open rules và hướng dẫn sử dụng, license theo bản phân phối và từng rule. https://rules.emergingthreats.net/open/ và https://community.emergingthreats.net/t/frequently-asked-questions/56

[9] Hadi, H. J., Yasmin, R. và Shoker, A. (2026). GenTI: Benchmarking LLMs for Autonomous IDPS Rule Generation for Unseen Attacks. arXiv:2606.05844. Bản gốc và bản dịch tiếng Việt do chủ đề tài cung cấp được dùng làm tài liệu phương pháp. https://arxiv.org/abs/2606.05844

[10] scikit-learn. GroupKFold. Tài liệu chia fold không giao nhau theo group. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GroupKFold.html. Đối chiếu ngày 09/11/2026.

[11] scikit-learn. Combine predictors using stacking. Cơ chế dùng dự đoán OOF để fit meta-model. https://scikit-learn.org/stable/auto_examples/ensemble/plot_stack_predictors.html. Đối chiếu ngày 09/11/2026.

[12] OWASP Foundation. OWASP Top 10. Tài liệu tổ chức các nhóm rủi ro web, không phải bộ chữ ký phát hiện. https://owasp.org/Top10/. Đối chiếu ngày 09/11/2026.

[13] Digininja và cộng đồng DVWA. Damn Vulnerable Web Application. Mã nguồn và tài liệu lab. https://github.com/digininja/DVWA. Đối chiếu ngày 09/11/2026.

[14] Alextrinity. SQLi XSS dataset, Kaggle release 2. Nguồn sơ cấp của bản phân phối và metadata Apache 2.0 do uploader khai báo; tác giả ghi dữ liệu chưa hoàn thiện. https://www.kaggle.com/datasets/alextrinity/sqli-xss-dataset. Truy cập 11/11/2026.

[15] Sid321axn (Kaggle uploader). Malicious URLs dataset, release 1. Metadata khai báo CC0; dữ liệu phân loại URL, không phải corpus sinh rule. https://www.kaggle.com/datasets/sid321axn/malicious-urls-dataset. Truy cập 11/11/2026.

[16] Qwen. Qwen2.5-Coder-1.5B-Instruct, model card và trọng số nền cho nhánh thực nghiệm cục bộ. https://huggingface.co/Qwen/Qwen2.5-Coder-1.5B-Instruct. Truy cập 11/11/2026.